

Zbigniew Michalewicz
David B. Fogel

How to Solve It: Modern Heuristics

Second Edition



Springer

How to Solve It: Modern Heuristics

Springer

Berlin

Heidelberg

New York

Hong Kong

London

Milan

Paris

Tokyo

Zbigniew Michalewicz • David B. Fogel

How to Solve It: Modern Heuristics

Second, Revised and Extended Edition

With 174 Figures and 7 Tables



Springer

Dr. Zbigniew Michalewicz
Department of Computer Science
University of North Carolina
9201 University City Boulevard
Charlotte, NC 28223-0001, USA
zbyszek@uncc.edu

Dr. David B. Fogel
Natural Selection, Inc.
Suite 200
3333 N. Torrey Pines Ct.
La Jolla, CA 92037, USA
dfogel@natural-selection.com

Artwork included in cover design by Ewa J. Michalewicz

Library of Congress Control Number: 2004094686

ACM Computing Classification (1998): I.2.8, F.2.2, K.3.2

ISBN 3-540-22494-7 Springer-Verlag Berlin Heidelberg New York
ISBN 3-540-66061-5 1st edition Springer-Verlag Berlin Heidelberg New York

This work is subject to copyright. All rights are reserved, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilm or in any other way, and storage in data banks. Duplication of this publication or parts thereof is permitted only under the provisions of the German Copyright Law of September 9, 1965, in its current version, and permission for use must always be obtained from Springer-Verlag. Violations are liable for prosecution under the German Copyright Law.

Springer is a part of Springer Science+Business Media
springeronline.com

© Springer-Verlag Berlin Heidelberg 2000, 2004
Printed in Germany

The use of general descriptive names, trademarks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

Cover Design: Künkel + Lopka, Werbeagentur, Heidelberg
Typesetting: Camera ready by the authors
Printed on acid-free paper 45/3142SR - 5 4 3 2 1

*To Ewa with love: the nicest 'puzzle'
I've been 'solving' for 25 years!*

Z.M.

*For my high school math teachers:
Mr. Flattum, Mr. McPhee,
Mr. Jackson, and Mr. Copperthwaite.*

D.B.F.

Preface to the Second Edition

No pleasure lasts long
unless there is variety in it.

Publilius Syrus, *Moral Sayings*

We've been very fortunate to receive fantastic feedback from our readers during the last four years, since the first edition of *How to Solve It: Modern Heuristics* was published in 1999. It's heartening to know that so many people appreciated the book and, even more importantly, were using the book to help them solve their problems. One professor, who published a review of the book, said that his students had given the best course reviews he'd seen in 15 years when using our text. There can be hardly any better praise, except to add that one of the book reviews published in a SIAM journal received the best review award as well. We greatly appreciate your kind words and personal comments that you sent, including the few cases where you found some typographical or other errors. Thank you all for this wonderful support.

One of the telltale signs that a book is having a significant impact is when it gets translated into another language. And we owe special thanks to Hongqing Cao, who prepared the Chinese edition of this book (and found a few remaining errors as well). An even more telling sign that a book is doing well is when it's translated into two languages, and we've learned recently that a Polish translation of the book is forthcoming. Still another good sign is when the publisher wants to publish a new edition of the book. Since the first printing, over 80 pages of the original edition have been replaced gradually by edited and corrected pages, so we hope this second edition will be error free!

You'll find that this second edition contains two new chapters in response to our readers' requests: one on coevolutionary systems and the other on multicriteria decision-making. Of course, in the spirit of our first edition, we've added some new puzzles for you as well. We hope you find these puzzles to be at the same challenging level as those from the first edition. You'll find a few other chapters have some changes, and a new section in Chap. 11.

We'd like to take this opportunity to thank everyone who took the time to share their thoughts on the text with us — they were most helpful. These thanks go to hundreds of readers who sent us e-mails commenting on different sections of the text, examples, references, and puzzles. We also express our gratitude to Rodney Johnson, Michael Melich, and Dave Schaffer for useful comments on parts of new chapters, Min Sun for her help in building some

new figures included in the text, Brian Mayoh and Antoni Mazurkiewicz, who pointed us to the “pirates puzzle” and the “bicycle puzzle,” respectively, both of which are placed in Chap. XIV. Special thanks are due to Carlos A. Coello Coello, who provided us with invaluable assistance and experimental results of a multiobjective optimization method on selected test cases. These are included in Sect. 15.2.2. We also thank Neville Hankins and Ronan Nugent for their assistance with writing style and Ingeborg Mayer (all from Springer-Verlag) for her help throughout the project.

As with the first edition, our purpose will be fulfilled if you find this book challenging, entertaining, and provocative.

Charlotte, NC
La Jolla, CA
April 2004

Zbigniew Michalewicz
David B. Fogel

Preface to the First Edition

‘I will tell you’
the hermit said to Lancelot
‘the right of the matter.’

Anonymous, *The Quest of the Holy Grail*

Gyorgy Polya’s *How to Solve It* [357] stands as one of the most important contributions to the problem-solving literature in the twentieth century. Even now, as we move into the new millennium, the book continues to be a favorite among teachers and students for its instructive heuristics. The first edition of the book appeared in 1945, near the end of the Second World War and a few years before the invention of the transistor. The book was a quick success, and a second edition came out in 1957.

How to Solve It is a compendium of approaches for tackling problems as we find them in mathematics. That is, the book provides not only examples of techniques and procedures, but also instruction on how to make analogies, use auxiliary devices, work backwards from the goal to the given, and so forth. Essentially, the book is an encyclopedia of problem-solving methods to be carried out by hand, but more than that, it is a treatise on how to think about framing and attacking problems.

Current texts in heuristic problem solving provide detailed descriptions of the algorithmic procedures for each of a number of classic algorithms. Sadly, however, they often fail to provide the proper guidance on when to apply these algorithms, and more importantly, when not to apply them. Instead, they rely on a cookbook of recipes and leave the responsibility of determining whether or not a particular method is appropriate for the task at hand to the reader. Most often, the reader is completely unprepared to make this determination, having never been taught the issues involved, nor even that there are indeed issues to consider!

This situation is undoubtedly a consequence of the computer revolution. If there were any single event that could necessitate an updated approach to problem solving nearly 50 years after Polya’s book, it would be the availability of powerful desktop computers at affordable prices. Solutions to real-world challenges are rarely computed with pencil and paper anymore. Instead, we use computer algorithms to numerically approximate and extend the range of questions for which we can generate useful answers. Because these computers are so effective, the problem solver often attempts to “hack” a solution, or at

least what masquerades as a solution, without giving due consideration to the assumptions that underlie the implemented method.

As a result, despite the enormous progress that has been made toward the optimization of performance in medicine, defense, industry, finance, and so forth, we have achieved little of our potential. For example, the amount of money saved by using the common methods of linear programming as opposed to hand calculation and guesswork must rise into literally billions of dollars annually, and yet linear programming is almost always used inappropriately in real-world conditions. Individuals and corporations look incessantly, almost blindly, for the quick fix in commercial-off-the-shelf (COTS) solutions to problems that don't exist. Imagine how much money could be saved (or earned!) if truly appropriate techniques were applied to problems that go beyond the simple heuristics of linear programming.

As the world moves toward more open and free markets, competition is the driving force that will mandate more effective methods for solving problems. The analogy between Darwinian variation-and-selection and free-market dynamics is apt. Those entities that fail to acquire the necessary resources will go bankrupt, the economic equivalent of death and survival of the fittest. All that's needed is a slight edge over a competitor to drive that competitor out of business. Only the enlightened individuals, corporations, and agencies that make the effort to adopt modern heuristics to solve their problems will endure.

What is needed now is a two-fold update to Polya's contribution. First, the reader must learn about the specific techniques that are available, mainly through the application of computer algorithms. Second, the reader must come to understand when each of these methods should and should not be used, and how to frame their own problems so that they can best apply the heuristics that Polya's *How to Solve It* could not have anticipated.

This book is a first attempt toward a comprehensive view of problem solving for the twenty-first century. The main points are communicated by direct instruction, analogy, example, and through a series of problems and puzzles that are integrated throughout the discourse and inbetween chapters. The text is intended to serve as the main text for a class on modern heuristics. We believe that such a course is a *must* for students in science, business, or engineering. As such, it assumes some basic knowledge of discrete mathematics and a familiarity with computer programming. Problem solvers who do not have these basic skills should invest the time to acquire them, for it will be well worth the effort. Those who want more in-depth mathematical treatment of algorithms will find the material in the text to be a useful stepping stone to more advanced courses.

The book is organized into 15 chapters which begin after we present a general discussion of solving problems in the Introduction. Chapter 1 indicates the main sources of difficulty in problem solving and a short chapter 2 provides basic notation. Chapters 3 and 4 survey classical optimization algorithms. Together with chapter 5, which covers two modern search algorithms, these chapters constitute the first part of the book. We then move to an evolutionary approach to problem solving. Chapters 6 and 7 introduce intuitions and

details about these new evolutionary algorithms. Chapters 8–10 then address some challenging issues concerning problems that require searching for a particular permutation of items, handling constraints, and tuning the algorithms to the problem. These chapters provide a detailed review of many efforts in these areas. Chapter 11 addresses the issue of time-varying environments and noise. Two additional chapters (12 and 13) provide tutorials for neural networks and fuzzy systems. Chapter 14 provides a short and general discussion on hybrid systems and extensions to evolutionary algorithms. Chapter 15 concludes the text by summarizing the material and providing some hints for practical problem solving. You'll note that each puzzle section (I, II, etc.) illustrates a point that is made in the subsequent chapter (1, 2, etc.) — problem solving should be fun and we hope this approach will be enjoyable and engaging.

In addition to the material in the main part of the text, two appendices offer supplemental information. Appendix A provides an overview of fundamental concepts in probability and statistics. It starts with the axioms of probability and proceeds to statistical hypothesis testing and linear regression. This appendix is not intended as a substitute for a thorough preparation in random processes, but can serve to refresh recollections and illuminate important issues in applying probability and statistics to problem solving. Appendix B offers a list of suggested problems and projects to tackle when the text is used as part of a university course. Even if you are reading the book without the benefit of a primary teacher, we strongly encourage you to take an active role in problem solving and implement, test, and apply the concepts that you learn here. The material in Appendix B can serve as a guide for such applications.

We thank everyone who took the time to share their thoughts on the text with us — they were most helpful. In particular, we express our gratitude to Dan Ashlock, Ole Caprani, Tom English, Larry Fogel, Larry Hall, Jim Keller, Kenneth Kreutz-Delgado, Martin Schmidt, and Thomas Stidsen. We also would like to express special appreciation to Kumar Chellapilla, who not only reviewed sections of the book but also provided invaluable assistance with the graphics for some figures. We are pleased to acknowledge the assistance of several co-authors who worked with us during the last two years; many results of this collaboration were included in this text. We thank Thomas Bäck, Hongqing Cao, Ole Caprani, Dipankar Dasgupta, Kalyan Deb, Gusz Eiben, Susana Esquivel, Raul Gallard, Özdemir Göl, Robert Hinterding, Sławomir Koziel, Lishan Kang, Moutaz Khouja, Witold Kosiński, Thiemo Krink, Guillermo Leguizamón, Brian Mayoh, Maciej Michalewicz, Guo Tao, Krzysztof Trojanowski, Marc Schoenauer, Martin Schmidt, Thomas Stidsen, Roman Śmierzchalski, Martyna Weigl, Janek Wieczorek, Jing Xiao, and Lixin Zhang.

We thank the executive editor of Springer-Verlag, Hans Wössner, for his help throughout the project, Professor Leonard Bolc, for his encouragement, and Professor Antoni Mazurkiewicz for interesting discussions on many puzzles presented in this text. Special thanks are due to Joylene Vette, the English copy editor, Springer-Verlag, for her precious comments on the first draft of the text. The first author would also like to acknowledge the excellent working environ-

ments at Aarhus University where he spent his sabbatical leave (August 1998 — July 1999) and grants from the National Science Foundation (IRI-9322400 and IRI-9725424) and from the ESPRIT Project 20288 Cooperation Research in Information Technology (CRIT-2): “Evolutionary Real-time Optimization System for Ecological Power Control,” which helped in preparing a few chapters of this text. Also, he would like to thank all of the graduate students from UNC-Charlotte (USA), Universidad Nacional de San Luis (Argentina), and Aarhus University (Denmark) who took part in various courses offered from 1997 to 1999 and went through a painful process of problem solving. The second author would like to thank the undergraduate students of a course he taught on machine learning and pattern recognition at UC San Diego in the winter quarter of 1999, as well as his fellow employees at Natural Selection, Bill Porto, Pete Angeline, and Gary Fogel, for their support and encouragement. Special thanks go to Jacquelyn Moore for giving up so many weekends and evenings so that this book could be completed.

Our purpose will be fulfilled if you find this book challenging, entertaining, and provocative. We hope that you enjoy the book at least as much as we enjoyed assembling it, and that you profit from it as well.

Charlotte, NC
La Jolla, CA
September 1999

Zbigniew Michalewicz
David B. Fogel

Table of Contents

Introduction	1
I What Are the Ages of My Three Sons?	9
1 Why Are Some Problems Difficult to Solve?	11
1.1 The size of the search space	11
1.2 Modeling the problem	16
1.3 Change over time	19
1.4 Constraints	21
1.5 The problem of proving things	24
1.6 Your chance for glory	26
1.7 Summary	29
II How Important Is a Model?	31
2 Basic Concepts	35
2.1 Representation	35
2.2 The objective	36
2.3 The evaluation function	37
2.4 Defining a search problem	39
2.5 Neighborhoods and local optima	40
2.6 Hill-climbing methods	43
2.7 Can you sink this trick shot?	45
2.8 Summary	48
III What Are the Prices in 7–11?	49
3 Traditional Methods — Part 1	55
3.1 Exhaustive search	58
3.1.1 Enumerating the SAT	59
3.1.2 Enumerating the TSP	61
3.1.3 Enumerating the NLP	62
3.2 Local search	64
3.2.1 Local search and the SAT	65
3.2.2 Local search and the TSP	66

XIV Table of Contents

3.2.3	Local search and the NLP	69
3.3	Linear programming: The simplex method	76
3.4	Summary	80
IV	What Are the Numbers?	83
4	Traditional Methods — Part 2	87
4.1	Greedy algorithms	87
4.1.1	Greedy algorithms and the SAT	87
4.1.2	Greedy algorithms and the TSP	89
4.1.3	Greedy algorithms and the NLP	90
4.2	Divide and conquer	90
4.3	Dynamic programming	93
4.4	Branch and bound	101
4.5	A* algorithm	105
4.6	Summary	109
V	What's the Color of the Bear?	111
5	Escaping Local Optima	115
5.1	Simulated annealing	117
5.2	Tabu search	125
5.3	Summary	134
VI	How Good Is Your Intuition?	135
6	An Evolutionary Approach	139
6.1	Evolutionary approach for the SAT	142
6.2	Evolutionary approach for the TSP	145
6.3	Evolutionary approach for the NLP	149
6.4	Summary	151
VII	One of These Things Is Not Like the Others	157
7	Designing Evolutionary Algorithms	161
7.1	Representation	165
7.1.1	Fixed-length vectors of symbols	166
7.1.2	Permutations	167
7.1.3	Finite state machines	168
7.1.4	Symbolic expressions	168
7.2	Evaluation function	169
7.3	Variation operators	172
7.3.1	Fixed-length vectors of symbols	173
7.3.2	Permutations	174
7.3.3	Finite state machines	176
7.3.4	Symbolic expressions	177
7.4	Selection	179

7.5	Initialization	181
7.6	Summary	183
VIII	What Is the Shortest Way?	185
8	The Traveling Salesman Problem	189
8.1	In search of good variation operators	191
8.2	Incorporating local search methods	214
8.3	Other possibilities	217
8.3.1	Edge assembly crossover	217
8.3.2	Inver-over operator	220
8.4	Summary	223
IX	Who Owns the Zebra?	225
9	Constraint-Handling Techniques	231
9.1	General considerations	232
9.1.1	Designing $eval_f$	234
9.1.2	Design of $eval_u$	236
9.1.3	Relationship between $eval_f$ and $eval_u$	237
9.1.4	Rejecting infeasible solutions	239
9.1.5	Repairing infeasible individuals	239
9.1.6	Replacing individuals by their repaired versions	240
9.1.7	Penalizing infeasible individuals	240
9.1.8	Maintaining a feasible population using special representations and variation operators	241
9.1.9	Using decoders	242
9.1.10	Separating individuals and constraints	243
9.1.11	Exploring boundaries between feasible and infeasible parts of the search space	243
9.1.12	Finding feasible solutions	244
9.2	Numerical optimization	245
9.2.1	Methods based on preserving the feasibility of solutions	246
9.2.2	Methods based on penalty functions	249
9.2.3	Methods based on a search for feasible solutions	257
9.2.4	Methods based on decoders	264
9.2.5	Hybrid methods	266
9.3	Summary	268
X	Can You Tune to the Problem?	271
10	Tuning the Algorithm to the Problem	277
10.1	Parameter control in evolutionary algorithms	277
10.2	Illustrating the case with an NLP	282
10.3	Taxonomy of control techniques	284
10.4	The possibilities for parameter control	288

XVI Table of Contents

10.4.1	Representation	288
10.4.2	Evaluation function	289
10.4.3	Mutation operators and their probabilities	290
10.4.4	Crossover operators and their probabilities	293
10.4.5	Parent selection	295
10.4.6	Population	295
10.5	Combining forms of parameter control	296
10.6	Summary	298
XI	Can You Mate in Two Moves?	303
11	Time-Varying Environments and Noise	307
11.1	Life presents a dynamic landscape	307
11.2	The real world is noisy	317
11.2.1	Ensuring diversity	321
11.3	Modeling of ship trajectory	325
11.4	Summary	329
XII	Day of the Week of January 1st	335
12	Neural Networks	339
12.1	Threshold neurons and linear discriminant functions	340
12.2	Back propagation for feed forward multilayer perceptrons	345
12.3	Training and testing	350
12.4	Recurrent networks and extended architectures	352
12.4.1	Standard recurrent network	352
12.4.2	Hopfield network	352
12.4.3	Boltzmann machine	354
12.4.4	Network of multiple interacting programs	354
12.5	Clustering with competitive networks	356
12.6	Using neural networks to solve the TSP	358
12.7	Evolving neural networks	360
12.8	Summary	361
XIII	What Was the Length of the Rope?	363
13	Fuzzy Systems	367
13.1	Fuzzy sets	368
13.2	Fuzzy sets and probability measures	368
13.3	Fuzzy set operations	369
13.4	Fuzzy relationships	372
13.5	Designing a fuzzy controller	375
13.6	Fuzzy clustering	380
13.7	Fuzzy neural networks	382
13.8	A fuzzy TSP	385
13.9	Evolving fuzzy systems	387

13.10 Summary	388
XIV Everything Depends on Something Else	389
14 Coevolutionary Systems	397
14.1 Playing a game	399
14.2 Learning to play checkers	400
14.3 Optimization through competing populations	403
14.4 Another example with a game	408
14.5 Artificial life	411
14.6 Games of cooperation and competition	412
14.7 Modeling intelligent agents	418
14.8 Issues in coevolution	420
14.9 An example of cooperative coevolution on the TSP	424
14.10 Summary	430
XV Who's Taller?	431
15 Multicriteria Decision-Making	435
15.1 Single numeric approach	440
15.1.1 The Valuated State Space approach	443
15.1.2 Fuzzy multicriteria decision-making	449
15.1.3 Summary	450
15.2 Evolutionary multiobjective optimization approach	451
15.2.1 Brief history	452
15.2.2 Evolutionary algorithm for multiobjective NLP	458
15.3 Summary	462
XVI Do You Like Simple Solutions?	465
16 Hybrid Systems	471
16.1 Summary	480
17 Summary	483
Appendix A: Probability and Statistics	495
A.1 Basic concepts of probability	495
A.2 Random variables	497
A.2.1 Discrete random variables	497
A.2.2 Continuous random variables	500
A.3 Descriptive statistics of random variables	502
A.4 Limit theorems and inequalities	505
A.5 Adding random variables	506
A.6 Generating random numbers on a computer	507
A.7 Estimation	508
A.8 Statistical hypothesis testing	511
A.9 Linear regression	512

XVIII Table of Contents

A.10 Summary	514
Appendix B: Problems and Projects	515
B.1 Trying some practical problems	517
B.2 Reporting computational experiments with heuristic methods . .	522
References	525
Index	551

Introduction

The reasonable man adapts
himself to the world;
the unreasonable one persists
in trying to adapt the world to himself.
Therefore all progress depends
on the unreasonable man.

George Bernard Shaw, *Maxims for Revolutionists*

This is not a book about algorithms. Certainly, it is full of algorithms, but that's not what this book is about. This book is about possibilities. Its purpose is to present you not only with the prerequisite mandatory knowledge of the available problem-solving techniques, but more importantly to expand your ability to frame new problems and to think creatively — in essence, to solve the problem of how to solve problems, a talent that has become a lost art. Instead of devoting the necessary time and critical thinking required to frame a problem, to adjust our representation of the pieces of the puzzle, we have become complacent and simply reach for the most convenient subroutine, a magic pill to cure our ills. The trouble with magic is that, empirically, it has a very low success rate, and often relies on external devices such as mirrors and smoke. As with magic, most of the seemingly successful applications of problem solving in the real world are illusory, mere specters of what could have been achieved.

The importance of effective problem solving has never been greater. Technology has enabled us with the ability to affect the environment to such an extent that the decisions we make today may have irrevocable future consequences. This same technology continues to expand the number of people we interact with and who affect our lives. By consequence, problem solving grows increasingly more difficult because there are more factors to consider — and the potentially worst course of action is to ignore these factors, hoping that a solution found for a “simpler problem” will be effective in spite of these other concerns. Whether you are determining the proper operation of your business, the proper use of environmental resources for your country, or merely the best route to get to work in the morning, you simply cannot afford to assume away your interaction with the rest of world. As these interactions become ever more frequent and complex, procedures for handling real-world problems become imperative. There is a great deal to be gained from solving problems, and a great deal to be lost in solving them poorly.

We all face problems. A problem exists when there is a recognized disparity between the present and desired state. Solutions, in turn, are ways of allocating

the available resources so as to reduce the disparity between the present and desired state. To understand the situation, we must first recognize that problems are the possessions of purpose-driven decision makers. If you don't have a purpose then you don't have a problem. The trouble is, not only does each of us have a purpose, but often times those purposes seem to be at odds, or at least not complementary. In determining how best to solve your problem, you must consider it not just in isolation but rather in the context of how others' actions affect your possible solution. It's likely that if they interact with you, they will change their allocation of resources — sometimes to assist you, other times to stand in your way. Implementing a solution without considering “what comes next” is much like playing poker as if it were a one-player game: you and your money would soon be parted.

A prerequisite to handling the real world as a multiplayer game is the ability to manipulate an arsenal of problem-solving techniques, namely, algorithms that have been developed for a variety of conditions. Unfortunately, it's almost always the case that the real world presents us with circumstances that are slightly or considerably different than are required by these methods. For example, one particular technique is available to calculate the minimum-cost allocation of resources for a problem where both the cost function and constraints (equality and inequality) are linear. The method is fast and reliable. And it is almost always applied *inappropriately* in real-world settings where the cost function and constraints are almost always nonlinear. In essence, anyone using this method runs the risk of generating the right answer to a problem that does not exist. The loss incurred using this solution is multifold: not only could a better solution be obtained by properly handling the available constraints in light of the real cost function, but a competitor who treats the situation appropriately may in fact find this better solution! It only takes a slight advantage to one side to bankrupt the other. (Las Vegas has operated on this principle for many years.) On the other hand, using a fast but approximate solution may in fact be the right thing to do: If a competitor is using a slow procedure to find the exact solution, you may be able to discover some inexact but useful solution before they do, and sometimes having any solution at all is better than having to wait for one that is superior. The proper manner in which to proceed requires judgment.

Regrettably, judgment seems to be waning, or at least is out of vogue. Perhaps this is because it requires hard work. In order to make proper judgments in problem solving you have to know what each particular problem-solving method assumes, its computational requirements, its reliability, and so forth. Gaining this information requires time and effort. It's easier to simply peruse the Internet or the commercially available software and obtain something that seems like it might (sort of) fit the problems at hand. Indeed, some businesses and agencies mandate the purchase of such commercial-off-the-shelf (COTS) software. Certainly, this often costs less at the initial outlay, but it's frequently penny wise and pound foolish. Years later, when the software has failed to address the business's real needs and when other competitors have brought their solutions forward in less time, the purchased software sits in an archive,

unused, while the workers go back to solving the original problem, sometimes by hand!

Rather than lament this situation, it should be recognized as an opportunity. A great reward awaits those who can effectively solve problems, and you can develop this talent. It begins with a proper understanding of the purpose to be achieved. All too often, in an attempt to utilize a familiar algorithm, we change the purpose to be achieved simply to fit the algorithm. It's as if, armed with only a hammer, we needed to screw a screw into a board: we pound the screw into the board even though we know we'll never be able to unscrew it again. Armed with just a hammer, everything looks like a nail. Similarly, armed with only a small subset of classic problem-solving techniques, all cost functions are magically transformed into smooth, linear, convex mappings, and every model is optimized in light of the mean squared error. This results in "perfect" answers that have questionable relevance. Understanding when the decision maker's purpose should or shouldn't be approximated by linearization and other simplifying techniques is of paramount importance. Formalizing subjective purpose in objective measurable terms so that you can identify important differences between approximations and reality is a challenging task that requires practice. There's no substitute for this experience.

Having made a judgment as to the degree of fidelity required in modeling the decision maker's purpose, the next step is to choose a tactic that will find an optimal allocation of the available resources. For this task, you have lots of options. There are many books devoted to algorithms for problem solving and each presents an array of individual techniques. It's tempting to simply look up an algorithm as you would look up a recipe for a cake. But this natural tendency foregoes creativity; it doesn't afford you the possibility of being ingenious. Instead, it forces you to fit the problem into the constraints of a particular algorithm. In the rush to present problem-solving techniques, each is typically offered independently, thereby overlooking potentially important synergies between different procedures. It's as if a complex jigsaw puzzle could be completed without considering how the pieces fit together. Better solutions to real-world problems can often be obtained by usefully hybridizing different approaches. Effective problem solving requires more than a knowledge of algorithms; it requires a devotion to determining the best combination of approaches that addresses the purpose to be achieved within the available time and in light of the expected reactions of others who are playing the same game. Recognizing the complexity of real-world problems is prerequisite to their effective solution.

Unfortunately, our troubles begin early on in life, even as early as elementary school. We're taught to decompose problems and treat the smaller simpler problems individually. This is wonderful when it works. But complex real-world problems don't often decompose easily or meaningfully. What might even be worse, we are spoon fed the solutions to problems, chapter by chapter, never once being forced to think about whether or not the problems we are facing should be solved with the technique just described in our textbook. Of course they must! *Why else would this problem be in this chapter?!* This is the case not

just with texts for elementary mathematics for primary and secondary schools, but also with most textbooks for university courses, and even many monographs that explain particular techniques and illustrate their application with various examples. The problem and its solution in these books are never far apart.

Let's consider a mathematics textbook for junior high school. There might be a chapter there on solving quadratic equations. At the end of the chapter there are some problems to solve. One of them reads: A farmer has a rectangular ranch with a perimeter of 110 meters and an area of 700 square meters. What are the dimensions of his ranch? The student *knows* that to solve the problem, he should use something that was discussed in the same chapter. So he tries to construct a quadratic equation. Sooner or later he comes up with two equations

$$\begin{cases} 2x + 2y = 110 \\ xy = 700, \end{cases}$$

that lead to a quadratic equation:

$$x(55 - x) = 700.$$

Then, following some given formulas, he easily calculates x , and solves the problem.

Some other chapter is on geometry and discusses properties of triangles. The Pythagorean theorem is explained, and the chapter concludes with yet another series of problems to solve. The student has no doubt that he should apply this theorem to solve these problems.

It seems, however, that this is not the right way to teach. The relationship between a problem and a method should be discussed from the perspective of a problem, and not the method. Doing otherwise actually causes more harm than good in the long run, because like all "hand outs," it leaves the students incapable of fending (i.e., thinking) for themselves.

Let's make this point by presenting two problems to solve. The first is as follows. There is a triangle ABC , and D is an arbitrary interior point of this triangle (see figure 0.1). All you have to do is prove that

$$AD + DB < AC + CB,$$

where AD denotes the distance between points A and D , and the other terms indicate the analogous distances between their representative vertices.

This problem is *so* easy, it seems that there is nothing to prove. It's so obvious that the sum of the two segments inside the triangle *must* be shorter than the sum of its two sides! But this problem now is removed from the context of its "chapter" and outside of this context, the student has no idea of whether to apply the Pythagorean theorem, build a quadratic equation, or do something else!

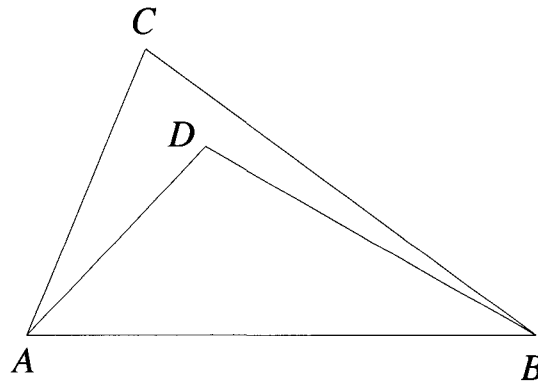


Fig. 0.1. Triangle ABC with an interior point D

The issue is more serious than it first appears. We have given this very problem to many people, including undergraduate and graduate students, and even full professors in mathematics, engineering, or computer science. Fewer than five percent of them solved this problem within an hour, many of them required several hours, and we witnessed some failures as well. What's interesting here is the fact that this problem was selected from a math text for fifth graders in the United States. It's easy to solve when placed at the end of the appropriate chapter! But outside of this context, it can be quite difficult.

At this stage we'd like to encourage you to put the book aside and try to solve this problem. To reduce your temptation to simply "keep reading" without *thinking*, we placed the solution to this problem in the final section of this text. If you can't solve it within an hour, this book is probably for you!¹

Let's consider another puzzle. There is a well, open at the top, with a diameter of three meters. We throw two sticks with lengths of four and five meters, respectively, into the well. Let's assume that these sticks are positioned as shown in figure 0.2, i.e., they stay in the same plane and they "cross each other" as shown. The problem is really two-dimensional, as the well can be replaced by a rectangle that is open at the top.

Now, the challenge is to determine the distance h from the bottom of the well to the point where the two sticks intersect. The position and the lengths of the sticks guarantee a unique solution, and it seems that the problem is easy. Indeed, it is easy; it takes less than a minute to solve and you shouldn't be surprised to learn that the problem is again taken from a primary school curriculum. However, the associated "chapter contents" for this problem are once again missing, and once more we encourage you, the reader, to solve it. If you solve it within an hour (!), you'll belong to the elite one percent of the people we tested who managed to get the right answer within that time. What's more, everyone we tested had at least an undergraduate degree either in mathematics,

¹Note, however, that we already gave you a valuable piece of information: the problem is elementary, so there's no need to use any "sophisticated" tools. Without this hint it would be more difficult to solve. And you'll probably benefit from this book even if you can solve this problem within an hour!

engineering, or computer science. As with the first problem, the solution is given in the final section of this text.

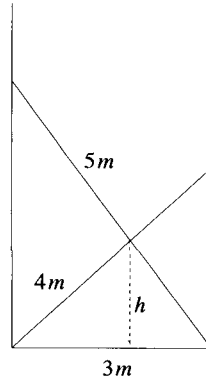


Fig. 0.2. Two sticks in a well

Our main point here is that we learn how to apply particular methods to particular problems, but only within the context of knowing that these exact methods ought to be perfect for these particular problems. We almost never learn how to think about solving problems in general — we're constrained to concentrate on each question in the back of each chapter using the information learned in that chapter. Hopefully, this will change. We look forward to the day when some problems in the math text for our first graders will be different. For example, when the problem

It takes 48 hours for a rocket to travel from the Earth to the Moon.
How long will this trip take if a new rocket is twice as fast?

is replaced by

It takes 48 hours for a rocket to travel from the Earth to the Moon.
How long will this trip take for two rockets?

which forces a child (or even an adult) to *think* (e.g., whether to multiply or divide 48 by 2, or whether it still take 48 hours). Even in higher grades there are some possibilities:

A coin is flipped. It comes up heads. What is your guess for the next toss?

What is missing in most curriculums — on all levels, starting from primary schools through university education — is a development of problem-solving skills. Young people often have serious difficulties in solving any real problem because real problems aren't found in any chapter! This book is designed not only to present you with an array of algorithms and techniques for solving textbook problems, but it's also designed to get you to *think* about how to

frame and solve problems, particularly those that you don't encounter at the end of some chapter in a book.

We humans like to tout our ability to solve problems. We have even gone so far as to define ourselves in terms of sapience: wisdom, enlightenment, genius. It would be an easy conclusion that such a wise and enlightened genius should be able to solve its problems. And yet, the world is full of people with problems. We hope this book helps you solve not only your own problems, but others' problems as well.

I. What Are the Ages of My Three Sons?

One of the reasons that problem solving is often difficult is that we don't know where to begin. Once we have some sort of course charted out in front of us, we can follow that path and hopefully arrive at a solution. But conjuring up that course to follow is a significant challenge. Often this is the most challenging part of problem solving because it is entirely creative: you have to literally *create* the plan for generating a solution. Without the plan, you're sunk. This is the time where you should revel in the opportunity to display your brilliance! One thing that may help in forging a path from the problem to the solution is to always consider all of the available data. This is good advice in any regard, but it's especially important when first considering how to find a solution. Failing to consider everything right at the start means that you might miss the one opportunity to begin the solution. Review all of the information that's provided, determine the implications of these data or facts, and then see if you can make a connection between the goal and what you are given.

Give it a try with the following problem. Two men meet on the street. They haven't seen each other for many years. They talk about various things, and then after some time one of them says: "Since you're a professor in mathematics, I'd like to give you a problem to solve. You know, today's a very special day for me: All three of my sons celebrate their birthday this very day! So, can you tell me how old each of them is?"

"Sure," answers the mathematician, "but you'll have to tell me something about them."

"OK, I'll give you some hints," replies the father of the three sons, "The product of the ages of my sons is 36."

"That's fine," says the mathematician, "but I'll need more than just this."

"The sum of their ages is equal to the number of windows in that building," says the father pointing at a structure next to them.

The mathematician thinks for some time and replies, "Still, I need an additional hint to solve your puzzle."

"My oldest son has blue eyes," says the other man.

"Oh, this is sufficient!" exclaims the mathematician, and he gives the father the correct answer: the ages of his three sons.

Your challenge now is to do the same: to follow the reasoning of the mathematician and solve the puzzle. Again, the puzzle is quite easy, yet most people have difficulties with it.

How did you do? If you didn't resist the temptation to simply flip the page and find the answer, do yourself a favor and turn back now before it's too late!

All right, to start let's examine all of the information supplied by the conversation carefully. What do we know on the basis of the first piece of information? If the product of the ages of the three sons is 36, there are only eight possibilities to consider; this reduces the search space to only eight cases:

Age of son 1	Age of son 2	Age of son 3
36	1	1
18	2	1
12	3	1
9	4	1
9	2	2
6	6	1
6	3	2
4	3	3

The second piece of information was that the sum of the sons' ages is the same as the number of windows in the building. We have to assume that the mathematician knew the number of windows, so he knew the total. What are the possibilities here? How can this be useful? Adding the numbers for all eight cases yields the following totals:

36	+	1	+	1	=	38
18	+	2	+	1	=	21
12	+	3	+	1	=	16
9	+	4	+	1	=	14
9	+	2	+	2	=	13
6	+	6	+	1	=	13
6	+	3	+	2	=	11
4	+	3	+	3	=	10

Suddenly, everything is clear. If the number of windows in the building had been 21 (or 38, 16, 14, 11, or 10), the mathematician would have given the answer immediately. Instead, he said that he still needed an additional piece of information. This indicates that the number of windows was 13, thereby leaving two and only two options:

(9, 2, 2) or (6, 6, 1).

As the second option does not have an *oldest* son ("the *oldest* son has blue eyes"), the ages of the three sons must be (9, 2, 2).

What makes this problem easier than it might otherwise be is that the order in which you should consider the information in the problem is the same as the order in which it's presented. It's almost as if the authors of the puzzle wanted you to find the right answer! Real-world problems aren't so neatly organized. Nevertheless, if you consider all of the available data then you'll give yourself every opportunity to find a useful starting point.

1. Why Are Some Problems Difficult to Solve?

Our problems are man-made;
therefore they may be solved by man.

John F. Kennedy, speech, June 10, 1963

At the risk of starting with a tautology, real-world problems are difficult to solve, and they are difficult for several reasons:

- The number of possible solutions in the *search space* is so large as to forbid an exhaustive search for the best answer.
- The problem is so complicated that just to facilitate any answer at all, we have to use such simplified models of the problem that any result is essentially useless.
- The *evaluation function* that describes the quality of any proposed solution is noisy or varies with time, thereby requiring not just a single solution but an entire series of solutions.
- The possible solutions are so heavily constrained that constructing even one feasible answer is difficult, let alone searching for an optimum solution.
- The person solving the problem is inadequately prepared or imagines some psychological barrier that prevents them from discovering a solution.

Naturally, this list could be extended to include many other possible obstacles. For example, we could include noise associated with our observations and measurements, uncertainty about given information, and the difficulties posed by problems that have multiple and possibly conflicting objectives (which may require a set of solutions rather than a single solution). The above list, however, is sufficient for now. Each of these are problems in their own right. To solve a problem, we have to understand the problem, so let's discuss each of these issues in turn and identify their inherent details.

1.1 The size of the search space

One of the elementary problems in logic is the Boolean satisfiability problem (SAT). The task is to make a compound statement of Boolean variables evaluate to TRUE. For example, consider the following problem of 100 variables given in conjunctive normal form:

$$F(\mathbf{x}) = (x_{17} \vee \bar{x}_{37} \vee x_{73}) \wedge (\bar{x}_{11} \vee \bar{x}_{56}) \wedge \dots \wedge (x_2 \vee x_{43} \vee \bar{x}_{77} \vee \bar{x}_{89} \vee \bar{x}_{97}).$$

The challenge is to find the truth assignment for each variable x_i , for all $i = 1, \dots, 100$ such that $F(\mathbf{x}) = \text{TRUE}$. We can use 1 and 0 as synonyms for TRUE and FALSE, and note that $\bar{x}_i$ here is the negation of x_i (i.e., if x_i were TRUE or 1, then $\bar{x}_i$ would be FALSE or 0).

Regardless of the problem being posed, it's always useful to consider the space of possible solutions. Here, any binary string of length 100 constitutes a potential solution to the problem. We have two choices for each variable, and taken over 100 variables, this generates 2^{100} possibilities. Thus the size of the search space $\mathcal{S}$ is $|\mathcal{S}| = 2^{100} \approx 10^{30}$. This is a huge number! Trying out all of these alternatives is out of the question. If we had a computer that could test 1000 strings per second and could have started using this computer at the beginning of time itself, 15 billion years ago right at the Big Bang, we'd have examined fewer than one percent of all the possibilities by now!

What's more, the choice of which evaluation function to use isn't very clear. What we'd like is for the evaluation function to give us some guidance on the quality of the proposed solution. Solutions that are closer to the right answer should yield better evaluations than those that are farther away. But here, all we have to operate on is $F(\mathbf{x})$ which can either evaluate to TRUE or FALSE. If we try out a string $\mathbf{x}$ and $F(\mathbf{x})$ returns TRUE then we're done: that's the answer. But what if $F(\mathbf{x})$ returns FALSE? Then what? Furthermore, almost every possible string of 0s and 1s that we could try would likely evaluate to FALSE, so how could we distinguish between "better" and "worse" potential solutions? If we were using an enumerative search we wouldn't care because we'd simply proceed through each possibility until we found something interesting. But if we want the evaluation function to help us find the best solutions faster than enumeration, we need more than just "right" or "wrong." The way we could accomplish that for the SAT problem isn't clear immediately.

Some problems seem easier than SAT problems because they suggest a possible evaluation function naturally. Even so, the size of the search space can still be enormous. For example, consider a traveling salesman problem (TSP). Conceptually, it's very simple: the traveling salesman must visit every city in his territory exactly once and then return home covering the shortest distance. Some closely related problems require slightly different criteria, such as finding a tour of the cities that yields minimum traveling time, or minimum fuel cost, or a number of other possibilities, but the idea is the same. Given the cost of traveling between each pair of cities, how should the salesman plan his itinerary to achieve a minimum cost tour?

Figure 1.1 illustrates a simple symmetric 20-city TSP where the distance between each pair of cities i and j is the same in either direction. That is, $\text{dist}(i, j) = \text{dist}(j, i)$. The actual distances aren't marked on the figure, but we could assume this is the case. Alternatively, we could face an asymmetric TSP where $\text{dist}(i, j) \neq \text{dist}(j, i)$ for some i and j . These two classes of TSP present different obstacles for finding minimum cost paths.

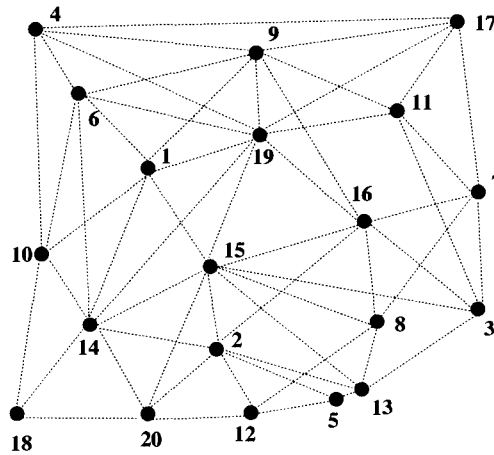


Fig. 1.1.1. A sample TSP. Textbook TSPs usually allow paths from every city to every other cities, but real-world problems don't always afford such opportunities.

So then, what is the search space for the TSP? One possibility would be to view it as the set of permutations of n cities. Any permutation of n cities yields an ordered list that defines the sequence of cities to be visited, starting at the salesman's home base, and continuing to the last location before returning home. The optimum solution is a permutation that yields the minimum cost tour. Note that tours such as:

2 - ... - 6 - 15 - 3 - 11 - 19 - 17,
 15 - 3 - 11 - 19 - 17 - 2 - ... - 6,
 3 - 11 - 19 - 17 - 2 - ... - 6 - 15, etc.

are identical because the circuit that each one generates is exactly the same regardless of the starting city, and there are n tours like that for any n -city TSP. It's easy to see then that every tour can be represented in $2n$ different ways (for a symmetrical TSP). And since there are $n!$ ways to permute n numbers, the size of the search space is then $|\mathcal{S}| = n!/(2n) = (n-1)!/2$.

Again, this is a huge number! For any $n > 6$, the number of possible solutions to the TSP with n cities is larger than the number of possible solutions to the SAT problem with n variables. Furthermore, the difference between the sizes of these two search spaces increases very quickly with increasing n . For $n = 6$, there are $5!/2 = 60$ different solutions to the TSP and $2^6 = 64$ solutions to a SAT. But for $n = 7$ these numbers are 360 and 128, respectively.

To see the maddening rate of growth of $(n-1)!/2$, consider the following numbers:

- A 10-city TSP has about 181,000 possible solutions.

- A 20-city TSP has about 10,000,000,000,000 possible solutions.
- A 50-city TSP has about 100,000,000,000,000,000,000,000,000,000,000,000,000,000 possible solutions.

There are only 1,000,000,000,000,000,000 liters of water on the planet, so a 50-city TSP has an unimaginably large search space. Literally, it's so large that as humans, we simply can't conceive of sets with this many elements.

Even though the TSP has an incredibly large search space, the evaluation function that we might use to assess the quality of any particular tour of cities is much more straightforward than what we saw for the SAT. Here, we can refer to a table that would indicate all of the distances between each pair of cities, and after n addition operations we could calculate the distance of any candidate tour and use this to evaluate its merit. For example, the cost of the tour

$$15 - 3 - 11 - 19 - 17 - 2 - \dots - 6$$

is

$$\text{cost} = \text{dist}(15, 3) + \text{dist}(3, 11) + \text{dist}(11, 19) + \dots + \text{dist}(6, 15).$$

We might hope that this, more natural, evaluation function would give us an edge in finding useful solutions to the TSP despite the size of the search space.

Let's consider a third example — a particular nonlinear programming problem (NLP). It's a difficult problem that has been studied in the scientific literature and no traditional optimization method has given a satisfactory result. The problem [254] is to maximize the function:¹

$$G2(\mathbf{x}) = \left| \frac{\sum_{i=1}^n \cos^4(x_i) - 2 \prod_{i=1}^n \cos^2(x_i)}{\sqrt{\sum_{i=1}^n ix_i^2}} \right|,$$

subject to

$$\prod_{i=1}^n x_i \geq 0.75, \quad \sum_{i=1}^n x_i \leq 7.5n, \quad \text{and bounds } 0 \leq x_i \leq 10 \text{ for } 1 \leq i \leq n.$$

The function $G2$ is nonlinear and its global maximum is unknown, but it lies somewhere near the origin. The optimization problem poses one nonlinear constraint and one linear constraint (the latter one is inactive near the origin).

What's the size of the search space now? In a sense, it depends on the dimensionality of the problem — the number of variables. When treated as a purely mathematical problem, with n variables, each dimension can contain an infinity of possible values, so we have an infinitely large space — maybe even several degrees of infinity. But on a computer, everything is digital and finite, so if we were going to implement some sort of algorithm to find the optimum of $G2$, we'd have to consider the available computing precision. If our precision guaranteed six decimal places, each variable could then take on

¹We're preserving the notation $G2$ used for this function in [319]. It's the second function in an 11-function testbed for nonlinear programming problems.

10,000,000 different values. Thus, the size of the search space would be $|\mathcal{S}| = 10,000,000^n = 10^{7n}$. That number is much much larger than the number of solutions for the TSP. Even for $n = 50$ there are 10^{350} solutions to the NLP with only six decimal places of precision. Most computers could give twice this precision.

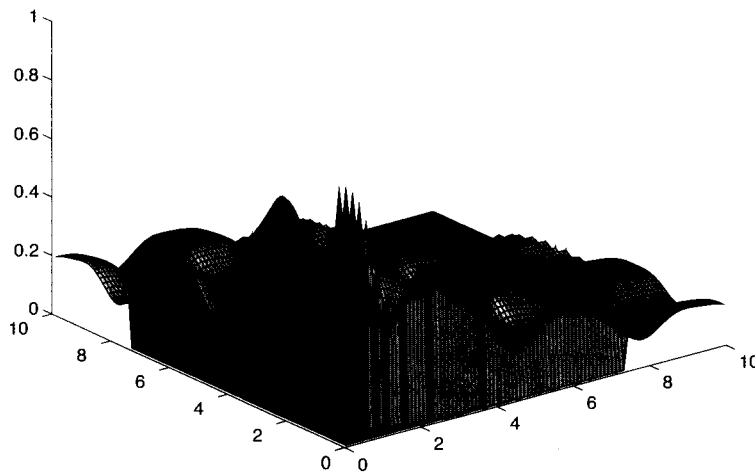


Fig. 1.2. The graph of function $G2$ for $n = 2$. Infeasible solutions were assigned a value of zero.

What about the evaluation function? How can we measure the quality of alternative solutions? One idea would be to use the function $G2$ itself as an evaluation function. Those solutions that yield higher values of $G2$ would be judged as being better than those that yield lower values. But there are some difficulties in this regard because, as illustrated in figure 1.2, there are infeasible regions, and all of the infeasible points were assigned a value of zero. The interesting boundary between feasible and infeasible regions is defined by the equation $\prod_{i=1}^n x_i = 0.75$, and the optimal solution lies on (or close to) this boundary. Searching boundaries of a feasible part of the search space isn't easy. It requires specialized operators that are tailored for just this purpose, on just this problem. This presents an additional level of difficulty that we didn't see in the SAT or TSP (although for a TSP that didn't allow transit between all possible pairs of cities, some permutations would also be infeasible). Even without this additional wrinkle, it's evident that problems that seem simple at first can offer significant challenges simply because of the number of alternative solutions. The means for devising ways to assess these solutions isn't always clear.

1.2 Modeling the problem

Every time we solve a problem we must realize that we are in reality only finding the solution to a *model* of the problem. All models are a simplification of the real world, otherwise they would be as complex and unwieldy as the natural setting itself. The process of problem solving consists of two separate general steps: (1) creating a model of the problem, and (2) using that model to generate a solution:

Problem $\Rightarrow$ Model $\Rightarrow$ Solution.

The “solution” is only a solution in terms of the model. If our model has a high degree of fidelity, we can have more confidence that our solution will be meaningful. In contrast, if the model has too many unfulfilled assumptions and rough approximations, the solution may be meaningless, or worse.

The SAT, TSP, and NLP are three canonical forms of models that can be applied in many different settings. For example, suppose a factory produces cars in various colors where there are n colors altogether. The task is to find an optimum production schedule that will minimize the total cost of painting the cars. Note, however, that each machine involved in the production line has to be switched from one color to another between jobs, and the cost of such a switch (called a *changeover*) depends on the two colors involved and their order. The cost of switching from *yellow* to *black* might be 30 units. This might be measured in dollars, minutes, or by some other reasonable standard. The cost of switching back from *black* to *yellow* might be 80 units.² The cost of going from *yellow* to *green* might be 35 units, and so forth. To minimize costs, we have to find a sequence of jobs that meets all of the production requirements for the number of cars of each color, in a timely fashion, while still keeping the costs of the operation as low as possible. This might be viewed in terms of a TSP, where each city is now a job that corresponds to painting a certain car with a particular color and the distance between cities corresponds to the cost of changing jobs. Here, the TSP would be asymmetric.

Consider the following scenario that illustrates how simplifications are inherent to modeling. Suppose a company has n warehouses that store paper supplies in reams. These supplies are to be delivered to k distribution centers. The warehouses and distribution centers can be viewed as *sources* and *destinations*, respectively. Every possible delivery route between a warehouse i and a distribution center j has a measurable transportation cost, which is determined by a function f_{ij} . The shape of this function depends on a variety of factors including the distance between the warehouse and the distribution center, the quality of the road, the traffic density, the number of required stops, the average speed limit, and so forth. For example, the transportation cost function between warehouse 2 and distribution center 3 might be defined as:

²Note the asymmetry here: the costs of switching between two colors need not be the same. Switching from *yellow* to *black* isn't usually as expensive as the obverse switch.

$$f_{23}(x) = \begin{cases} 0 & \text{if } x = 0 \\ 4 + 3.33x & \text{if } 0 < x \leq 3 \\ 19.5 & \text{if } 3 < x \leq 6 \\ 0.5 + 10\sqrt{x} & \text{if } 6 < x, \end{cases}$$

where x refers to the quantity of supplies transported from 2 to 3 (figure 1.3 displays the graph of the function f_{23}).³

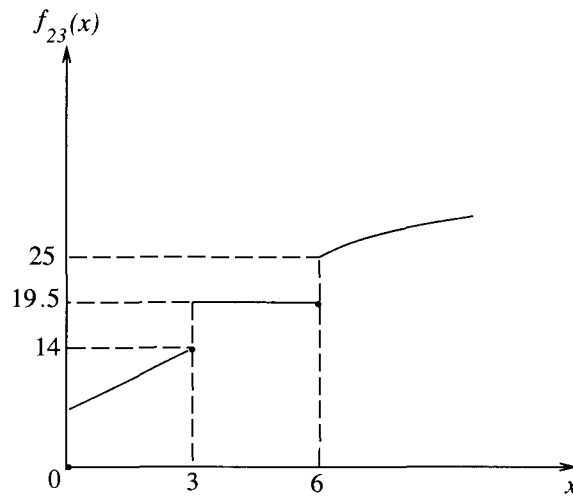


Fig. 1.3. An example transportation cost function for a given source and a given destination

Even though this function looks a bit unconventional, the justification for using it in the model of our transportation problem might be quite straightforward. If there's no delivery, the cost is zero, of course. If up to three reams of paper are transported we can use a special shipping container. This incurs an overhead cost of four units for the container and an additional cost of 3.33 units per ream. So the cost for this case increases linearly. However, if we transport more than three reams but not more than six, we can use a special wire mesh box. In this case, the cost is a flat 19.5 units, regardless of the number of reams being shipped. Finally, if we are shipping more than six reams we have to use a large reinforced crate, with a total transportation cost that depends on the number of reams being shipped and grows as a square root of that quantity plus a small overhead of 0.5 units.

Given these preliminaries, we can construct a model of the problem:

$$\text{minimize } \sum_{i=1}^n \sum_{j=1}^k f_{ij}(x_{ij}),$$

subject to

³Note that discontinuities are quite typical for most *real* transportation cost functions.

$$\begin{aligned} \sum_{j=1}^k x_{ij} &\leq \text{sour}(i), \text{ for } i = 1, 2, \dots, n, \\ \sum_{i=1}^n x_{ij} &\geq \text{dest}(j), \text{ for } j = 1, 2, \dots, k, \\ x_{ij} &\geq 0, \text{ for } i = 1, 2, \dots, n \text{ and } j = 1, 2, \dots, k, \end{aligned}$$

where *sour* is the source and *dest* is the destination. The constraints of the problem define a feasible solution: no transport from any warehouse exceeds the number of available reams in that warehouse, and the total transport to any distribution center must satisfy its demand (i.e., the total transport is at least equal to the number of ordered reams).

It might just be that this cost function describes the real-world situation faithfully (i.e., exactly), neglecting the costs for the other aspects we mentioned earlier such as the traffic density between the source and destination and so forth. And we might be able to construct similar exact functions that describe the costs of transporting the reams of paper from every warehouse to every distribution center. Still, such a precise model of the problem might be of only limited utility because these functions are too complex for many traditional optimization algorithms. For starters, they are discontinuous, and discontinuities present severe problems. The results that we would obtain after using some gradient-based methods on these functions would likely be quite poor [320]. Thus, we cannot derive a solution based on this model, so the model — as perfect as it is — is useless for deciding what to do!

What options do we have? There are at least two ways to proceed:

1. We can try to simplify the model so that traditional optimizers might return better answers.
2. We can keep the model as it is, and use a nontraditional approach to find a near-optimum solution.

The first idea is quite tempting. For example, we can *approximate* the function f_{23} as follows:

$$f'_{23}(x) = 2.66x + 8.25,$$

where x denotes the number of reams transported from 2 to 3 (figure 1.4 displays the graph of the approximate function f'_{23} together with the original f_{23}).

In this case, we simplified the transportation cost function f_{23} , and we can perform similar simplifications for the other functions. Note that if all of the f'_{ij} s were linear, we'd obtain a linear model of the problem that can be solved precisely by a linear programming method. But note that this exact solution would then be a solution for the simplified model and not for the real problem!

The second option is to leave the precise model as it is — with all of its discontinuities — and use a nontraditional method (e.g., simulated annealing or an evolutionary algorithm) to find a near-optimal solution. A large volume of experimental evidence shows that this latter approach can often be used to practical advantage.

Let's rephrase this discussion. There are two possible approaches. The first uses an approximate model, $Model_a$, of a problem, and then finds the precise solution $Solution_p(Model_a)$ for this approximate model:

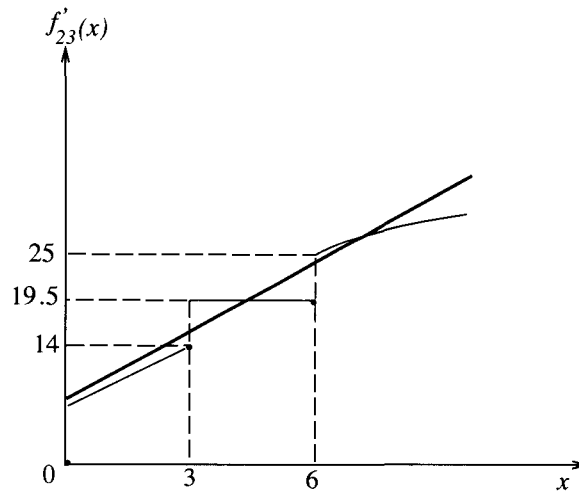


Fig. 1.4. An approximation of the transportation cost function (*bold line*) for a given source and a given destination

$$\text{Problem} \Rightarrow \text{Model}_a \Rightarrow \text{Solution}_p(\text{Model}_a).$$

The second approach uses a precise model, Model_p , of the problem, and then finds an approximate solution $\text{Solution}_a(\text{Model}_p)$ for this precise model:

$$\text{Problem} \Rightarrow \text{Model}_p \Rightarrow \text{Solution}_a(\text{Model}_p).$$

Of these two approaches, the latter one is often superior; i.e., $\text{Solution}_a(\text{Model}_p)$ is better than $\text{Solution}_p(\text{Model}_a)$ as a solution of the original problem.

But either way, this is the second source of difficulties we face in problem solving: it's difficult to obtain a precise solution to a problem because we either have to approximate a model or approximate the solution.

1.3 Change over time

As if the above concerns weren't enough, real-world problems often present another set of difficulties: they change. They change before you model them, they change while you are deriving a solution, and they change after you execute your solution. Let's look at some of the sources of trouble.

Think back to the traveling salesman problem from figure 1.1. Suppose you are the salesman and you are leaving your home at city 1 on your way to city 6. You've driven this route before and you know the distance between the two towns is, say, 20 miles. But do you know how long it will take you to travel between the cities? Not precisely. You might figure that typically you can average 30 miles per hour and so on average this trip will take 40 minutes. But

what is the probability that 40 minutes will be the exact travel time today? That's very unlikely.

The travel time depends on many factors. For example, you might be lucky and make all of the green traffic lights along the way. Or you might be unlucky and not only hit the red lights but also get stuck behind a slow-moving truck. Worse, you might get a flat tire, which would add a significant amount of time to your trip. All of these possibilities, and an unimaginable number of other outcomes including the weather, road conditions, traffic accidents, emergency vehicles requiring you to pull aside, a train on tracks crossing your path, and so forth, can be described under the heading of *noise* or *randomness*. You can't predict whether or not any of these events will happen before you leave. All you might be able to know, or estimate, is the likelihood of each of the anticipated events and the associated consequences to your travel time. But you must acknowledge that you'll never be able to account for every possibility.

Suppose you decided to simply calculate the expected travel time based on the probabilities of the known possible events and the effect they have on your travel time. Let's simplify things a little. Say there are only two possibilities for your trip: (1) everything goes fine and you can travel to city 6 in 40 minutes, or (2) you get stuck behind a slow vehicle and it takes you 60 minutes. Furthermore, let's say that these two events are equally likely. So the expected time for the trip is 50 minutes. But note that neither of the two possibilities above takes 50 minutes. If you use 50 minutes as the approximate time you ensure using a value that *will never happen in reality*. You can be 100 percent certain that your value will be wrong. It might be easy to imagine that if you used a series of these incorrect approximations to determine your travel time to each of the cities in your route, compounding your errors between each city as you go, that your final estimated time might be very different from any of the possible times that it would really take. Any decision you make based on this *average* time fails to take into account the *variability* of the time, and this can be much more important in effecting good decisions.

Random chance isn't the only source of change in real-world problems. Sometimes there are purely deterministic troubles as well. You know, for example, that travel at rush hour will be more time consuming in each city than travel at midnight. You might not know exactly how much more time you'll spend — that's a random event — but you know there is a bias that rush hour will stall your progress in traffic. That bias is a regular, predictable pattern, and you need to consider it or else your model might not correspond sufficiently to reality, and your solutions with the model won't be useful. In the worst case, a particular route between two cities might be available only during certain times of day, and not available otherwise (this often happens in cities where city planners limit your options for turning into side streets at rush hour in order to increase traffic flow). Acting as if the unavailable route were still an option would lead to an infeasible solution, and that means no solution at all.

It's also important to be sure that the model reflects current knowledge about the problem. It might be that road improvements or a new freeway system

between two of the cities on your list now allow for more rapid transit. If you fail to update your model to account for this change, you'll be deriving a solution to a problem that no longer exists.

The situation, however, is even more complex than this. The above vagaries might occur as a function of environmental changes or uncontrollable events, but none of them were conspiring against you. Unfortunately, in the real world, it's often the case that other people are trying to defeat your solution, and this requires you to continually update your model and anticipate other people's actions.

For example, suppose you are the owner of a major supermarket chain. You need to decide where to place a new store, so you calculate the cost of construction in each possible location, the demographics of the neighboring areas, the existing competition, etc., formulate an evaluation function, which would likely result in a nonlinear programming problem, and set out to decide where the best place for the store is. But there's more to the problem: as you are deciding where to put your new store, your competition is anticipating your choice of location, and they have their own new store to construct. They are actively trying to figure out how best to place their new store so as to minimize your success. If you only solve the problem of the best position for your store given the current conditions, you are treating the situation as if it were a one-player game. The real-world problem often changes while you are deriving a solution, and sometimes it changes in ways that are designed to make your life difficult.

1.4 Constraints

Unfortunately, things are often even worse than we've made it out so far because real-world problems don't offer you all of the possibilities that you might like to have. Almost all practical problems pose constraints, and if you violate the constraints you can't implement your solution. Think back to the NLP from section 1.1. There, we had a case where it wasn't enough to find the maximum of the function $G2$, we had to ensure that the solution we proposed was in the feasible region bounded by the product and summation constraints (see figure 1.2). Now at first you might think constraining problems like this would make life easier — after all, we have a smaller search space to worry about and therefore fewer possibilities to consider. That's true, but remember that to *search* for improved solutions we have to be able to move from one solution to the next. We need operators that will act on feasible solutions and hopefully in turn generate new feasible solutions that are an improvement over what we've already found. It's here where the geometry of the search space gets tricky.

For example, suppose we're faced with the problem of making a timetable for all of the classes at a college in one semester. Think about what this entails. First, we have to make a list of all the courses that will be offered. Next, we need a list of all the students assigned to each class, and let's not forget the

professor assigned to each class too. Third, we need a list of available classrooms, noting the size and other facilities that each offers (e.g., a white board, a video projector, laboratory equipment, and so forth). So then, what are we trying to accomplish? There are three hard constraints:

- Each class must be assigned to an available room that has enough seats for every assigned student and has the requisite facilities for the type of instruction (e.g., a chemistry lab must have beakers, Bunsen burners, the appropriate chemicals, safeguards, etc.).
- Students who are enrolled in more than one class can't have their classes held at the same time on the same day.
- Professors can't be assigned to teach courses that overlap in time.

We said those are the *hard constraints*. By that, we mean these are the things that absolutely must be satisfied in order to have a feasible solution. Moreover, with what we've presented so far, any assignment that meets the constraints would solve our problem. So this means the task is quite similar to the SAT problem: we have to find an assignment of classes (as compared with Boolean variables) such that an overall evaluation function returns a value of TRUE. Anything that violates the constraints means our evaluation function returns a value of FALSE. But this alone doesn't give us sufficient information to guide the search for a feasible solution.

We might be able to employ some strategy that could provide this additional information. For example, we might judge the quality of the solution not just by whether or not it satisfies the constraints, but for those assignments that fail to meet the constraints, we could tally the number of times that the constraints are violated (e.g., each time a student is assigned to two classes that meet at the same time we increase the tally). This would give us a quantitative measure of how poor our infeasible solutions were, and it might be useful in guiding us toward successively better solutions, minimizing the number of constraint violations. We could apply different operators for reassigning courses to classrooms, professors to courses, and so forth, and over time we'd hope to generate a solution that met the available constraints.

But then there are the *soft constraints*, the things we hope to accomplish but aren't mandatory. These include:

- Courses that meet twice a week should preferably be assigned to Mondays and Wednesdays or Tuesdays and Thursdays. Having these courses meet on consecutive days or with two or more days inbetween is not desired.
- Courses that meet three times per week should preferably be assigned to Mondays, Wednesdays, and Fridays. Other assignments are not desired.
- Course times should be assigned so that students don't have to take final exams for multiple courses without any breaks in between (final exam times are typically based on the time for the course).

- If undergraduate prerequisite courses are scheduled for the same day as their counterpart graduate courses, they should preferably be given earlier than the graduate course (this facilitates learning foundational material prior to advanced material in the same day).
- If more than one room satisfies the requirements for a course and is available at the designated time, the course should be assigned to the room with the capacity that is closest to the class size (this means that large auditoriums aren't used for small classes, thus enhancing student participation).

Certainly we could imagine many more such soft constraints. Any assignment that meets the hard constraints is feasible, but not necessarily optimal in light of the soft constraints. Here is where the problem gets sticky. First, we have to quantify each of the soft constraints into mathematical terms so that we can evaluate any two candidate assignments and decide that one is better than the other. Next, we have to be able to modify one feasible solution and, hopefully, generate another feasible solution that better meets the soft constraints.

Let's take the first issue: each soft constraint has to be quantified. Considering the first soft constraint, we could say that for each case where a solution is feasible, we could count up the number of times twice-a-week courses become separated by two or more days, or are placed on consecutive days, and use this as a penalty term. The lower the term, the better the solution. In fact, we could employ a similar approach to each of the soft constraints. But what would we do when we're through? We'd still need an overall method for considering the degree of violation of each of these constraints. That is, we'd have to determine the answers to questions such as: which is worse, scheduling five students to have back-to-back final exams, or scheduling back-to-back classrooms at opposite ends of the campus? Each of these possible trade-offs would have to be considered and quantified in some evaluation function, which poses quite a challenge!

Of course it's worse than that because even after all of these soft constraints have been quantified, we are still left with the problem of searching for the best assignment: the solution that is both feasible and minimizes our evaluation function for the soft constraints. Suppose we have found a feasible solution, but it doesn't do very well with regard to the soft constraints. Say we apply some variation operators to this solution and we significantly improve the situation with respect to the soft constraints, but in so doing, we generate a solution that violates one hard constraint. Now what? We might choose to discard the solution since it's infeasible, or we might see if we can repair it to generate a feasible solution that still handles the soft constraints well. Either way, this is typically a difficult chore. It would be even better to devise variation operators that never corrupt a feasible solution into an infeasible solution while still searching vigorously over the space of feasible solutions to find those that best handle the soft constraints. That's a nice aspiration, but it's often not much more than

wishful thinking. Effectively handling real-world constrained problems is one of the most challenging tasks we face.

1.5 The problem of proving things

Although it seems a bit strange, in our quest to solve problems, we sometimes make matters more difficult than they have to be. Having worked with students at many universities and in many countries, we've experienced the phenomenon that if you ask someone to *find* some solution to a problem, they'll typically find this much easier than if you had asked them to *prove* something about the solution, even when the two tasks are exactly the same mathematically.

As an example, think about any mathematical problem with just a moderate degree of difficulty. The task in the problem should be to find a particular value, whether it's the height of a building, the speed of a car, or the time to complete your homework. Regardless, the task should be to find the value of x . For example, you'll take four hours to fill a pool using a large pipe. You'll take six hours if you use a small pipe. How long would it take if you used both pipes? If the problem is formulated in terms of "find the value x " — such as, find the amount of time required to fill the pool using both pipes — this is a fairly easy task. But, for some reason, if we change the task and instead ask: prove that the amount of time required to fill the pool using both pipes is less than a , fewer students will manage this problem despite the fact that it's not the slightest bit more difficult. If you can find the time required to fill the pool, and if it's smaller than a constant a , then the proof is completed. To verify this for yourself, take the values given above for the times required to fill the pool using either pipe alone and test it out on your friends. Ask some to find the time required when using both pipes, and ask others to prove that the time required is less than 2 hours and 25 minutes.⁴

We believe that the reason for this aversion to proving things is that most people simply aren't experienced in proving things and don't know how to begin. Generalizing on this observation, many problems are apparently difficult simply because of the difficulty encountered when facing the question: "How should I start?"

Here's an example to help you exercise your ability to frame problems and get started on their solution:

Prove that any polyhedron must have at least two faces with the same number of edges.

No doubt, the first inclination when reading this is to think "Do I have to?" Yes, you do. So let's get started.

First, it's helpful to remind ourselves of some basic ideas about proving things. One way to prove something is simply to show that the conclusion

⁴Actually, the required time is 2 hours and 24 minutes.

follows directly from the given knowledge. For example, suppose that $a > b$ and $b > c$, then prove that $a > c$. You simply cite the transitive law and you're done. Another way to prove something is to consider the contrapositive. Recall that "if p then q " is logically equivalent to "if not q then not p ." Sometimes it might be easier to prove this relationship. Still another way to prove something is to assume that the condition you are trying to prove is false and then show that this is impossible. This is called *proof by contradiction*. Let's try that here.

Restating our problem, then, we need a proof that shows it's impossible to construct a polyhedron such that all of the faces have a different number of edges. The problem seems difficult because there aren't that many general theorems about polyhedrons. There is a famous one — Euler's theorem — which states that the following formula holds for every polyhedron:

$$v + f = e + 2,$$

where v , e , and f represent the number of vertices, edges, and faces of the polyhedron, respectively. But just how we might actually use this theorem in proving that all of the faces must have a different number of edges isn't very clear.

The initial step is the hardest part of the puzzle. Indeed, here the problem doesn't provide us with any convenient starting point. There's no number in the problem, nothing for us to factorize, divide by two, multiply by six, or anything else. In situations like this, it's often advantageous to introduce such a number ourselves: Let's consider a polyhedron with f faces. Now we have something, a variable, to work with.

We have to prove something about the edges of faces for this polyhedron. Each face has a number of edges. To say something about these numbers, it would be nice at least to know the range of values that we might expect. So, let's ask the question: what is the minimum number of edges that a face may have? The answer is three, and in that case the face is a triangle. This minimum number is independent of the total number f of faces of the polyhedron.

And what is the maximum number of edges a face may have? Well, each edge belongs to precisely two faces, so if we have a face with six edges, we know that the face is part of a polyhedron with at least seven faces (the current face plus six new faces, one for each edge; see figure 1.5). Thus, any face of a polyhedron that has f faces in total can't have more than $f - 1$ edges, since a new face originates from each edge.

This concludes the proof.

If this fact surprises you then take note that you have lost sight of what you are trying to prove. Go back and remind yourself of the problem. The reason the proof is complete is because if there are f faces in total, and if each face must have a number of edges between 3 and $f - 1$, then some repetitions in the number of edges must occur across the f faces. There are more faces than possibilities for the number of edges, so you'll have to use some of these numbers more than once. Therefore, at least two faces will have the same number of edges.

The keys to solving this problem were to invent a starting point to pursue and not lose sight of the goal.

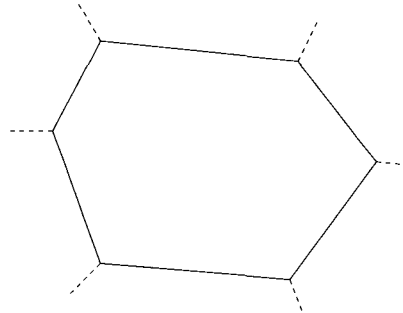


Fig. 1.5. A single face of a polyhedron with six edges

1.6 Your chance for glory

Now that you're primed for solving a problem, here's your chance to *prove* you can do it. This problem was offered to us by Peter Ross of the University of Edinburgh.

Mr. Smith and his wife invited four other couples for a party. When everyone arrived, some of the people in the room shook hands with some of the others. Of course, nobody shook hands with their spouse and nobody shook hands with the same person twice.

After that, Mr. Smith asked everyone how many times they shook someone's hand. He received different answers from everybody.

How many times did Mrs. Smith shake someone's hand?

We encourage you to put the book aside and try to frame this problem for yourself. Try to think of a starting point, maybe a graphical image that describes the problem, and see if you can follow it through to a successful conclusion.

The reason that this problem is a challenge is that once again there's no obvious starting point, but it's really quite simple to develop a graphical model for the problem (see figure 1.6).

This is a good model because we can clearly see Mr. Smith (shaded circle) and the remaining nine people (including his wife). Now, what information do we have? (Remember, think about all the given information.) The only information provided in this problem is that Mr. Smith asked every person in the room how many times they shook someone's hand, and that all of the answers he received were different.

What answers did he get then? Well, the minimum number of handshakes is zero: it's possible that some antisocial person didn't shake anyone's hand at all. But what is the maximum number of handshakes for a single person? It is more difficult to think of *asking* this question than it is to answer it. The maximum number of handshakes for a person is eight, as there are ten people

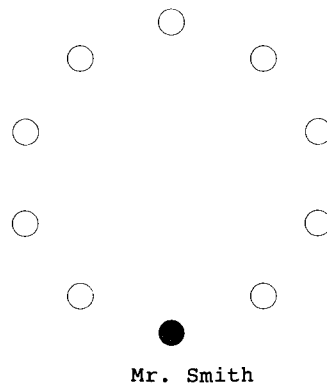


Fig. 1.6. Mr. Smith and the other people

in the room, and a person can't shake hands with himself or herself nor with his or her spouse.

Let's collect the facts we have now. Mr. Smith asked the question to nine people in the room and all of the answers were different. Furthermore, each answer was a number between zero and eight. Therefore, the answers he received were zero, one, two, three, four, five, six, seven, and eight (not necessarily in that order, of course!). So, we can update our model accordingly (figure 1.7).

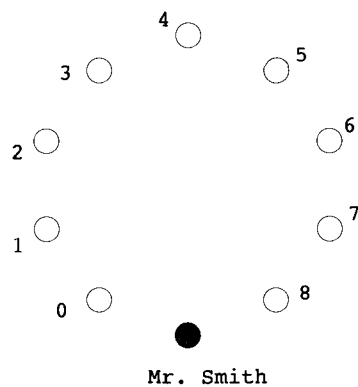


Fig. 1.7. Mr. Smith and the other people. The number of handshakes is indicated for each person (except Mr. Smith).

What's the next step, then? What can we infer? It seems, not much. We've already taken advantage of the fact that all of the answers were different, and we now know all of these answers. But how many times did Mrs. Smith shake hands? And which of the people in figure 1.7 is Mrs. Smith, anyway? If we only knew who has shaken hands with whom, then everything would be easier.

But, can't we? Let's try to draw all of the handshakes that were exchanged. The person 8 (we'll name everyone except Mr. Smith by the number of handshakes they exchanged) shook hands eight times, i.e., with everyone else in the room except himself or herself and his or her spouse. We can do two things based on this observation. We can draw all of the handshakes made by person 8 (see figure 1.8) and we can also conclude that the spouse of person 8 is person 0.

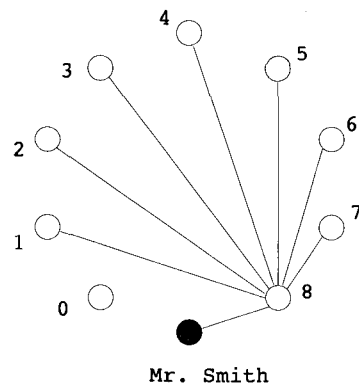


Fig. 1.8. Mr. Smith and the other people. Each handshake from person 8 is indicated.

Did the proverbial light bulb just turn on? We can turn our attention to person 7. This person exchanged seven handshakes, i.e., with everyone in the room except himself or herself, his or her spouse, and the spouse of person 8. Again, we can add all of the handshakes made by person 7 to our model (see figure 1.9) and we can also conclude that the spouse of person 7 is person 1.

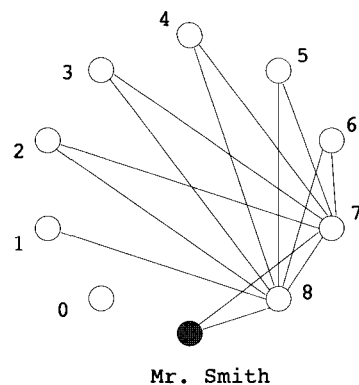


Fig. 1.9. Mr. Smith and the other people. All of the handshakes from persons 8 and 7 are indicated.

We can repeat this reasoning for persons 6 and 5. After adding the lines that correspond to the handshakes they exchanged we find an interesting graph shown in figure 1.10. The immediate conclusions are:

- The spouse of person 6 is person 2.
- The spouse of person 5 is person 3.

Using these conclusions, we know that the spouse of Mr. Smith is person 4. Therefore, Mrs. Smith exchanged four handshakes.

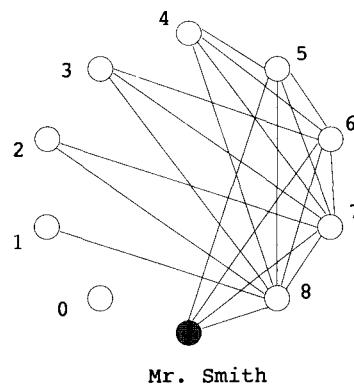


Fig. 1.10. Mr. Smith and the other people. All of the handshakes are indicated.

How did you do?

1.7 Summary

Problem solving is difficult for several reasons:

- Complex problems often pose an enormous number of possible solutions.
- To get any sort of solution at all, we often have to introduce simplifications that make the problem tractable. As a result, the solutions that we generate may not be very valuable.
- The conditions of the problem change over time and might even involve other people who want you to fail.
- Real-world problems often have constraints that require special operations to generate feasible solutions.

Furthermore, problem solving is often more difficult than it needs to be because we are threatened by the idea of *proving* things.

- Don't let the word *prove* intimidate you.
- Think about different ways to prove the solution. Sometimes it's useful to assume that what you are trying to prove is false, and then show that the false condition is impossible.
- Be ready to take a first step even if there doesn't seem to be anything in the problem for you to step on. Invent something, a variable, that describes a facet of the problem and see if that helps. Don't worry if you have to start over a few times before you finally find a path that takes you to the answer.

One way to facilitate taking the first step is to understand the search space: what are the variables? What are their possible values? What are the constraints? Most of all:

- Don't lose sight of the goal!

If you forget what you are trying to prove, you may as well watch television because your success rate will be the same either way: zero! Stay focused on the end result. Always ask yourself if what you are doing facilitates getting to where you want to go. You may not know the answer with certainty, but keep asking yourself this anyway. At least, you'll more quickly identify those paths that don't lead to the goal and you'll minimize the time spent on dead ends. At most, you'll readily identify the right path to choose and be on your way!

II. How Important Is a Model?

Whenever you solve a real-world problem, you have to create a model of that problem first. It is critical to make the distinction that the model that you work with isn't the same as the problem. Every model leaves something out. It has to — otherwise it would be as complicated and unwieldy as the real-world itself. We always work with simplifications of how things really are. We have to accept that. Every solution we create is, to be precise, a solution *only to the model* that we postulate as being a useful representation of some real-world setting that we want to capture.

The trouble with models is that every one of them has an associated set of assumptions. The most common mistake when dealing with a model is to forget the assumptions. For example, suppose you throw a ball into the air and plot its trajectory as it falls to the earth. What is the shape of that trajectory? The most likely answer is that it's parabolic, but that's incorrect. "Oh, yes, wind resistance, we forgot about that," you might say. True, we did omit that here, but that's not why the answer is wrong. Even with no wind resistance, the trajectory isn't parabolic. The correct answer is that the trajectory is elliptical. Why? Because it's only parabolic under the assumption that *the earth is flat*. Once we assume that the earth is curved, the trajectory is no longer parabolic.

Certainly, within your local neighborhood (i.e., the distance you could possibly throw a ball) the earth seems pretty flat. Thus there is very little difference in the model developed under the parabolic trajectory and the one described using an elliptical path. To illustrate the consequence of unstated assumptions more dramatically, we'll use a simple example that was offered to us by Jan Plaza of State University of New York at Plattsburgh.

There is a circle R with a radius of 1 on the ground in front of you. A straight line L of infinite length (in both directions) is thrown on the ground such that it intersects the circle. What is the probability p that the length l of the chord created by the intersection of the circle and the line is greater than or equal to $\sqrt{3}$?

To calculate this probability, we have to envision a space of possible outcomes and then assign an appropriate likelihood to the event that contains all of the outcomes such that the length of the intersecting chord is greater than or equal to $\sqrt{3}$.

Before we consider a few possibilities for constructing the space of outcomes, let's remind ourselves of some elementary geometrical facts: (1) the length of

a side of an equilateral triangle T inscribed in the circle R is $\sqrt{3}$, and (2) the radius of a circle r inscribed into the triangle T is $1/2$ (see figure II.1).

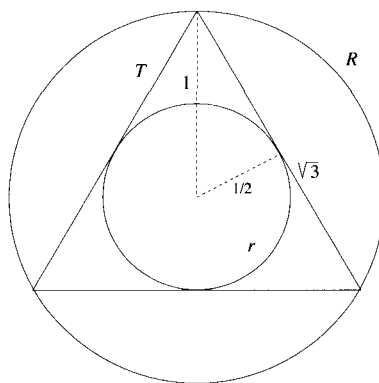


Fig. II.1. Circle R , equilateral triangle T , and inscribed circle r

Possibility 1.

We know that the length l of the chord may vary from 0 to 2, since 2 is the diameter of R . Let's consider the midpoint m of the chord. If m is in the interior of r then $l \geq \sqrt{3}$, otherwise m is in the interior of R but not in the interior of r and $l < \sqrt{3}$ (see figure II.2a). Since the midpoint could turn up anywhere inside R , the probability that it turns up inside r , implying $l \geq \sqrt{3}$, is determined as the ratio of the area of r to the area of R . Thus

$$p = \frac{\pi(\frac{1}{2})^2}{\pi(1)^2} = \frac{1}{4}$$

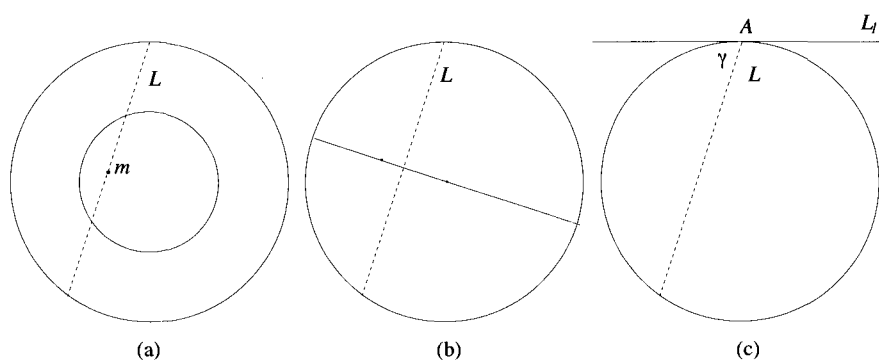


Fig. II.2. Three interpretations of throwing a line

Possibility 2.

Let's draw an arbitrary diameter of R . When a line L is thrown, we can rotate R until the diameter is perpendicular to the line, as shown in (figure II.2b). Now we only have to look at half the circle because the length of the

chord must be less than or equal to the diameter of R . The midpoint of the chord will either be less than a $1/2$ unit away from the center of R or not. If it is, the length of the chord is greater than or equal to $\sqrt{3}$. Otherwise, it's less than $\sqrt{3}$. Thus, the probability p can be determined as a ratio between the diameters of r and R :

$$p = \frac{\frac{1}{2}}{1} = \frac{1}{2}$$

Possibility 3.

Let's fix an arbitrary point A on the boundary of R . When a line L is thrown, we can rotate R until A belongs to L . Let's also draw a tangent line L_1 to R at A (see figure II.2c). From the figure we can see that the angle γ between lines L and L_1 determines the outcome of the trial. If γ is between $\pi/3$ and $2\pi/3$ then the midpoint of L will be inside r and therefore l will be at least $\sqrt{3}$. On the other hand, if $0 \leq \gamma < \pi/3$ or $2\pi/3 < \gamma \leq \pi$ then l will be smaller than $\sqrt{3}$. Since there are π possible radians for γ and a range of $\frac{1}{3}\pi$ generate lines of length $l \geq \sqrt{3}$, the probability is:

$$p = \frac{\frac{1}{3}\pi}{\pi} = \frac{1}{3}$$

Which of these three possibilities is correct? It seems that all three cases are reasonable and all three values found for p make sense! Yet they're all different! How can this be?

Still, all of these possible solutions are correct. It just depends on how you model the possibilities for how you'll generate a random line. The reason is that each model makes implicit assumptions about the way in which a line is "thrown" at the circle. Can you make them explicit in each case? We'll give you the answer for the first possibility: the assumption is that the midpoint of the line L can occur at any interior point of R , and that no points or regions are favored over any others. If you can make the assumptions for the second and third possibilities explicit, you'll see why the answers to the question differ in each case. If you can't, you should take away a healthy respect for models, because even in this very simple situation, an implicit assumption can lead to drastically different results. Just imagine what sort of trouble you can get into when things are more complicated!

A puzzle from the early sixteenth century (this problem has staying power!) also illustrates the importance of having the right model.

In front of you is a small 3×3 chess board (figure II.3). Two black and two white knights are placed in the corners. Your task is to make the two white knights exchange places with the black knights in as few moves as possible. Of course, the knights have to move the same way they do in the game of chess.

B		B
W		W

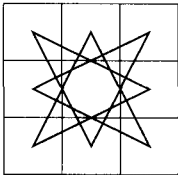
Fig. II.3. Two black (B) and two white (W) knights on a 3×3 chess board

This is a great puzzle to start out with a “trial and error” approach — the problem is that you’ll find mostly “errors.” If you have the right model in mind, however, everything turns out to be straightforward.

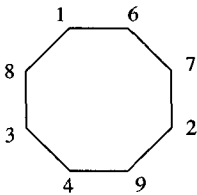
If we redraw the board marking all of the squares from 1 to 9 (figure II.4a), we can indicate every possible move for a knight between these squares. For a knight to cycle all the way back to its original position it requires a particular path of eight steps (moves): starting from the top-left square, move to the middle-right square, then (to avoid moving back) continue to bottom-left square, top-middle, etc. (figure II.4b). If we “untie” this path to make a simple loop (figure II.4c), it’s easy to see that every knight must follow a particular direction and no knight can pass any other along the way. Since it takes 4 moves for a knight to arrive in its required position, the total number of moves that’s necessary and sufficient is 16. The order of the moves is also apparent: we move each knight one step at a time and repeat this sequence of four moves, four times.

1	2	3
4	5	6
7	8	9

(a)



(b)



(c)

Fig. II.4. (a) The board and numbers of the squares. (b) The path (loop) of possible moves. (c) The untied loop

There’s also another version of this problem where any number of successive moves by the same piece is considered as one move. In that case seven moves are required. Can you see how?

2. Basic Concepts

If we can really understand the problem,
the answer will come out of it,
because the answer is not separate
from the problem.

Krishnamurti, *The Penguin Krishnamurti Reader*

Three basic concepts are common to every algorithmic approach to problem solving. Regardless of the technique that you employ, you'll need to specify: (1) the representation, (2) the objective, and (3) the evaluation function. The representation encodes alternative candidate solutions for manipulation, the objective describes the purpose to be fulfilled, and the evaluation function returns a specific value that indicates the quality of any particular solution given the representation (or minimally, it allows for a comparison of the quality of two alternative solutions). Let's consider each of these aspects of problem solving in turn.

2.1 Representation

The three problems that we examined in the previous chapter provide a good basis for examining alternative representations. Starting with the satisfiability problem (SAT) where we have n variables that are logical bits, one obvious way to represent a candidate solution is a binary string of length n . Each element in the string corresponds to a variable in the problem. The size of the resulting search space under this representation is 2^n , as there are 2^n different binary strings of length n . Every point in this search space is a feasible solution.

For the n -city traveling salesman problem (TSP), we already considered one possible representation: a permutation of the natural numbers $1, \dots, n$ where each number corresponds to a city to be visited in sequence. Under this representation, the search space $\mathcal{S}$ consists of all possible permutations, and there are $n!$ such orderings. However, if we're concerned with a symmetric TSP, where the cost of traveling from city i to j is the same in either direction, then we don't care if we proceed down the list of cities from right to left, or left to right. Either way, the tour is the same. This means that we can shrink the size of the search space by one-half. Furthermore, the circuit would be the same regardless of which city we started with, so this reduces the size of the search space by factor of n . Consequently, the real size of the search space reduces to $(n-1)!/2$, but we've seen that this is a huge number for even moderately large TSPs.

For a nonlinear programming problem (NLP), the search space consists of all real numbers in n dimensions. We normally rely on floating-point representations to approximate the real numbers on a computer, using either single or double precision. We saw in the previous chapter that for six digits of precision on variables in a range from 0 to 10 there are 10^{7n} different possible values.

For each problem, the representation of a potential solution and its corresponding interpretation implies the search space and its size. This is an important point to recognize: The size of the search space is not determined by the problem; it's determined by your representation and the manner in which you handle this encoding.

Choosing the right search space is of paramount importance. If you don't start by selecting the correct domain to begin your search, you can either add numerous unviable or duplicate solutions to the list of possibilities, or you might actually preclude yourself from ever being able to find the right answer at all.

Here's a good example. There are six matches on a table and the task is to construct four equilateral triangles where the length of each side is equal to the length of a match. It's easy to construct two such triangles using five matches (see figure 2.1), but it's difficult to extend it further into four triangles, especially since only one match remains. Here, the formulation of the problem is misleading because it suggests a two-dimensional search space (i.e., the matches were placed on a table). But to find a solution requires moving into three dimensions (figure 2.2). If you start with the wrong search space, you'll never find the right answer.¹

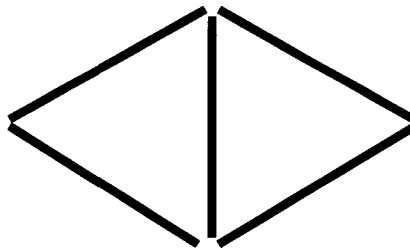


Fig. 2.1. Two triangles, five matches used, one match left

2.2 The objective

Once you've defined the search space, you have to decide what you're looking for. What's the objective of your problem? This is a mathematical statement of the task to be achieved. It's not a function, but rather an expression. For

¹If you give this problem to another person, you can experiment by giving them some additional "hints" like: you can break the matches into smaller pieces. Such "hints" are *harmful* in the sense that (1) they don't contribute anything towards obtaining the solution, and (2) they expand the search space in a significant way! But don't try this on your friends.

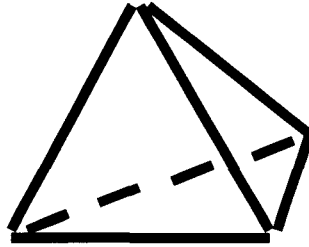


Fig. 2.2. Four triangles, six matches

example, consider the TSP. The objective is typically to minimize the total distance traversed by the salesman subject to the constraint of visiting each city once and only once and returning to the starting city. In mathematical terms, the objective is:

$$\min \sum dist(x, y).$$

For the SAT, the objective is to find the vector of bits such that the compound Boolean statement is satisfied (i.e., made TRUE). For the NLP, the objective is typically to minimize or maximize some nonlinear function of candidate solutions.

2.3 The evaluation function

The objective, however, is not the same thing as the evaluation function. The latter is most typically a mapping from the space of possible candidate solutions under the chosen representation to a set of numbers (e.g., the reals), where each element from the space of possible solutions is assigned a numeric value that indicates its quality. The evaluation function allows you to compare the worth of alternative solutions to your problem as you've modeled it. Some evaluation functions allow for discerning a ranking of all possible solutions. This is called an *ordinal* evaluation function. Alternatively, other evaluation functions are *numeric* and tell you not only the order of the solutions in terms of their perceived quality, but also the degree of that quality.

For example, suppose we wanted to discover a good solution to a TSP. The objective is to minimize the sum of the distances between each of the cities along the route while satisfying the problem's constraints. One evaluation function might map each tour to its corresponding total distance. We could then compare alternative routes and not only say that one is better than another, but exactly how much better. On the other hand, it's sometimes computationally expensive to calculate the exact quality of a particular solution, and it might only be necessary to know approximately how good or bad a solution is, or simply whether or not it compares favorably with some other choice. In such

a case, the evaluation function might operate on two candidate solutions and merely return an indication as to which solution was favored.

In every real-world problem, you choose the evaluation function; it isn't given to you with the problem. How should you go about making this choice? One obvious criterion to satisfy is that when a solution meets the objective completely it also should have the best evaluation. The evaluation function shouldn't indicate that a solution that fails to meet the objective is better than one that meets the objective. But this is only a rudimentary beginning to designing the evaluation function.

Often, the objective suggests a particular evaluation function. For example, just above, we considered using the distance of a candidate solution to a TSP as the evaluation function. This corresponds to the objective of minimizing the total distance covered: the evaluation function is suggested directly from the objective. This often occurs in an NLP as well. For the problem of maximizing the function depicted in figure 1.2, the function itself can serve as an evaluation function with better solutions yielding larger values. But you can't always derive useful evaluation functions from the objective. For the SAT problem, where the objective is to satisfy the given compound Boolean statement (i.e., make it TRUE), every approximate solution is FALSE, and this doesn't give you any useful information on how to improve one candidate solution into another, or how to search for any better alternative. In these cases, we have to be more clever and adopt some surrogate evaluation function that will be appropriate for the task at hand, the representation we choose, and the operators that we employ to go from one solution to the next.

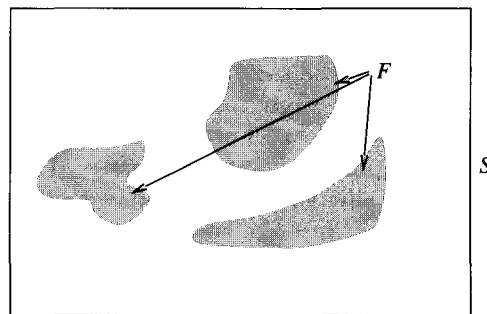


Fig. 2.3. A search space S and its feasible part F . Note that the feasible regions may be disjoint.

When you design the evaluation function, you also have to consider that for many problems, the only solutions of interest will be in a subset of the search space S . You're only interested in *feasible* solutions, i.e., solutions that satisfy problem-specific constraints. The NLP of figure 1.2 constitutes a perfect example: we're only interested in floating-point vectors such that the product of their component values isn't smaller than 0.75 and their total doesn't exceed

the product of 7.5 and the number of variables. In other words, the search space $\mathcal{S}$ for the NLP can be defined as the set of all floating-point vectors such that their coordinates fall between 0 and 10. Only a subset of these vectors satisfies the additional constraints (concerning the product and the sum of components). This subset constitutes the feasible part of the search space $\mathcal{F}$. In the remainder of the book, we will always distinguish between the search space $\mathcal{S}$ and the feasible search space $\mathcal{F} \subseteq \mathcal{S}$ (see figure 2.3). Note, however, that for the SAT and TSP, $\mathcal{S} = \mathcal{F}$; all of the points from the search space are feasible (presuming for the TSP that there is a path from every city to every other city).

2.4 Defining a search problem

We can now define a search problem.² Given a search space $\mathcal{S}$ together with its feasible part $\mathcal{F} \subseteq \mathcal{S}$, find $x \in \mathcal{F}$ such that

$$eval(x) \leq eval(y),$$

for all $y \in \mathcal{F}$. Note that here we're using an evaluation function for which solutions that return smaller values are considered better (i.e., the problem is one of minimization), but for *any* problem, we could just as easily use an evaluation function for which larger values are favored, turning the search problem into one of maximization. There is nothing inherent in the original problem or an approximate model that demands that the evaluation function be set up for minimization or maximization. Indeed, the objective itself doesn't appear anywhere in the search problem defined above. This problem statement could just as easily be used to describe a TSP, a SAT, or an NLP. The search itself doesn't know what problem you are solving! All it "knows" is the information that you provide in the evaluation function, the representation that you use, and the manner in which you sample possible solutions. If your evaluation function doesn't correspond with the objective, you'll be searching for the right answer to the wrong problem!

The point x that satisfies the above condition is called a *global* solution. Finding such a global solution to a problem might be very difficult. In fact, that's true for all three of the example problems discussed so far (i.e., the SAT, TSP, and NLP), but sometimes finding the best solution is easier when we can concentrate on a relatively small subset of the total (feasible or perhaps also infeasible) search space. Fewer alternatives can sometimes make your life easier. This is a fundamental observation that underlies many search techniques.

²The terms "search problem" and "optimization problem" are considered synonymous. The search for the best feasible solution is the optimization problem.

2.5 Neighborhoods and local optima

If we concentrate on a region of the search space that's “near” some particular point in that space, we can describe this as looking at the *neighborhood* of that point. Graphically, consider some abstract search space $\mathcal{S}$ (figure 2.4) together with a single point $x \in \mathcal{S}$. The intuition is that a neighborhood $N(x)$ of x is a set of all points of the search space $\mathcal{S}$ that are close in some measurable sense to the given point x .

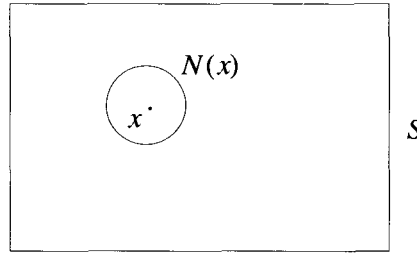


Fig. 2.4. A search space $\mathcal{S}$, a potential solution x , and its neighborhood $N(x)$ denoted by the interior of the circle around x

We can define the nearness between points in the search space in many different ways. Two important possibilities include:

- We can define a distance function *dist* on the search space $\mathcal{S}$:

$$\text{dist} : \mathcal{S} \times \mathcal{S} \longrightarrow \mathbb{R},$$

and then define the neighborhood $N(x)$ as

$$N(x) = \{y \in \mathcal{S} : \text{dist}(x, y) \leq \epsilon\},$$

for some $\epsilon \geq 0$. Another way to say this is that if y satisfies the condition for $N(x)$ above then it is in the ϵ -neighborhood of x . Note that when we're dealing with search spaces defined over continuous variables (e.g., the NLP) a natural choice for defining a neighborhood is the Euclidean distance:

$$\text{dist}(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}.$$

This is also the case for the SAT problem where the distance between two binary strings can be defined as a Hamming distance, i.e., the number of bit positions with different truth assignments. Be wary, though, that just because something is a “natural” choice doesn't necessarily mean it's the best choice in light of other considerations. Natural choices do, however, often provide a good starting position.

- We can define a mapping m on the search space $\mathcal{S}$ as

$$m : \mathcal{S} \longrightarrow 2^{\mathcal{S}},$$

and this mapping defines a neighborhood for any point $x \in \mathcal{S}$. For example, we can define a *2-swap* mapping for the TSP that generates a new set of potential solutions from any potential solution x . Every solution that can be generated by swapping two cities in a chosen tour can be said to be in the neighborhood of that tour under the 2-swap operator. In particular, a solution x (a permutation of $n = 20$ cities):

$$15 - 3 - 11 - 19 - 17 - 2 - \dots - 6,$$

has $\frac{n(n-1)}{2}$ neighbors. These include:

$$\begin{aligned} &15 - 17 - 11 - 19 - 3 - 2 - \dots - 6 \text{ (swapping cities at the second} \\ &\quad \text{and fifth locations),} \\ &2 - 3 - 11 - 19 - 17 - 15 - \dots - 6 \text{ (swapping cities at the first} \\ &\quad \text{and sixth locations),} \\ &15 - 3 - 6 - 19 - 17 - 2 - \dots - 11 \text{ (swapping cities at the third} \\ &\quad \text{and last locations),} \\ &\text{etc.} \end{aligned}$$

In contrast, for the SAT problem we could define a *1-flip* mapping that generates a set of potential solutions from any other potential solution x . These are all the solutions that can be reached by flipping a single bit in a particular binary vector, and these would be in the neighborhood of x under the 1-flip operator. For example, a solution x (a binary string of, say, $n = 20$ variables):

$$01101010001000011111$$

has n neighbors. These include:

$$\begin{aligned} &11101010001000011111 \text{ (flipping the first bit),} \\ &00101010001000011111 \text{ (flipping the second bit),} \\ &01001010001000011111 \text{ (flipping the third bit),} \\ &\text{etc.} \end{aligned}$$

In this case we could also define the neighborhood in terms of a distance function: the neighborhood contains all of the strings with a Hamming distance from x that is less than or equal to one.

With the notion of a “neighborhood” defined, we can now discuss the concept of a *local* optimum. A potential solution $x \in \mathcal{F}$ is a local optimum with respect to the neighborhood N , if and only if

$$eval(x) \leq eval(y),$$

for all $y \in N(x)$ (again assuming a minimization criterion). It's often relatively easy to check whether or not a given solution is a local optimum when the size of the neighborhood is very small, or when we know something about the derivative of the evaluation function (if it exists).

Many search methods are based on the statistics of the neighborhood around a given point; that is, the sequence of points that these techniques generate while searching for the best possible solution relies on *local* information at each step along the way. These techniques are designed to locate solutions within a neighborhood of the current point that have better corresponding evaluations. Appropriately, they are known as “neighborhood” or “local” search strategies [476]. For example, suppose you're facing the problem of maximizing the function $f(x) = -x^2$ and you've chosen to use $f(x)$ itself as an evaluation function. Better solutions generate higher values of $f(x)$. Say your current best solution x is 2.0, which has a worth of -4.0 . You might define an ϵ -neighborhood with an interval of 0.1 on either side of 2.0 and then search within that range. If you sampled a new point in $[1.9, 2.1]$ and found that it had a better evaluation than 2.0, you'd replace 2.0 with this better solution and continue on from there; otherwise, you'd discard the new solution and take another sample from $[1.9, 2.1]$.

Naturally, most evaluation functions in real-world problems don't look like a quadratic bowl such as $f(x) = -x^2$. The situation is almost always more difficult than this. The evaluation function defines a response surface that is much like a topography of hills and valleys (e.g., as shown in figure 1.2), and the problem of finding the best solution is similar to searching for a peak on a mountain range while walking in a dense fog. You can only sample new points in your immediate vicinity and you can only make local decisions about where to walk next. If you always walk uphill, you'll eventually reach a peak, but this might not be the highest peak in the mountain range. It might be just a “local optimum.” You might have to walk downhill for some period of time in order to find a position such that a series of local decisions within successive neighborhoods leads to the global peak.

Local search methods (chapter 3) present an interesting trade-off between the size of the neighborhood $N(x)$ and the efficiency of the search. If the size of the neighborhood is relatively small then the algorithm may be able to search the entire neighborhood quickly. Only a few potential solutions may have to be evaluated before a decision is made on which new solution should be considered next. However, such a small range of visibility increases the chances of becoming trapped in a local optimum! This suggests using large neighborhoods: a larger range of visibility could help in making better decisions. In particular, if the visibility were unrestricted (i.e., the size of the neighborhood were the same as the size of the whole search space), then eventually we'd find the best series of steps to take. The number of evaluations, however, might become enormous, so we might not be able to complete the required calculations within the lifetime of the universe (as noted in chapter 1). When using local search methods, the

appropriate size of the neighborhood can't be determined arbitrarily. It has to fit the task at hand.

2.6 Hill-climbing methods

Let's examine a basic *hill-climbing* procedure and its connection with the concept of a neighborhood. Hill-climbing methods, just like all local search methods (chapter 3), use an iterative improvement technique.³ The technique is applied to a single point — the current point — in the search space. During each iteration, a new point is selected from the neighborhood of the current point. If that new point provides a better value in light of the evaluation function, the new point becomes the current point. Otherwise, some other neighbor is selected and tested against the current point. The method terminates if no further improvement is possible, or we run out of time or patience.

It's clear that such hill-climbing methods can only provide locally optimum values, and these values depend on the selection of the starting point. Moreover, there's no general procedure for bounding the relative error with respect to the global optimum because it remains unknown. Given the problem of converging on only locally optimal solutions, we often have to start hill-climbing methods from a large variety of different starting points. The hope is that at least some of these initial locations will have a path that leads to the global optimum. We might choose the initial points at random, or on some grid or regular pattern, or even in the light of other information that's available, perhaps as a result of some prior search (e.g., based on some effort someone else made to solve the same problem).

There are a few versions of hill-climbing algorithms. They differ mainly in the way a new solution is selected for comparison with the current solution. One version of a simple iterated hill-climbing algorithm is given in figure 2.5 (steepest ascent hill-climbing). Initially, all possible neighbors of the current solution are considered, and the one $\mathbf{v}_n$ that returns the best value $eval(\mathbf{v}_n)$ is selected to compete with the current string $\mathbf{v}_c$. If $eval(\mathbf{v}_c)$ is worse than $eval(\mathbf{v}_n)$, then the new string $\mathbf{v}_n$ becomes the current string. Otherwise, no local improvement is possible: the algorithm has reached a local or global optimum ($local = TRUE$). In such a case, the next iteration ($t \leftarrow t + 1$) of the algorithm is executed with a new current string selected at random.

The success or failure of a single iteration (i.e., one complete climb) of the above hill-climbing algorithm is determined completely by the initial point. For problems with many local optima, particularly those where these optima have large *basins of attraction*, it's often very difficult to locate a globally optimal solution.

³The term *hill-climbing* implies a maximization problem, but the equivalent *descent* method is easily envisioned for minimization problems. For convenience, the term hill-climbing will be used here to describe both methods without any implied loss of generality.

```

procedure iterated hill-climber
begin
   $t \leftarrow 0$ 
  initialize  $best$ 
  repeat
     $local \leftarrow \text{FALSE}$ 
    select a current point  $\mathbf{v}_c$  at random
    evaluate  $\mathbf{v}_c$ 
    repeat
      select all new points in the neighborhood of  $\mathbf{v}_c$ 
      select the point  $\mathbf{v}_n$  from the set of new points
        with the best value of evaluation function  $eval$ 
      if  $eval(\mathbf{v}_n)$  is better than  $eval(\mathbf{v}_c)$ 
        then  $\mathbf{v}_c \leftarrow \mathbf{v}_n$ 
        else  $local \leftarrow \text{TRUE}$ 
    until  $local$ 
     $t \leftarrow t + 1$ 
    if  $\mathbf{v}_c$  is better than  $best$ 
      then  $best \leftarrow \mathbf{v}_c$ 
  until  $t = MAX$ 
end

```

Fig. 2.5. A simple iterated hill-climber

Hill-climbing algorithms have several weaknesses:

- They usually terminate at solutions that are only locally optimal.
- There is no information as to the amount by which the discovered local optimum deviates from the global optimum, or perhaps even other local optima.
- The optimum that's obtained depends on the initial configuration.
- In general, it is *not* possible to provide an upper bound for the computation time.

On the other hand, there is one alluring advantage for hill-climbing techniques: they're very easy to apply! All that's needed is the representation, the evaluation function, and a measure that defines the neighborhood around a given solution.

Effective search techniques provide a mechanism for balancing two apparently conflicting objectives: *exploiting* the best solutions found so far and at the same time *exploring* the search space.⁴ Hill-climbing techniques exploit the best available solution for possible improvement but neglect exploring a large

⁴This balance between exploration and exploitation was noted as early as the 1950s by the famous statistician G.E.P. Box [54].

portion of the search space $\mathcal{S}$. In contrast, a random search — where points are sampled from $\mathcal{S}$ with equal probabilities — explores the search space thoroughly but foregoes exploiting promising regions of the space. Each search space is different and even identical spaces can appear very different under different representations and evaluation functions. So there's no way to choose a single search method that can serve well in every case. In fact, this can be proved mathematically [499, 156].

Getting stuck in local optima is a serious problem. It's one of the main deficiencies that plague industrial applications of numerical optimization. Almost every solution to real-world problems in factory scheduling, demand forecasting, land management, and so forth, is at best only locally optimal.

What can we do? How can we design a search algorithm that has a chance to escape local optima, to balance exploration and exploitation, and to make the search independent from the initial configuration? There are a few possibilities, and we'll discuss some of them in chapter 5, but keep in mind that the proper choices are always problem dependent. One option, as we discussed earlier, is to execute the chosen search algorithm for a large number of initial configurations. Moreover, it's often possible to use the results of previous trials to improve the choice of the initial configuration for the next trial. It might also be worthwhile to introduce a more complex means for generating new solutions, or enlarge the neighborhood size. It's also possible to modify the criteria for accepting transitions to new points that correspond with a *negative* change in the evaluation function. That is, we might want to accept a worse solution from the local neighborhood in the hope that it will eventually lead to something better.

But before we move to more sophisticated search methods, we need to review the basic, classic problem-solving techniques, such as dynamic programming, the A^* algorithm, and a few others. Before we do that, you get another chance to display your prowess by solving an interesting problem in geometry.

2.7 Can you sink this trick shot?

Now that you've seen a number of different search strategies and have been introduced to the issues of choosing a representation, the objective, and an evaluation function, let's see if you can solve this challenging puzzle. Remember, you might have to rely on some information that we presented in an earlier section — or maybe not! We're not telling!

A single ball rests calmly on a square-shaped billiard table. The ball is hit and it moves on the table for an infinite amount of time, obeying an elementary law of physics: The angles α and β (figure 2.6) are always equal. Under what circumstances will the movement of the ball be cyclic?

Please put the book aside and think about it.

One way we might approach the problem is to restrict our attention to the surface of the billiard table and try to reason “within” this square. It's very

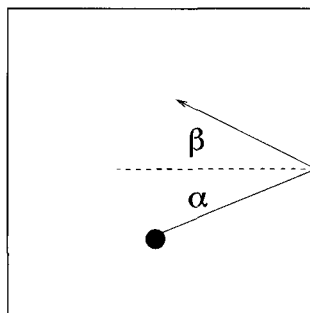


Fig. 2.6. A square-shaped billiard table and the movement of the ball

difficult to see any pattern of the ball's movement there, however, as the number of reflections of the ball might be very high (infinitely high in the limit), and the ball's path may consist of many segments.

Let's "expand" the space for reasoning about the problem. Instead of imagining the ball to reflect off of each side of the billiard table, let's imagine the ball as if it were traveling in a straight line and consider a *reflection of the table!* Figure 2.7 illustrates the concept.

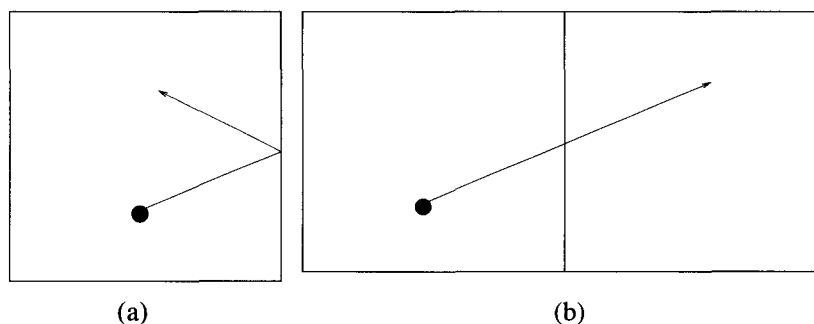


Fig. 2.7. (a) Reflection of a ball on the billiard table, and (b) the corresponding reflection of the billiard table

This is a significant departure from the way we solve most problems. We usually think about manipulating objects within a framework. However, it's possible to change the orientation and think about manipulating the framework that surrounds the objects. If you made this leap without cheating and reading ahead, then congratulations, your future is bright!

Each time the ball strikes a wall of the table, we can draw a reflection of the table against that wall. Note that now the ball can only strike the top or right wall, and it will cycle if it returns to its initial position (see figure 2.8).

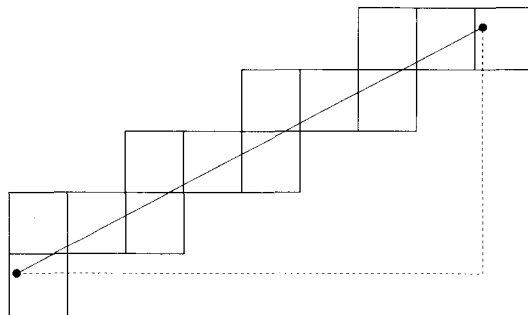


Fig. 2.8. Getting a cycle. The ball returns to its initial position on a subsequent reflection of the billiard table.

Suppose that the ball does cycle. This means that it “traveled” some number of tables “up” (say, p tables) and some number of tables “right” (say, q tables). For angle $\alpha = 0$, $p = 0$. For $\alpha = \pi/2$, $q = 0$. For $0 < \alpha < \pi/2$, $p > 0$ and $q > 0$.

With this in mind we can answer the question. The ball will cycle on the billiard table if and only if either $\alpha = \pi/2$ or the tangent of the angle α is a rational number (i.e., $\tan(\alpha) = p/q$ for $0 \leq \alpha < \pi/2$).

In the first case this is clear: The ball will just move up and down forever. In the second case, if it’s not clear, then remember that in a right triangle, the tangent of an acute angle is equal to the length of the opposite side over the adjacent side. Take a look at figure 2.8 and imagine making the path of the ball corresponding to the hypotenuse of a right triangle. The base is eight tables long and the height is four tables. To reach the identical point on some future reflected table, the ball will have had to travel some whole, even number of tables up and right (the numbers must be even to preserve the ball’s original angle of motion). Thus the tangent of the angle α must be a rational number. And in the case that α is equal to zero, then the ball simply moves horizontally *ad infinitum*.

Remembering “SOHCAHTOA” from high school trigonometry is really the easy part of this exercise.⁵ The hard part is to avoid getting trapped in the description of the problem which lures you into using a framework that doesn’t help you find the answer. It’s all the more challenging to escape this trap because in real life, billiard tables don’t move and billiard balls do! But in solving problems, you have to learn to “think outside the box” — or outside the billiard table.

⁵SOHCAHTOA is a common mnemonic device for remembering that, in a right triangle, the Sine of an acute angle is the length of the Opposite side over the Hypotenuse, the Cosine is the length of the Adjacent side over the Hypotenuse, and the Tangent is the length of the Opposite side over the Adjacent side.

2.8 Summary

In order to implement a search algorithm you must consider

- The representation.
- The objective.
- The evaluation function.

The representation is how the computer stores your candidate solutions and the objects that it manipulates in searching for new ones. The objective is what you are trying to accomplish. The evaluation function is a means for measuring the quality of any potential solution to the problem. Taken together, these three elements form a strategic troika. You have to give careful consideration to each facet.

Sometimes it's useful to look at a search space in geometric terms, thinking about the neighborhood that surrounds a particular solution and whether or not we can find better solutions within that neighborhood. If we can't, then we've discovered a local optimum. It might be that this solution is really a global optimum; that is, no matter what size neighborhood we'd pick, the solution would still be the best. It's often too computationally expensive to expand the search beyond small neighborhoods. As a result, we face a danger of getting trapped in local optima, particularly when using a hill-climbing method.

Finally, when framing a problem, don't be constrained by the physics of the real world. Allow yourself the freedom to ask questions like: what if it was the billiard table that was moving relative to the ball? What if I went outside the plane of the table to construct a pyramid with these match sticks?

III. What Are the Prices in 7–11?

There are many ways to search for the answer to a problem. To be effective, particularly when the domain of possible solutions is very large, you have to organize the search in a systematic way. What's more, the structure of your search has to in some way "match" the structure of the problem. If it doesn't, then your search may perform even worse than random guesswork.

Some of the heuristics for searching the spaces of solutions involve pruning out those cases that can't possibly be correct. In this way, the size of the total space is reduced and with luck, you'll be able to more quickly zoom in on feasible solutions. So think about how you might go about eliminating the impossibilities in this perplexing case of prices in the so-called "7–11" problem, which goes like this.

There's a chain of stores in the United States called 7–11. They're probably called this because they used to be open from 7 a.m. until 11 p.m., but now they're usually open all the time. One day a customer arrived in one of these 7–11 shops and selected four items. He then approached the counter to pay for these items. The salesman took his calculator, pressed a few buttons and said, "The total price is \$7.11."

The customer tried a joke, "Why? Do I have to pay \$7.11 because the name of your shop is 7–11?"

The salesman didn't get the joke and answered, "Of course not! I have multiplied the prices of these four items and I have just given you the result!"

The customer was very surprised. "Why did you multiply these numbers? You should have added them to get the total price!"

The salesman said, "Oh, yes, I'm sorry, I have a terrible headache and I pressed the wrong button!"

Then the salesman repeated all the calculations, i.e., he added prices of these four items, but to his and customer's great surprise, the total was still \$7.11.

Now, the task is to find the prices of these four items!

This is a typical search problem. We have four variables, x , y , z , and t , that represent the prices of four items. Each variable takes a value from its domain, and in this case every domain is the same: from 0.01 up to 7.08. The smallest price might be 0.01 and since the total price for all four items is 7.11, the largest possible price of any single item is 7.08. Thus, the domain of each variable is the set

$$\{0.01, 0.02, 0.03, \dots, 7.06, 7.07, 7.08\}.$$

To avoid dealing with fractional numbers, let's change its formulation into integers. We'll talk about prices in cents rather than in dollars and cents. In that way the domain of each variable is the set of integer numbers

$$P = \{i \in N : 1 \leq i \leq 708\}.$$

The problem is then to find x, y, z , and t , all from the set P , such that

$$\begin{aligned} x + y + z + t &= 711, \text{ and} \\ xyzt &= 711000000. \end{aligned}$$

Note that during the conversion from dollars and cents into just cents, each variable was multiplied by 100. Thus their product 7.11 increased 10^8 times, resulting in $7.11 \cdot 10^8 = 711000000$.

As $711000000 = 2^6 \cdot 3^2 \cdot 5^6 \cdot 79$, there are many cases to consider. Exhaustively searching every possible combination would give us a solution, but it would also take some time — a great deal of time. Let's investigate some shortcuts that are similar to what's called a *branch and bound* approach.

One thing to think about here, particularly because we have two equations where the second one is a product of terms, is the idea of factoring. We can factor 711000000 into its prime factors. This might help because the variables in question would have to be multiples of those factors. In this case, 79 is the largest prime factor of 711000000. It's clear then that the price (in cents, of course) of one of the items (say, x), is divisible by 79.

Let's consider a few cases that revolve around 79. We choose 79 because, as the largest prime factor of 711000000, it means we have to consider the minimum number of cases.

- If $x = 79$, the problem for the remaining three variables is

$$\begin{aligned} y + z + t &= 632, \text{ and} \\ yzt &= 2^6 \cdot 3^2 \cdot 5^6. \end{aligned}$$

Here, it's impossible that all of variables y , z , and t are divisible by 5 (as their total is not), so the product of two of the variables (say, y and z) is divisible by $5^6 = 15625$. Now we have several cases:

- Case (a): $y = 15625y'$
- Case (b): $y = 3125y'$ and $z = 5z'$
- Case (c): $y = 625y'$ and $z = 25z'$
- Case (d): $y = 125y'$ and $z = 125z'$

for some $y' \geq 1$ and $z' \geq 1$. For cases (a)–(c), $y + z > 632$ (which is impossible), so the only case we really have to consider is case (d). In this case, the problem becomes:

$$\begin{aligned} 125(y' + z') + t &= 632, \text{ and} \\ y' \cdot z' \cdot t &= 2^6 \cdot 3^2. \end{aligned}$$

The first equation implies that $y' + z' \leq 5$, so all we have to do now is consider six additional possibilities because we can assume that $y' \leq z'$:

- Case (d1): $y' = 1$ and $z' = 1$
- Case (d2): $y' = 1$ and $z' = 2$
- Case (d3): $y' = 1$ and $z' = 3$
- Case (d4): $y' = 1$ and $z' = 4$
- Case (d5): $y' = 2$ and $z' = 2$
- Case (d6): $y' = 2$ and $z' = 3$.

None of these six cases gives a solution. For example, for case (d1) we have

$$\begin{aligned} x &= 79, \\ y &= 125 \cdot y' = 125, \text{ and} \\ z &= 125 \cdot z' = 125, \end{aligned}$$

which requires $t = 382$ (so four variables would sum to 711), but then their product wouldn't be equal to 711000000.

- If $x = 2 \cdot 79 = 158$, the problem is

$$\begin{aligned} y + z + t &= 553, \text{ and} \\ yzt &= 2^5 \cdot 3^2 \cdot 5^6. \end{aligned}$$

In this case we can repeat the same reasoning as we did with the previous case: y must be $125y'$ and $z = 125z'$, and

$$125(y' + z') + t = 553,$$

thus $y' + z' \leq 4$. None of the possibilities

- $y' = 1$ and $z' = 1$
- $y' = 1$ and $z' = 2$
- $y' = 1$ and $z' = 3$
- $y' = 2$ and $z' = 2$

leads to a solution either though.

It's an easy exercise to see that if

- $x = 3 \cdot 79 = 237$
- $x = 5 \cdot 79 = 395$
- $x = 6 \cdot 79 = 474$
- $x = 7 \cdot 79 = 553$
- $x = 8 \cdot 79 = 632$,

the problem has no solutions. In each of these cases, the number of subcases to consider shrinks. For example, if $x = 553$, then for $y = 125y'$ and $z = 125z'$ we have $125(y' + z') + t = 158$, which gives an immediate contradiction because $y' + z' \geq 2$.

If the problem actually has a solution, then $x = 316$. In this (final) case, the problem becomes

$$y + z + t = 395, \text{ and}$$

$$yzt = 2^4 \cdot 3^2 \cdot 5^6.$$

Again, we have to consider a few cases. Since the total of the three remaining variables is divisible by 5, either one or all three variables are divisible by 5. If only one variable (say, y) is divisible by 5, then $y = 5^6 y'$ and we get a contradiction, $y > 395$. Thus, we know that all of the variables y , z , and t are divisible by 5:

$$y = 5y', \quad z = 5z', \quad \text{and} \quad t = 5t'.$$

Now the problem becomes

$$y' + z' + t' = 79, \text{ and}$$

$$y'z't' = 2^4 \cdot 3^2 \cdot 5^3.$$

As $y' + z' + t' = 79$, one of these primed variables is not divisible by 5, so some other variable must be divisible by 25, say, $y' = 25y''$. In that case we have

$$25y'' + z' + t' = 79, \text{ and}$$

$$y''z't' = 2^4 \cdot 3^2 \cdot 5 = 720.$$

This means that y'' is either 1, or 2, or 3 (otherwise $25y'' > 79$). The second two choices don't lead to solution (e.g., if $y'' = 2$, $z' + t' = 29$ and $z't' = 360$, which doesn't have solution in the real numbers, and things are "complex" enough, aren't they?). Thus $y'' = 1$ (i.e., $y = 125$). In that case we get

$$z' + t' = 54, \text{ and}$$

$$z't' = 720,$$

which gives us $z' = 24$ and $t' = 30$, or the other way around. Thus $z = 5 \cdot 24 = 120$ and $t = 5 \cdot 30 = 150$.

That solves the problem. Indeed, it has a unique solution (we've converted the prices back into dollars and cents):

$$x = \$3.16, \quad y = \$1.25, \quad z = \$1.20, \quad \text{and} \quad t = \$1.50.$$

It's important to point out the systematic approach to solving this problem. Step by step, we eliminated the impossible solutions and even more importantly, we started by pruning the space of the solution dramatically by examining the prime factors of the product of the prices.

Of course, it's possible to use a different approach. It's also possible to do some guesswork. Usually some guesswork is desirable as it may provide insight into a problem. We'll see this more clearly in the next chapter. It's also worthwhile to observe that simply guessing may lead to some amazing "discoveries," like getting a set of prices

$$(\$3.16, \$1.25, \$1.25, \$1.44) \text{ or } (\$2.37, \$2.00, \$2.00, \$0.75).$$

For both sets, the product is \$7.11, however, the former set results in a sum of \$7.10, while for the latter, the sum is \$7.12. The problem with guessing is that it doesn't often lead you to the next solution, which is hopefully better than the last. The concept of generating improvements in the quality of a solution is intrinsic to a great many classic and modern methods of optimization. And hey, it's comforting to know that things are getting better!

3. Traditional Methods — Part 1

... are all alike in their promises.
It is only in their deeds that they differ.

Molière, *The Miser*

There are many classic algorithms that are designed to search spaces for an optimum solution. In fact, there are so many algorithms that it's natural to wonder why there's such a plethora of choices. The sad answer is that none of these traditional methods is robust. Every time the problem changes you have to change the algorithm. This is one of the primary shortcomings of the well-established optimization techniques. There's a method for every problem — the problem is that most people only know one method, or maybe a few. So they often get stuck using the wrong tool to attack their problems and consequently generate poor results.

The classic methods of optimization can be very effective when appropriate to the task at hand. It pays for you to know when and when not to use each one. Broadly, they fall into two disjoint classes:

- Algorithms that only evaluate complete solutions.
- Algorithms that require the evaluation of partially constructed or approximate solutions.

This is an important difference. Whenever an algorithm treats complete solutions, you can stop it at any time and you'll always have at least one potential answer that you can try. In contrast, if you interrupt an algorithm that works with partial solutions, you might not be able to use any of its results at all.

Complete solutions mean just that: all of the decision variables are specified. For example, binary strings of length n constitute complete solutions for an n -variable SAT. Permutations of n cities constitute complete solutions for a TSP. Vectors of n floating-point numbers constitute complete solutions for an NLP. We can easily compare two complete solutions using an evaluation function. Many algorithms rely on such comparisons, manipulating one single complete solution at a time. When a new solution is found that has a better evaluation than the previous best solution, it replaces that prior solution. Examples include exhaustive search, local search, hill climbing, and gradient-based numerical optimization methods. Some modern heuristic methods such as simulated annealing, tabu search, and evolutionary algorithms also fall into this class as well (see chapters 5 and 6).

Partial solutions, on the other hand, come in two forms: (1) an incomplete solution to the problem originally posed, and (2) a complete solution to a reduced (i.e., simpler) problem.

Incomplete solutions reside in a subset of the original problem's search space. For example, in an SAT, we might consider all of the binary strings where the first two variables were assigned the value 1 (i.e., TRUE). Or in the TSP, we might consider every permutation of cities that contains the sequence 7 – 11 – 2 – 16. Similarly, for an NLP, we might consider all of the solutions that have $x_3 = 12.0129972$. In each case, we fix our attention on a subset of the search space that has a particular property. Hopefully, that property is also shared by the real solution!

Alternatively, we can often decompose the original problem into a set of smaller and simpler problems. The hope is that in solving each of these easier problems, we can eventually combine the partial solutions to get an answer for the original problem. In a TSP, we might only consider k out of n cities and try to establish the shortest path from city i to city j that passes through all k of these cities (see section 4.3). Or, in the NLP, we might limit the domains of some variables x_i , thereby reducing the size of the search space significantly, and then search for a complete solution within that restricted domain. Such partial solutions can sometimes serve as building blocks for the solution to the original problem.

Making a complex problem simpler by dividing it up into little pieces that are easy to manage sounds enticing. But algorithms that work on partial solutions pose additional difficulties. You have to: (1) devise a way to organize the subspaces so that they can be searched efficiently, and (2) create a new evaluation function that can assess the quality of partial solutions. This latter task is particularly challenging in real-world problems: what is the value of a remote control channel changer? The answer depends on whether or not there is a television to go along with it! The utility of the remote control device depends heavily on the presence or absence of other items (e.g., a television, batteries, perhaps a cable connection). But when you evaluate a partial solution you intrinsically cannot know what the other external conditions will be like. At best, you can try to estimate them, but you could always be wrong.

The former task of organizing the search space into subsets that can be searched efficiently is also formidable, but many useful ideas have been offered. You might organize the search space into a tree, where complete solutions reside at the leaves of the tree. You could then progress down each branch in the tree, constructing the complete solution in a series of steps. You might be able to eliminate many branches where you know the final solutions will be poor. And there are other ways to accomplish this sort of “branch and bound” procedure. Each depends on how you arrange the solutions in the search space, and that in turn depends on your representation.

Remember, there's never only one way to represent different solutions. We've seen three common representations for the SAT, TSP, and NLP: (1) binary strings, (2) permutations, and (3) floating-point vectors, respectively. But we

don't have to limit ourselves to these choices. We could, for example, represent solutions to an NLP in binary strings. Each solution would then be of length kn , where n still represents the number of variables in the problem, and k indicates the number of bits used to describe a value for each variable. It's straightforward to map a binary string $\langle b_{k-1} \dots b_0 \rangle$ into a floating-point value x_i from some range $[l_i, u_i]$ in two steps:

- Convert the binary string $\langle b_{k-1} \dots b_0 \rangle$ from base 2 to base 10:

$$(\langle b_{k-1} \dots b_0 \rangle)_2 = (\sum_{i=0}^{k-1} b_i \cdot 2^i)_{10} = x'.$$

- Find a corresponding floating-point value x within the required range:

$$x = l_i + x' \cdot \frac{u_i - l_i}{2^k - 1}.$$

For example, if the range were $[-2, 3]$ and we chose to use binary strings of length 10, then the string $\langle 1001010001 \rangle$ would map to 0.89833822 as:

$$0.89833822 = -2 + 593 \cdot \frac{5}{1023}.$$

Note that as we increase the value of k , we generate more precision in approximating a particular floating-point value. Here, with 10 bits, the successor to $\langle 1001010001 \rangle$ is $\langle 1001010010 \rangle$, which maps to 0.90322581 — the gap between floating-point numbers is almost 0.005. If we needed more precision, we'd have to increase the value of k . This would also increase the computation time because we'd have to decode longer binary strings each time we wanted to determine the value of a particular variable in the NLP. But there's nothing that forces us to solve NLPs with floating-point values directly, even though that's the domain over which they're defined.

Actually, this freedom to choose the representation allows you to become really creative. If we're solving a TSP, we might use the straightforward permutation representation we've discussed earlier. But even then, we might consider interpreting that permutation in a little different manner. For instance, suppose the i -th element of the permutation is a number in the range from 1 to $n - i + 1$ and indicates a city from the remaining part of some reference list of cities C for a given TSP. Say $n = 9$ and the reference list C is

$$C = (1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9).$$

Then the following tour,

$$5 - 1 - 3 - 2 - 8 - 4 - 9 - 6 - 7,$$

is represented as a vector,

$$(5 \ 1 \ 2 \ 1 \ 4 \ 1 \ 3 \ 1 \ 1).$$

This is interpreted as follows: (1) take the fifth city from the current reference list C and remove it (this would be city 5); (2) take the first city from the current reference list C and remove it (this is city 1); (3) take the second city from the current reference list C and remove it (this is now city 3); and so on.

This representation seems to be only remotely connected to the problem itself, but you can indeed use it to identify every possible tour, and perhaps it could offer some interesting possibilities when it comes to organizing the search space that wouldn't be found in the typical permutation representation (see chapter 8 for a discussion on other possible representations for the TSP).

There are lots of possible representations even for the SAT. We might represent a solution as a vector of floating-point values from the range $[-1, 1]$, and then assign the condition TRUE to each Boolean variable if its corresponding floating-point variable is non-negative and FALSE otherwise (see, for example, section 9.1 of this book).

Whatever representation you choose for a particular problem, it's possible to consider the implied search space as a set of complete solutions and search through them in a particular manner. You can also divide the space into subsets and work with partial or incomplete solutions. Since there are so many different algorithms to examine, we've divided them up. We'll look at those that work with complete solutions in this chapter and then turn our attention to those that work with partial or incomplete solutions in chapter 4.

3.1 Exhaustive search

As the name implies, exhaustive search checks each and every solution in the search space until the best global solution has been found. That means if you don't know the value that corresponds to the evaluated worth of the best solution, there's no way to be sure that you've found that best solution using exhaustive search unless you examine everything. No wonder it's called *exhaustive*. You'll be tired and old before you get an answer to problems of even modest size! Remember, the size of the search space of real-world problems is usually enormous. It might require centuries or more of computational time to check every possible solution. From section 1.1, we saw that a 50-city TSP generates 10^{62} different possible tours. You'd be waiting a very long time for an exhaustive search of that space!

But exhaustive algorithms (also called *enumerative*) are interesting in some respects. At the very least, they are simple. The only requirement is for you to generate every possible solution to the problem systematically. In addition, there are ways to reduce the amount of work you have to do. One such method is called *backtracking* and we'll take a closer look at that later. What's more, some classic optimization algorithms that construct a complete solution from partial solutions (e.g., branch and bound or the A^* algorithm) are based on an exhaustive search, so let's at least give a little bit more time to this approach.

The basic question to answer when applying an exhaustive search is: How can we generate a sequence of every possible solution to the problem? The order in which the solutions are generated and evaluated is irrelevant because the plan is to evaluate every one of them. The answer for the question depends on the selected representation.

3.1.1 Enumerating the SAT

To enumerate the search space for the SAT with n variables, we have to generate every possible binary string of length n . There are 2^n such strings, from $\langle 0\dots 000 \rangle$ to $\langle 1\dots 111 \rangle$. All of these binary strings correspond to whole numbers in a one-to-one mapping (see table 3.1).

Table 3.1. The correspondence between a binary string of length $n = 4$ and its decimal equivalent

0000	0	0100	4	1000	8	1100	12
0001	1	0101	5	1001	9	1101	13
0010	2	0110	6	1010	10	1110	14
0011	3	0111	7	1011	11	1111	15

All we do is generate the non-negative integers from 0 to $2^n - 1$. We then convert each integer into the matching binary string of length n and the bits of that string are taken as the truth assignments of the decision variables.

Another possibility is to operate on the binary strings directly and add a binary one $\langle 0\dots 001 \rangle$ to get each string in succession. Once a new string is generated, it's evaluated and the string with the best score is kept as the best solution found so far. This still leaves us with the problem of choosing an evaluation function, which could simply be

- 1, if the string satisfies the compound Boolean statement, and
- 0, otherwise.

In this case, we could actually halt the search whenever the string at hand had a corresponding evaluation of 1.

Even in this simple case we can take advantage of an opportunity to organize the search and prune the number of alternative strings that we really *need* to examine. Suppose we partition the search space of the SAT into two disjoint subspaces: the first contains all of the strings such that $x_1 = F$ (FALSE), whereas the second contains all of the vectors where $x_1 = T$ (TRUE). We could then organize the entire space as a tree (see figure 3.1).

On the next level down in the tree we again divide the space into two disjoint subspaces based on the assignment to x_2 , and continue like this for all n variables. The resulting binary tree has n levels and the i -th variable is assigned on the i -th level.

Having organized the space into a tree like this, we can use an exhaustive search technique that is designed especially for trees: depth-first search. This simple recursive procedure (figure 3.2) traverses a directed graph (a tree, in particular) starting from some node and then searches adjacent nodes recursively.

The advantage of organizing the search in this way is that sometimes we can deduce that the remaining branch below a particular node can lead to nothing

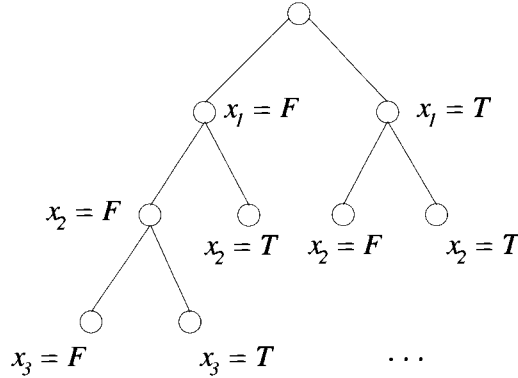


Fig. 3.1. A binary search tree for the SAT

```

procedure depth-first( $v$ )
begin
  visit  $v$ 
  for each child  $w$  of  $v$  do
    dept-first( $w$ )
end

```

Fig. 3.2. Depth-first search

but a dead end. We can then “backtrack” to the parent of this node at the next higher level and proceed further with the search in another branch. For example, it might happen that after assigning truth values to a few variables, the whole SAT formula evaluates to FALSE. In that case, there’s no need to go further down that branch. Thus, if the SAT formula contains a clause

$$\dots \wedge (x_1 \vee \bar{x}_3 \vee x_4) \wedge \dots,$$

and the assignments of the first four variables are

$$x_1 = \text{FALSE}; x_2 = \text{TRUE}; x_3 = \text{TRUE}; x_4 = \text{FALSE}, \text{ or}$$

$$x_1 = \text{FALSE}; x_2 = \text{FALSE}; x_3 = \text{TRUE}; x_4 = \text{FALSE},$$

then no matter what we assign to x_5 or any other variable, the whole SAT formula will evaluate to FALSE. It’s time to pick another branch.

Depth-first search eliminates parts of the search space by checking whether or not a given node (subspace) can contain a solution. If it can, then the search proceeds further, but if it can’t, the search returns to the parent of the given node. This backtracking is a common practice in many tree-based algorithms. We’ll see a similar idea later in chapter 4 when discussing branch and bound methods (section 4.4).

3.1.2 Enumerating the TSP

Applying an exhaustive algorithm to the TSP isn't as straightforward. Here the question is again first and foremost: How can we generate all of the possible permutations? The primary difficulty is that unless the TSP is "fully connected," some permutations might not be feasible. City i might not be connected to city j , at least not directly, so every permutation where these two cities are adjacent would be illegal. We might get around this obstacle by allowing these infeasible connections anyway, but then giving any such infeasible solution a very high cost, much higher than the length of any legal tour. In that case, the best solution among all of the permutations would also be the best legal solution, provided that one exists.

The second problem is the simple nuts and bolts of generating all possible permutations of n numbers. Let's consider a few possibilities. We could use an approach that was similar to what we used for the SAT, where there was a mapping between the whole numbers and binary strings. Since there are exactly $n!$ different permutations of n numbers, we can define a function that ranks all permutations from 0 to $n! - 1$ as follows. Note that any integer k between 0 and $n! - 1$ can be uniquely represented by an array of integers:

$c[n], \dots, c[2]$, where $0 \leq c[i] \leq i - 1$ (for $i = 2, \dots, n$), as $k = \sum_{i=2}^n c[i] \cdot (i - 1)!$.

To generate a permutation corresponding to k (or an array $c[i]$), we can use the following procedure [426]:

```

for  $i = 1$  to  $n$  do
     $P[i] \leftarrow i$ 
for  $i = n$  step  $-1$  to  $2$  do
    swap ( $P[i], P[c[i] + 1]$ )

```

The above method can be used in various ways. In particular, instead of generating permutations for consecutive numbers (from 0 to $n! - 1$), we could generate these integers randomly; however, if we do that then we also have to keep track of the numbers that we've already generated (e.g., by maintaining a binary vector of length $n!$).

Another possibility [425] is to use the following recursive procedure (figure 3.3). The procedure `gen1_permutation`, when called with k initialized to -1 , the parameter i set to 0, and all of the entries of the array P initialized to 0, prints every permutation of $(1, \dots, n)$. It does this by fixing 1 in the first position and generating the remaining $(n - 1)!$ permutations of numbers from 2 to n . The next $(n - 1)!$ permutations are then listed with 1 in the second position, and so forth. For $n = 3$, the procedure produces the following output: (1 2 3), (1 3 2), (2 1 3), (3 1 2), (2 3 1), (3 2 1). Of course, in real applications, we don't usually print the current permutation but instead do something useful with it (e.g., evaluate the sum of all distances between all of the cities in the permutation).

The problem with the previous methods is that they aren't very practical. If you try any of them for $n > 20$ you'll see why! A more fruitful approach

```

procedure gen1_permutation( $i$ )
begin
   $k \leftarrow k + 1$ 
   $P[i] \leftarrow k$ 
  if  $k = n$  then
    for  $q = 1$  to  $n$  do
      print  $P[q]$ 
    for  $q = 1$  to  $n$  do
      if  $P[q] = 0$  then gen1_permutation( $q$ )
   $k \leftarrow k - 1$ 
   $P[i] \leftarrow 0$ 
end

```

Fig. 3.3. A procedure to generate all permutations of integer numbers from 1 to n

relies on transforming one permutation into another new one (this is the easy part) and doesn't repeat any permutation until all of them have been generated (that's the tricky part). The advantage of this approach is that there's no need to keep track of which permutations have already been generated. The current permutation always implies the next one! Many algorithms can perform this task, and you'll find one that prints one permutation at a time in figure 3.4 [80].

Another possibility that is specific to the TSP is to enumerate all possible permutations by successively exchanging adjacent elements. This saves a significant amount of time because all that you have to do in order to calculate the cost of each new permutation is determine the incremental effect of the exchange. The rest of the permutation remains the same so you don't have to recalculate its cost. You can find more information on other algorithms to generate permutations in [426], as well as some hints on how to implement them efficiently.

3.1.3 Enumerating the NLP

Strictly speaking, it isn't possible to search exhaustively in continuous domains because there is an infinite number of alternatives, but it is possible to divide the domain of each continuous variable into a finite number of intervals and consider the Cartesian product of these intervals. For example, when finding the maximum of a function $f(x_1, x_2)$ where $x_1 \in [-1, 3]$ and $x_2 \in [0, 12]$, we can divide the range of x_1 into 400 intervals, each of length 0.01, and divide the range of x_2 into 600 intervals, each of length 0.02. This creates $400 \times 600 = 240,000$ cells that we can search enumeratively. The merit of each cell can be defined by applying the evaluation function to a particular corner of the cell, or perhaps the midpoint. Once the cell with the best merit is selected we have the choice of going back and doing the same thing all over again within that cell.

```

procedure gen2_permutation( $I, P, n$ )
  begin
    if not  $I$  then
      begin
        for  $i \leftarrow 0$  step 1 until  $n$  do
           $x[i] \leftarrow 0$ 
           $x[n] \leftarrow -1$ 
          goto  $E$ 
        end
        for  $i \leftarrow n$  step  $-1$  until 0 do
          begin
            if  $x[i] \neq i$  then goto  $A$ 
             $x[i] \leftarrow 0$ 
          end
           $P[0] \leftarrow -1$ 
          goto  $E$ 
         $A:$   $x[i] \leftarrow x[i] + 1$ 
           $P[0] = 0$ 
          for  $i \leftarrow 1$  step 1 until  $n$  do
            begin
               $P[i] \leftarrow P[i - x[i]]$ 
               $P[i - x[i]] \leftarrow i$ 
            end
          end
         $E:$  end

```

Fig. 3.4. This procedure produces all permutations of integers from 0 to n . Upon entry with $I = \text{FALSE}$, the procedure initializes itself producing no permutation. Upon each successive entry into the procedure with $I = \text{TRUE}$, a new permutation is stored in $P[0]$ through $P[n]$. A sentinel is set, $P[0] = -1$, when the process has been exhausted.

This is a form of exhaustive search in the sense that we're trying to "cover" all possible solutions. But it isn't really exhaustive since only one solution of the many solutions in each cell is considered and evaluated. The size of the cells is important here: the smaller the cell, the better our justification for calling this method "exhaustive." But the disadvantages of the approach are also important:

- Using a fine granularity of the cells increases the total number of cells significantly. If the domain of the first variable were divided into 4,000 intervals, each of length 0.001, and the domain of the second variable were divided into 6,000 ranges, each of length 0.002, the total number of cells would grow by two orders of magnitude from 240,000 to 24,000,000.
- A low granularity of the cells increases the probability that the best solution will not be discovered, i.e., the cell that contains the best solution wouldn't have the best merit because we'd be looking at a level that was too coarse.

- When facing a large number of dimensions (variables), the method becomes impractical because the number of cells can be very large. An optimization problem with 50 variables, each divided into only 100 intervals, yields 10^{100} cells.

When you face a small problem and have the time to enumerate the search space, you can be guaranteed to find the best solution. Don't consider this approach for larger problems, however, because you'll simply never complete the enumeration.

3.2 Local search

Instead of exhaustively searching the entire space of possible solutions, we might focus our attention within a local neighborhood of some particular solution. This procedure can be explicated in four steps:

1. Pick a solution from the search space and evaluate its merit. Define this as the *current* solution.
2. Apply a transformation to the current solution to generate a new solution and evaluate its merit.
3. If the new solution is better than the current solution then exchange it with the current solution; otherwise discard the new solution.
4. Repeat steps 2 and 3 until no transformation in the given set improves the current solution.

The key to understanding how this *local search* algorithm works lies in the type of transformation applied to the current solution. At one extreme, the transformation could be defined to return a potential solution from the search space selected uniformly at random. In this case, the current solution has no effect on the probabilities of selecting any new solution, and in fact the search becomes essentially enumerative. Actually, it's possible for this search to be even worse than enumeration because you might resample points that you've already tried. At the other extreme lies the transformation that always returns the current solution — and this gets you nowhere!

From a practical standpoint, the right thing to do lies somewhere in between these extremes. Searching within some local neighborhood of the current solution is a useful compromise. In that way, the current solution imposes a bias on where we can search next, and when we find something better we can update the current point to this new solution and retain what we've learned. If the size of the neighborhood is small then we might be able to search that neighborhood very quickly, but we also might get trapped at a local optimum. In contrast, if the size of the neighborhood is very large then we'll have less chance of getting stuck, but the efficiency of the search may suffer. The type of

transformation that we apply to the current solution determines the size of the neighborhood, so we have to choose this transformation wisely in light of what we know about the evaluation function and our representation.

3.2.1 Local search and the SAT

Local search algorithms are surprisingly good at finding satisfying assignments for certain classes of SAT formulas [428]. One of the best-known (randomized) local search algorithms for the SAT (given in conjunctive normal form) is the GSAT procedure shown in figure 3.5.

```

procedure GSAT
begin
  for  $i \leftarrow 1$  step 1 until MAX-TRIES do
    begin
       $T \leftarrow$  a randomly generated truth assignment
      for  $j \leftarrow 1$  step 1 until MAX-FLIPS do
        if  $T$  satisfies the formula then return( $T$ )
        else make a flip
      end
    return('no satisfying assignment found')
  end

```

Fig. 3.5. The GSAT procedure. The statement “make a flip” flips the variable in T that results in the largest decrease in the number of unsatisfied clauses.

GSAT has two parameters: MAX-TRIES, which determines the number of new search sequences (trials), and MAX-FLIPS, which determines the maximum number of moves per try.

GSAT begins with a randomly generated truth assignment. If we happen to be lucky and this assignment satisfies the problem, the algorithm terminates. Rarely are we so lucky. At the next step, it flips each of the variables from TRUE to FALSE or FALSE to TRUE in succession and records the decrease in the number of unsatisfied clauses. After trying all of these possible flips, it updates the current solution to the new solution that had the largest decrease in unsatisfied clauses. Again, if this new solution satisfies the problem, then we’re done; otherwise, the algorithm starts flipping again.

There’s an interesting feature of this algorithm. The best available flip might actually *increase* the number of unsatisfied clauses. The selection is only made from the neighborhood of the current solution. If every neighbor (defined as being one flip away) is worse than the current solution, in that each neighbor satisfies fewer clauses, then GSAT will take the one that is the least bad. This procedure might remind you of a presidential election! But as we’ll see in chapter 5, this can actually be very beneficial: the algorithm has a chance

to escape from a local optimum. It's only a chance, however, not a guarantee, because the algorithm might oscillate between points and never escape from some plateaus. In fact, experiments with GSAT indicate that plateau moves dominate the search [428]. Many variations of the procedure have been offered to avoid this problem [182, 429]. For example, we can assign a weight to each clause [429] and then, at the end of a try, we can increase the weights for all of the clauses that remain unsatisfied. A clause that remains unsatisfied over many tries will have a higher weight than other clauses, therefore, any new solution that satisfies such a clause will be judged to have an accordingly higher score.

3.2.2 Local search and the TSP

There's a huge variety of local search algorithms for the TSP. The simplest is called *2-opt*. It starts with a random permutation of cities (call this tour T) and tries to improve it. The neighborhood of T is defined as the set of all tours that can be reached by changing two nonadjacent edges in T . This move is called a *2-interchange* and is illustrated in figure 3.6.

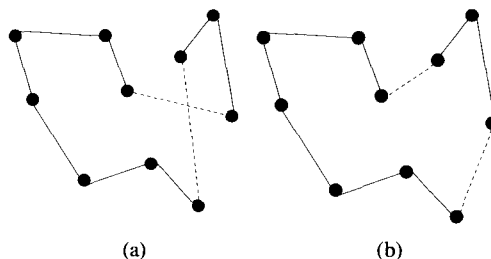


Fig. 3.6. A tour (a) before and (b) after the *2-interchange* move is applied. Broken lines indicate affected edges.

A new tour T' replaces T if it is better.¹ If none of the tours in the neighborhood of T is better than T , then the tour T is called *2-optimal* and the algorithm terminates. As with GSAT, the algorithm should be restarted from several random permutations before we really give up.

The *2-opt* algorithm can be generalized easily to a *k-opt* procedure, where either k , or up to k , edges are selected for removal and are replaced by lower-cost connections. The trade-off between the size of the neighborhood and the efficiency of the search is directly relevant here. If k is small (i.e., the size of the neighborhood is small), the entire neighborhood can be searched quickly, but this small range increases the likelihood of yielding only a suboptimal answer. On the other hand, for larger values of k the number of solutions in the neighborhood become enormous — the number of subsets grows exponentially with k . For this reason, the *k-opt* algorithm is seldomly used for $k > 3$.

¹Note that we replace the tour *every* time we find an improvement. In other words, we terminate the search in the neighborhood of T when the “first improvement” is found.

The best-known local search procedure for the TSP is the Lin-Kernighan algorithm [284]. In a sense, this procedure refines the k -opt strategy by allowing k to vary from one iteration to another. Moreover, instead of taking the “first improvement” as is done with k -opt, it favors the largest improvement. We don’t immediately replace the current best tour when we find something better.

There are several versions of the basic Lin-Kernighan method that differ in their details. Here we describe one of these versions briefly (also see [78]) in which the representation isn’t based on a Hamiltonian path but rather a so-called δ -path. You’ll see how this works below.

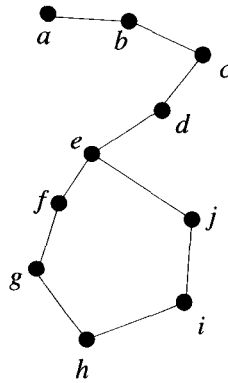


Fig. 3.7. A δ -path

A δ -path is defined as a path where the number of nodes is one greater than the number of edges, and all of the nodes appear only once in the path except for the last node which appears earlier in the path. A picture makes the explanation much easier so look at figure 3.7, which illustrates a δ -path

$$a - b - c - d - e - f - g - h - i - j - e.$$

This is a δ -path because there are 10 edges, 11 nodes, and the nodes are all distinct except for the last node, e .

Any δ -path can be repaired into a legal tour by replacing a single edge: the last one. For example, replacing the edge $(j e)$ by $(j a)$ generates a legal tour. Also, by replacing the last edge $(j e)$ by any other edge that originates in j we can also obtain a new δ -path, e.g., replacing the edge $(j e)$ by $(j f)$. Let’s call such a replacement of one δ -path by another a $switch(e, f)$, where the labels e and f indicate the switched nodes. The order of the nodes is important: The first node (e) is removed and replaced by the second (f). If the labels weren’t essential, we’d just refer to this move as a $switch$.

The cost of a new δ -path increases by

$$cost(j, f) - cost(j, e),$$

and this “increase” might be negative. It’s important to note that a δ -path has two “last edges.” In figure 3.7, we could replace the edge $(j\ e)$ or the edge $(f\ e)$ when building a legal tour or generating a new δ -path.

Now we’re ready to describe the basic steps of Lin-Kernighan. It starts with a random tour T and generates a sequence of δ -paths. The first is constructed from T and the next one is always constructed from the previous one by using a *switch* move.

```

procedure Lin-Kernighan
begin
  generate a tour  $T$ 
  Best_cost = cost( $T$ )
  for each node  $k$  of  $T$  do
    for each edge  $(i\ k)$  of  $T$  do
      C: begin
        if there is  $j \neq k$  such that  $\text{cost}(i, j) \leq \text{cost}(i, k)$ 
        then create a  $\delta$ -path  $p$  by removing  $(i\ k)$  and adding  $(i\ j)$ 
        else goto B
      A: construct a tour  $T$  from  $p$ 
        if cost( $T$ ) < Best_cost then
          Best_cost  $\leftarrow$  cost( $T$ )
          store  $T$ 
        if there is a switch of  $p$  resulting in a
           $\delta$ -path with cost not greater than cost( $T$ )
        then
          make the switch getting a new  $\delta$ -path  $p$ 
          goto A
      B: if cost( $T$ ) < Best_cost then
        Best_cost  $\leftarrow$  cost( $T$ )
        store  $T$ 
        if there remain untested node/edge combinations
        then goto C
      end
    end
  end

```

Fig. 3.8. The structure of Lin-Kernighan algorithm

Figure 3.8 provides the basic outline of the Lin-Kernighan algorithm without the details. A few lines of code,

```

if there is cost improving switch of  $p$ 
then
  make the switch getting a new  $\delta$ -path  $p$ 

```

display the general idea, but they also hide important details. The process of generating a new δ -path is based on adding and removing edges. For a given node k and an edge (i, k) (i.e., for any single iteration of the **for** loop), an edge can either be added or removed but not both. This requires maintaining two lists with *added* edges and *deleted* edges, respectively. If an edge is already present in one of these lists, then the switch moves are restricted. If an edge is in the *added* list, it can only be removed in a switch move. Thus, this part of the algorithm is limited to n switches, where n is the number of nodes. Note also that each generated δ -path has a cost that is at most $\text{cost}(T)$. Another important detail is connected with the selection of a node in both **if there is** statements for converting a tour into a δ -path and for generating a new δ -path from the current one (figure 3.8). Since it might be too expensive to check all possible nodes, it's reasonable to limit this part of the search, and this can be done in a variety of ways [242].

The Lin-Kernighan procedure can run very fast and generate near-optimal solutions (e.g., within two percent of the optimum path [243]) for TSPs with up to a million cities. It can be done in under an hour on a modern workstation [239].

3.2.3 Local search and the NLP

The majority of numerical optimization algorithms for the NLP are based on some sort of local search principle. However, there's quite a diversity of these methods. Classifying them neatly into separate categories is difficult because there are many different options. Some incorporate heuristics for generating successive points to evaluate, others use derivatives of the evaluation function, and still others are strictly local, being confined to a bounded region of the search space. But they all work with complete solutions and they all search the space of complete solutions.

Recall that the NLP concerns finding the vector $\mathbf{x}$ so as to

$$\text{optimize } f(\mathbf{x}), \mathbf{x} = (x_1, \dots, x_n) \in \mathbf{R}^n,$$

where $\mathbf{x} \in \mathcal{F} \subseteq \mathcal{S}$. The *evaluation function* f is defined on the *search space* $\mathcal{S} \subseteq \mathbf{R}^n$ and the set $\mathcal{F} \subseteq \mathcal{S}$ defines the *feasible region*. Usually, the search space $\mathcal{S}$ is defined as an n -dimensional rectangle in $\mathbf{R}^n$, where the domains of the variables are defined by their lower and upper bounds,

$$l(i) \leq x_i \leq u(i), \quad 1 \leq i \leq n,$$

whereas the feasible region $\mathcal{F}$ is defined by a set of m additional constraints ($m \geq 0$),

$$g_j(\mathbf{x}) \leq 0, \text{ for } j = 1, \dots, q, \text{ and } h_j(\mathbf{x}) = 0, \text{ for } j = q + 1, \dots, m.$$

At any point $\mathbf{x} \in \mathcal{F}$, the constraints g_k that satisfy $g_k(\mathbf{x}) = 0$ are called the *active constraints* at $\mathbf{x}$. By extension, equality constraints h_j are also described as active at all points $\mathcal{S}$.

One reason that there are so many different approaches to the NLP is that no single method is superior to all others. In general, it's impossible to develop a deterministic method for finding the best global solution in terms of f that would be better than exhaustive search. Gregory [204] stated:

It's unrealistic to expect to find one general NLP code that's going to work for every kind of nonlinear model. Instead, you should try to select a code that fits the problem you are solving. If your problem doesn't fit in any category except 'general', or if you insist on a globally optimal solution (except when there is no chance of encountering multiple local optima), you should be prepared to have to use a method that boils down to exhaustive search, i.e., you have an intractable problem.

Nevertheless, it's possible to classify the proposed methods from two perspectives: (1) the type of problem being addressed, and (2) the type of technique being used [219]. The first perspective provides a few orthogonal categories of problems: constrained² vs. unconstrained, discrete vs. continuous, and special cases of evaluation functions that are convex, quadratic, linearly separable, etc. The second perspective provides a classification of the methods, such as derivative-based³ vs. derivative-free, analytical vs. numerical, deterministic vs. random, and so forth.

There are several fundamental problems within the broader category of NLPs. One problem involves finding the extremum of a functional, as described above. Another concerns finding the zeros of a function. Although this seems like a completely different problem, and is often treated as if it were different, we could imagine posing a surrogate evaluation function that would judge the merit of any possible root of a function. We could then perform an optimization over that surrogate function and if we chose the evaluation function wisely then we'd get the same answer either way. Let's take a look at some local methods for addressing tasks that arise within the framework of the NLP.

Bracketing methods. Consider the function in figure 3.9 for which $f(x^*) = 0$. Our task is to find x^* . Suppose that we know how to bracket x^* with two other numbers a and b . Then one very simple way to find x^* is the method of bisection: the range between a and b , finding this midpoint, m , determining the value of $f(m)$ and then resetting the left or right limit of the range to m depending on whether or not $f(m)$ is positive. If we continue to iterate this procedure, we eventually converge arbitrarily close to a value x^* such that $f(x^*) = 0$. (This is a perfect example of the divide and conquer approach discussed later in section 4.2.)

²Note that constrained problems can be further divided into those with equality constraints, inequality constraints, or both, as well as the type of constraints (e.g., linear).

³This can be further divided into first-order and second-order derivatives.

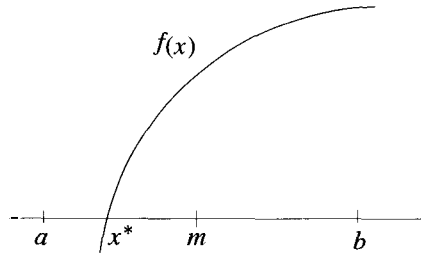


Fig. 3.9. The bisection method finds the zero of a function by first bracketing that zero and then iteratively dividing the region of interest in half. Here, a and b are the left and right limits of the bracketing interval, m is the midpoint that serves as the next point to try, and x^* is the actual answer. The method terminates when the width of the interval shrinks below a given tolerance.

Note that this *bisection* is a local search method. It relies on two bracketing points, a and b , but it still updates only one point at a time and has one current best solution at any iteration.

The assumption is that $f(a)f(b) < 0$. That is, we must be certain that $f(a)$ and $f(b)$ lie on opposite sides of zero. The bisection algorithm is simple:

1. Choose a and b such that $f(a)f(b) < 0$ and $x^* \in [a, b]$.
2. Generate $m \leftarrow (a + b)/2$.
3. If $f(m) \neq 0$ then either $f(a)f((a + b)/2) < 0$ or $f((a + b)/2)f(b) < 0$. If the former case is true, then set $b \leftarrow m$, otherwise set $a \leftarrow m$.
4. If $(b - a) < \epsilon$ then stop; otherwise, proceed to step 2.

The value of ϵ is chosen as a small number that represents the degree of error we're willing to tolerate.

We are interested in how quickly the error bound on a zero of $f(x)$ shrinks because this will determine how many iterations we'll have to make with the algorithm. If we start with an interval of length $L_0 = b - a$, the length will be $L_1 = (b - a)/2$ after the first step. After the next step, it will be $L_2 = (b - a)/2^2$, and so forth. Thus, the length of the interval decreases geometrically as a function of the number of iterations: $L_n = (b - a)/2^n$. As we'll see shortly, although this seems attractive, there are many other methods that will converge much faster.

One such method is another bracketing technique called *Regula Falsi*. This method also starts with an interval $[a, b]$ that bounds a zero of $f(x)$ where $f(a)f(b) < 0$. Instead of bisecting the interval, however, the idea is to construct the secant line between $(a, f(a))$ and $(b, f(b))$ and find the point s where this line intersects the x -axis. The secant line is found by

$$\frac{y - f(b)}{x - b} = \frac{f(a) - f(b)}{a - b}.$$

Therefore,

$$s = \frac{af(b) - bf(a)}{f(b) - f(a)}.$$

The value of s is the first approximation to the root of $f(x)$. As before, we replace the left or right bound on the interval $[a, b]$ depending on whether $f(a)f(s)$ is positive or negative and continue iterating the procedure until the length of the interval is sufficiently small. Figure 3.10 shows how the method works graphically.

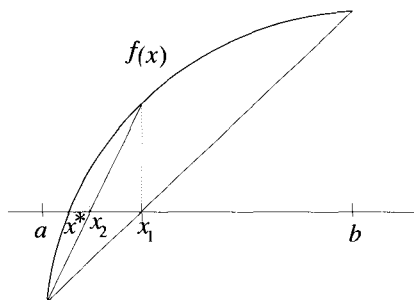


Fig. 3.10. The method of *Regula Falsi* relies on a bracket around the zero of the function. Instead of generating the midpoint between a and b , it connects the line between a and b and takes the intersection of that line with the x -axis as the next point to try. The interval is reduced over successive iterations and the method terminates when the interval is below a given tolerance.

The length of the interval L_n at the n -th iteration under *Regula Falsi* may not converge to zero as $n \rightarrow \infty$. It depends on the function $f(x)$. But the method may converge faster than bisection. The underlying reason is that if $|f(a)| > |f(b)|$ then we would expect that the zero of $f(x)$ is closer to b than to a , and vice versa. Using the secant line to determine the next bounding point will take advantage of this condition when it is true. It can be shown that if the second derivative $f''(x)$ is continuous then *Regula Falsi* converges at a rate of $|s_n - x^*| \leq \lambda^{n-1}\epsilon$, where $0 < \lambda < 1$ and ϵ is a fixed constant.

Fixed-point methods. Although the bisection and *Regula Falsi* methods will converge for continuous functions, they are slow in comparison to other methods such as the following so-called *Newton's method*. This procedure, however, may not always converge even on continuous functions so we must be more careful in applying it. The concept is shown in figure 3.11. Given a function $f(x)$ and an initial starting guess at a value for which $f(x) = 0$, say s , find the line tangent at $f(s)$ and locate where this tangent line intersects the x -axis. Then use this crossing point as the next guess for the root, and so forth, until the distance between guesses over two successive iterations is sufficiently small.

Mathematically, the sequence of guesses at the zero of $f(x)$ proceeds as

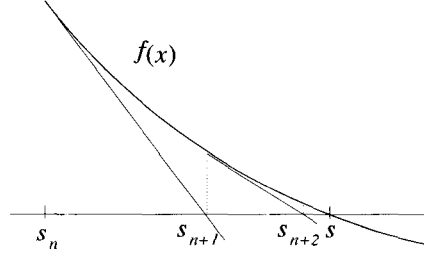


Fig. 3.11. An illustration of Newton's method for finding the zero of a function. Instead of bracketing the zero, as is done in bisection or *Regula Falsi*, the method uses the slope of the function at a trial point and calculates the intersection of the line tangent at that point with the x -axis. That intersecting point is the next point to try. The method halts when the difference between successive points falls below a threshold.

$$s_{n+1} = s_n - \frac{f(s_n)}{f'(s_n)},$$

for $n = 0, 1, \dots$. We can view this as a special case of a sequence $s_{n+1} = g(s_n)$, which is a fixed-point problem: we want to find a function $g(x)$ such that if $f(s) = 0$, then $s = g(s)$. Often, there are many choices for this auxiliary function $g(x)$. For example, if $f(x) = x^2 - 4x + 4$ then $g(x)$ could be $g(x) = \frac{-4}{x-4}$, $g(x) = x^2 - 3x + 4$, or $g(x) = \sqrt{4x-4}$. In all three cases, when $x = 2$, which yields $f(x) = 0$, then $g(2) = 2$.

To find a suitable auxiliary function $g(x)$, it's helpful to impose some constraints. The first is that for any interval $L = [a, b]$, for every $x \in L$, $g(x) \in L$. In words, when x is in the interval $[a, b]$, so is $g(x)$. If we also require $g(x)$ to be continuous then it's guaranteed to have a fixed point in L .

Newton's method, then, consists of finding the auxiliary function $g(x)$ subject to the above constraints and an interval $L = [a, b]$ that bounds the root of $f(x)$, and then iterating on the update equation, $s_{n+1} = g(s_n)$. If $|g'(x)| \leq B < 1$, there will be only one fixed point in L , and if x^* is the root of $f(x)$, the error: $e_n = s_n - x^*$ satisfies $|e_n| \leq \frac{B^n}{1-B} \cdot |s_1 - s_0|$. The method can be stopped when $|s_{n+1} - s_n| < \epsilon$. If we can make a good guess at the location of x^* , Newton's method can converge to it very quickly. Under the condition where $f''(x)$ is continuous and $f'(x) \neq 0$ in an interval L that contains x^* , there will be some ϵ such that Newton's method converges *quadratically* whenever the starting position s_0 is such that $|s_0 - x^*| < \epsilon$.

Newton's method, although it converges quickly under the above conditions, has two main drawbacks: (1) it may require a close initial guess for x^* , and (2) the update equation for s_{n+1} requires computing $f'(x)$, which may be computationally expensive. Instead of computing this derivative, we can substitute an approximation for it:

$$\frac{f(s_n) - f(s_{n-1})}{s_n - s_{n-1}}.$$

The update equation for s_{n+1} then becomes

$$s_{n+1} = s_n - \frac{f(s_n)(s_n - s_{n-1})}{f(s_n) - f(s_{n-1})}.$$

This is the so-called *secant* method, which is similar to the rationale of the *Regula Falsi* technique, but will converge at a superlinear rate (but not a quadratic rate like Newton's method). Formally, the secant method isn't a fixed-point iteration because it depends on two previous values of s rather than only the most recent value. Note too that for the method to be valid, $f(s_n) - f(s_{n-1})$ can never equal zero.

Gradient methods of minimization. Returning to the typical problem of finding the extremum of some functional $f(x)$, as we've seen with Newton's method, one thing that we might try to take advantage of is the gradient of the evaluation function at a particular candidate solution. We could use this information to direct our search toward improved solutions, at least in a local neighborhood. There are so many methods that rely on the gradient in some manner that it would require a complete textbook to review them all. Instead, we'll focus here on the main underlying principles, and refer you to [35] for more detailed information on particular variants of gradient methods.

The basic idea is that we need to find a *directional derivative* so that we can proceed in the direction of the steepest ascent or descent, depending on whether we are maximizing or minimizing. We will only consider minimization here, but we'll do that without loss of generality. If the evaluation function is sufficiently smooth at the current candidate solution, the directional derivative exists. Our challenge is to find the angle around the current solution for which the magnitude of the derivative of the evaluation function with respect to some step size s is maximized. By applying calculus, we can derive that this maximum occurs in the direction of the negative gradient $-\nabla f(\mathbf{x})$ [245]. Recall that

$$\nabla f(\mathbf{x}) = \left[\frac{\delta f}{\delta x_1}, \frac{\delta f}{\delta x_2}, \dots, \frac{\delta f}{\delta x_n} \right]_{\mathbf{x}}.$$

The method of steepest descent, then, is essentially this: Start with a candidate solution $\mathbf{x}_k$, where $k = 1$. Then, generate a new solution using the update rule:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k),$$

where $k \geq 1$, $\nabla f(\mathbf{x}_k)$ is the gradient at $\mathbf{x}_k$, and α_k is the step size. The bigger α_k is, the less *local* the search will be. The method is designed to ensure reductions in the evaluation function for small enough steps, so the idea is to find the right step size to guarantee the best rate of reduction in the evaluations of candidate solutions over several iterations.

If we incorporate second-order derivative information into the update, we get an update equation that is essentially Newton's method:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - (H(f(\mathbf{x}_k)))^{-1} \nabla f(\mathbf{x}_k),$$

where $H(f(\mathbf{x}_k))$ is the *Hessian* matrix of f :⁴

$$H(f(\mathbf{x}_k)) = \begin{bmatrix} \frac{\delta^2 f}{\delta x_1^2} & \frac{\delta^2 f}{\delta x_1 \delta x_2} & \cdots & \frac{\delta^2 f}{\delta x_1 \delta x_n} \\ \frac{\delta^2 f}{\delta x_2 \delta x_1} & \frac{\delta^2 f}{\delta x_2^2} & \cdots & \frac{\delta^2 f}{\delta x_2 \delta x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\delta^2 f}{\delta x_n \delta x_1} & \frac{\delta^2 f}{\delta x_n \delta x_2} & \cdots & \frac{\delta^2 f}{\delta x_n^2} \end{bmatrix}_{\mathbf{x}_k}$$

This method is designed to find the minimum of a quadratic bowl in one step. That's fast convergence! Let's try it out.

Suppose $f(x_1, x_2) = x_1^2 + x_2^2$. Let's take our starting guess to be $\mathbf{x}_1 = [5 \ 1]^T$. To apply the method we have to compute the gradient vector

$$\nabla f(\mathbf{x}_1) = [2x_1 \ 2x_2]^T = [2(5) \ 2(1)]^T = [10 \ 2]^T$$

and the Hessian matrix

$$H(f(\mathbf{x}_1)) = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}.$$

Then, we have to take the inverse of the Hessian, yielding

$$(H(f(\mathbf{x}_1)))^{-1} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix}.$$

To complete the update equation and generate the next point,

$$\mathbf{x}_{k+1} = [5 \ 1]^T - \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix} [10 \ 2]^T$$

and that's $[0 \ 0]^T$, which is the minimum all right! It's easy to see that for any $\mathbf{x}_k$, the update equation will generate the next solution $\mathbf{x}_{k+1}$ at the origin if the evaluation function is a quadratic bowl. For functions that aren't quadratic bowls, however, things aren't as nice. The primary consideration is that since the method relies on gradient information, we could get stuck at a local optimum. We also have to choose an error tolerance ϵ and halt the algorithm when the difference between $\mathbf{x}_{k+1}$ and $\mathbf{x}_k$ is less than ϵ .

Admittedly, calculating the inverse of the Hessian matrix is rarely as easy as in this example. Many so-called *quasi-Newton* methods use different mechanisms for estimating the Hessian. Again, there is a cornucopia of different techniques here, so please see [35] for more details on their derivation. If Gaussian elimination is used to generate the inverse Hessian then the method is called Newton-Gauss. In the remainder of the book we'll use Newton's method and Newton-Gauss synonymously unless otherwise noted.

⁴The Hessian matrix $H(f(\mathbf{x}_k))$ is often denoted by $\nabla^2 f(\mathbf{x}_k)$.

Newton-Gauss, and all of the quasi-Newton methods are essentially local search procedures. They update a single complete solution with another complete solution. They use information that is local to the current best point to generate a new point. And they also work to the boundary of a neighborhood, typically defined by the inverse Hessian.

Another somewhat related technique is *Brent's method*, which is a bracketing technique designed to find the minimum of a quadratic function. It works with three points — instead of just two as with bisection and *Regula falsi* — and fits a parabola to those points. It then uses the minimum of that parabola as the next guess for the minimum of the overall function. The method iterates by narrowing the bracket based on each new point tested. Figure 3.12 illustrates the procedure and you'll find the algorithm coded in [363].

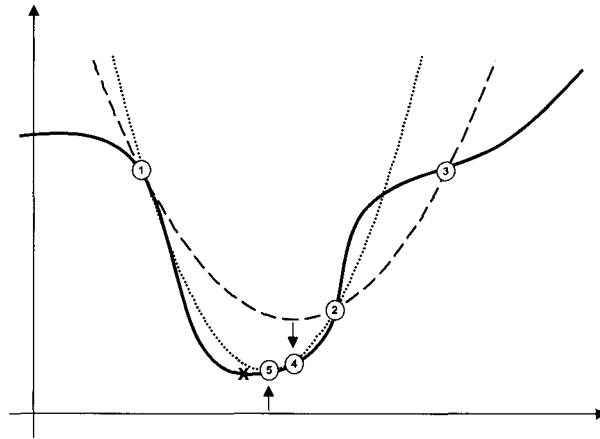


Fig. 3.12. Brent's method is a bracketing technique that relies on three points. In order to find the minimum of a function, it calculates a parabolic auxiliary function through the three points and locates the minimum of that parabola. That minimum is then used as one of the three points to define the next parabola, and so forth. The method terminates when the difference between guesses at the minimum of the function fall below a threshold.

3.3 Linear programming: The simplex method

At last we come to the final traditional technique for processing complete solutions that we'll discuss here: the simplex method for linear programming (LP) problems. Problems in LP require finding the extremum (let's say the maximum) of a linear combination of variables,

$$eval = a_{01}x_1 + \dots + a_{0n}x_n,$$

subject to the primary constraints,

$$x_1 \geq 0, \dots, x_n \geq 0,$$

and also subject to $M = m_1 + m_2 + m_3$ additional constraints; m_1 of the form:

$$a_{i1}x_1 + \dots + a_{in}x_n \leq b_i, (b_i \geq 0), i = 1, \dots, m_1,$$

m_2 of the form

$$a_{j1}x_1 + \dots + a_{jn}x_n \geq b_j, (b_j \geq 0), j = m_1 + 1, \dots, m_1 + m_2,$$

and m_3 of the form

$$a_{k1}x_1 + \dots + a_{kn}x_n = b_k, (b_k \geq 0), k = m_1 + m_2 + 1, \dots, M.$$

Any vector $\mathbf{x}$ that satisfies all of the constraints is said to be *feasible* and our goal is to find the feasible vector that yields the best result in terms of the evaluation function.

The simplex method constructs the best vector $\mathbf{x}$ in a series of operations that take advantage of the linear characteristics of the problem. Figure 3.13 depicts the situation. Potentially, there are equality and inequality constraints imposed over the evaluation function, but since the evaluation function is linear, the optimum solution lies at one of the vertexes of the simplex (the feasible region), or in a degenerate case, anywhere along a boundary. The simplex procedure is guaranteed to find the best vertex of the simplex.

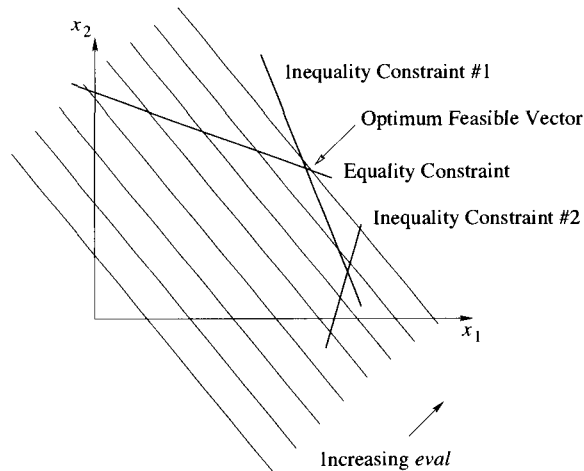


Fig. 3.13. A graphical interpretation of the linear programming problem. The contour lines indicate the values of the function *eval* which increase into the first quadrant. The bold lines indicate constraints (either equality or inequality). The optimum vector lies on the equality constraint and at a vertex of the simplex.

The method is best illustrated on a simple problem. Consider the following scenario. A factory produces two types of items: small fancy chairs and large

simple tables. The profit per chair is \$20, whereas the profit per table is \$30. Each chair requires a single unit of wood and three hours of labor. Each table, on the other hand, requires six units of wood and one hour of labor. The production process has some restrictions: There are 288 units of wood available and 99 hours of labor available during a particular period of time. The task is to maximize the factory's profit.

We can formulate this problem as a linear programming problem:

$$\text{maximize } eval = 20x_1 + 30x_2,$$

such that

$$\begin{aligned} x_1 + 6x_2 &\leq 288 \text{ (limitation for units of wood),} \\ 3x_1 + x_2 &\leq 99 \text{ (limitation for hours of labor).} \end{aligned}$$

Additionally, $x_1 \geq 0$ and $x_2 \geq 0$.

Note that we can reformulate the above problem in the following way:

$$\text{maximize } 20x_1 + 30x_2,$$

such that

$$\begin{aligned} x_3 &= -x_1 - 6x_2 + 288, \\ x_4 &= -3x_1 - x_2 + 99, \text{ and} \\ x_1 &\geq 0, x_2 \geq 0, x_3 \geq 0, x_4 \geq 0. \end{aligned} \tag{3.1}$$

We proceed by making a *tableau*, which is just a fancy name for a table, where each entry is copied from the coefficients of the constraints and the evaluation function into the last row.

	x_1	x_2	x_3	x_4	
x_3	-1	-6	0	0	288
x_4	-3	-1	0	0	99
<i>eval</i>	20	30	0	0	0

Our initial feasible point is generated by setting the variables on the right-hand side of the constraints (3.1) equal to zero and then solving for the variables on the left-hand side. Here we get $(x_1, x_2, x_3, x_4) = (0, 0, 288, 99)$.

The main idea behind the simplex method is to exchange variables on the right-hand side with those on the left-hand side wherever we can improve the evaluation function. In our example, we have to examine variables x_1 and x_2 and see if increasing either variable would increase the evaluation function *eval*. Note that the maximum increase in x_1 is 33 (because of the constraint $x_4 \geq 0$, (3.1)), which yields an increase in the evaluation function of 660. The maximum increase in x_2 is 48 (because of the constraint $x_3 \geq 0$, (3.1)), which yields an increase in the evaluation function of 1440. Thus we set

$$x_2 \leftarrow x_2 + 48.$$

Note that at this stage x_3 becomes zero and we have to rewrite (3.1) to keep all of the “zero” variables (e.g., x_1 and x_3) on the right-hand side. Thus we have to rewrite the original problem expressing variable x_2 as a function of variables x_1 and x_3 . This leads us to a new problem:

maximize $15x_1 - 5x_3 + 1440$,

such that

$$\begin{aligned} x_2 &= -\frac{1}{6}x_1 - \frac{1}{6}x_3 + 48, \\ x_4 &= -\frac{17}{6}x_1 + \frac{1}{6}x_3 + 51, \text{ and} \\ x_1 &\geq 0, x_2 \geq 0, x_3 \geq 0, x_4 \geq 0. \end{aligned} \quad (3.2)$$

Now our tableau is:

	x_1	x_2	x_3	x_4	
x_2	$-1/6$	0	$-1/6$	0	48
x_4	$-17/6$	0	$1/6$	0	51
<i>eval</i>	15	0	-5	0	1440

This time we have to examine variables x_1 and x_3 and see if increasing either variable would increase the evaluation function *eval*. This is only true for x_1 ; note that the maximum increase in x_1 is 18 because of the constraint $x_4 \geq 0$, (3.2). Thus we set

$$x_1 \leftarrow x_1 + 18$$

and this leads us to a new problem:

maximize $-\frac{100}{17}x_3 - \frac{90}{17}x_4 + 1710$,

such that

$$\begin{aligned} x_1 &= 18 + \frac{1}{17}x_3 - \frac{6}{17}x_4, \\ x_2 &= 45 - \frac{3}{17}x_3 + \frac{1}{17}x_4, \text{ and} \\ x_1 &\geq 0, x_2 \geq 0, x_3 \geq 0, x_4 \geq 0. \end{aligned}$$

Now our tableau is:

	x_1	x_2	x_3	x_4	
x_1	0	0	$-1/17$	$-6/17$	18
x_2	0	0	$-3/17$	$1/17$	45
<i>eval</i>	0	0	$-100/17$	$-90/17$	1710

In this case, we're stuck. Any increase for x_3 or x_4 would *decrease* the value of *eval* — and that means we've found the right answer, $x_1 = 18$, $x_2 = 45$, for which *eval* = 1710 (representing the maximum profit).

Note that in general, we focused on all the “right-hand variables” where we could increase *eval*. This process is illustrated in figure 3.14.

There are many tricks for converting LP problems into the form required for this procedure, and there are variations of the procedure for different cases. The basic idea, however, lies somewhere between working with complete solutions and working with partial solutions. Through analytic means, we decide which variable to increase and over a series of swaps with other variables, we alternately examine other dimensions of the problem to see if we can obtain

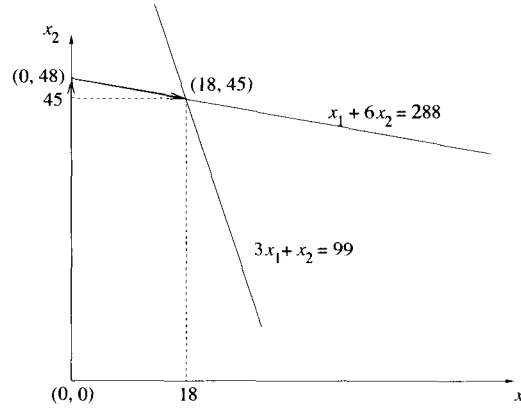


Fig. 3.14. Moving from one vertex to another with the simplex method. The *boldface arrows* indicate the steps of the procedure. The feasible area lies below both lines $x_1 + 6x_2 = 288$ and $3x_1 + x_2 = 99$.

improvements. The linear nature of the problem is what makes this a feasible routine. Once the evaluation function is nonlinear you have to compensate with other tricks that are beyond the discourse here. In the limit, when the evaluation function is potentially multimodal or even discontinuous, this class of method breaks down. It leaves you with the option of trying to approximate the problem as if it were linear or perhaps quadratic or restricted to integer values, but if you do that then you will generate the right answer to the wrong problem, and there's no rule for just how bad that answer might be.

3.4 Summary

There are many classic algorithms that operate on complete solutions. You can interrupt these procedures at any iteration and they will provide some potential solution. The quality of that solution, however, depends largely on the degree to which the procedure matches the problem. For example, if your task is to maximize an evaluation function that's a linear combination of variables that are subject to linear equality and inequality constraints, then linear programming methods are appropriate. If the evaluation function is smooth (i.e., differentiable), unimodal, and you want to find its minimum, then methods based on gradients and higher-order functions can be useful. If you face a combinatorial problem, like a TSP, there are many different local operators you can employ to take advantage of the characteristics that are inherent to circuits in the Euclidean plane. These will iteratively improve complete tours and may quickly find one that is nearly optimal.

The drawback that you face is that if you really don't know much about your problem, or if it doesn't fit well within the class of problems that each of these

algorithms can tackle, you are left with enumerating all possible solutions. For any real-world problems of concern, this approach is almost always impractical because the number of alternative solutions is prohibitively large.

We mentioned that there are two classes of classic algorithms: (1) those that operate on complete solutions, and (2) those that evaluate partially constructed or approximate solutions. When the methods of this chapter break down, it is sometimes reasonable to think about the latter class of techniques and attempt to decompose problems into simpler subproblems that can be reassembled, or perhaps to solve complex problems in a series of simpler stages. These are treated in detail in the subsequent chapter.

IV. What Are the Numbers?

Let's consider a puzzle which illustrates that the process of finding all the solutions to some problems isn't all that simple. The problem was offered to us by Peter Ross from the University of Edinburgh. It's a seemingly simple problem: Find the natural numbers a , b , c , and d , such that

$$\begin{aligned}ab &= 2(c + d), \text{ and} \\cd &= 2(a + b).\end{aligned}$$

For example, one of the solutions to this problem is $(a, b) = (1, 54)$ and $(c, d) = (5, 22)$. Clearly, some solutions are far from trivial.

It's important here to analyze the problem before we attempt to solve it. It might be a good idea to get four random integers which we can use to check if the above equations hold. Certainly, the chances for that occurrence are very slim! But we can then change one number at the time to get "a feel" for the problem. After a short time we'd discover that it's unlikely that all of these numbers are large. If a , b , c , and d are sufficiently large, then the product of a and b is much greater than twice their sum. The same is true for c and d . For example, if $a \geq 5$ and $b \geq 5$, then

$$\begin{aligned}ab &= (5 + a')(5 + b') = 25 + 5a' + 5b' + a'b' = (5 + 3a' + 3b' + a'b') \\&\quad + (2a' + 10) + (2b' + 10) > (2a' + 10) + (2b' + 10) = 2a + 2b,\end{aligned}$$

for any $a' \geq 0$ and $b' \geq 0$ (we set $a = 5 + a'$ and $b = 5 + b'$). This simple observation leads to the following conclusion: If all of the numbers a , b , c , and d are 5 or more, then

$$ab > 2(a + b) = cd > 2(c + d) = ab,$$

resulting in a contradiction!

Assuming, then, that $a = \min\{a, b, c, d\}$, it's sufficient to consider only four cases, namely $1 \leq a \leq 4$. So, if $a = 1$, the problem reduces to

$$\begin{aligned}b &= 2(c + d), \text{ and} \\cd &= 2(1 + b) = 2 + 2b.\end{aligned}$$

Clearly, b must be even, so $b = 2b'$ and we have

$$\begin{aligned}b' &= c + d, \text{ and} \\cd &= 2 + 4b'.$$

Replacing b' in the second equation, we get

$$cd = 2 + 4c + 4d,$$

which is easy to solve:

$$c = \frac{4d + 2}{d - 4} = 4 + \frac{18}{d - 4}.$$

As c is a natural number, d must be 5, 6, 7, 10, 13, or 22. For these we get three pairs of (symmetrical) solutions:

$$(c, d) = (5, 22); (c, d) = (6, 13); \text{ or } (c, d) = (7, 10).$$

These three solutions imply $b = 54$, $b = 38$, and $b = 34$, respectively, and in that way we have found the first three solutions of the problem:

$$(1, 54), (5, 22), \\ (1, 38), (6, 13), \text{ and} \\ (1, 34), (7, 10).$$

The second case is for $a = 2$. In that case the problem reduces to:

$$2b = 2(c + d), \text{ and} \\ cd = 2(2 + b),$$

which simplifies to

$$b = c + d, \text{ and} \\ cd = 4 + 2b.$$

Again, replacing b in the second equation we get

$$cd = 4 + 2c + 2d,$$

which gives us

$$c = \frac{2d + 4}{d - 2} = 2 + \frac{8}{d - 2}.$$

As before, d must be 3, 4, 6, or 10. For these we get two pairs of symmetrical solutions

$$(c, d) = (3, 10) \text{ and } (c, d) = (4, 6).$$

These two solutions imply $b = 13$ and $b = 10$, respectively, and in that way we have found two additional solutions of the problem:

$$(2, 13), (3, 10), \text{ and} \\ (2, 10), (4, 6).$$

The third case to consider is for $a = 3$. In that case b must be even ($b = 2b'$ as the product of a and b is even) and we have

$$3b' = c + d, \text{ and} \\ cd = 6 + 4b'.$$

In that case (replacing b' in the second equation) we get

$$c = \frac{4d + 18}{3d - 4} = 1 + \frac{d + 22}{3d - 4}.$$

We can see that $d > 6$ implies $c < 3$ (as $\frac{d+22}{3d-4} < 2$) which gives a contradiction (as $a = 3 < c$). Thus all we have to do is consider cases for d equal to 3, 4, 5, and 6. The first and last cases give the same (i.e., symmetrical) solution,

$$(c, d) = (3, 6),$$

which also implies $b = 6$. Thus, we found the sixth solution of the problem:

$$(3, 6), (3, 6).$$

The final case is for $a = 4$. Here

$$\begin{aligned} 4b &= 2(c + d), \text{ and} \\ cd &= 2(4 + b), \end{aligned}$$

which leads to

$$cd = 8 + c + d.$$

Thus,

$$c = \frac{d + 8}{d - 1} = 1 + \frac{9}{d - 1}.$$

Since $d \geq a = 4$, d is either 4 or 10. The latter value results in $c = 2$, which is a contradiction, whereas $d = 4$ implies $c = 4$ and $d = 4$. This case yields the final solution of the problem,

$$(4, 4), (4, 4),$$

and the problem has seven solutions in total.

It's important to keep in mind the way the search space was reduced here. "Playing" with the problem may give us important insights into its nature and provides the opportunity to discover ways for considering only a fraction of the possible solutions. Again, the key point is don't give up! And don't settle for some solution that's less than perfect if you don't have to.

You can practice these skills right away. Another similar puzzle was offered to us by Antoni Mazurkiewicz from the Polish Academy of Sciences.

The problem is to find the natural numbers a , b , and c , such that

$$a + b + 1 \text{ divides } c, a + c + 1 \text{ divides } b, \text{ and } b + c + 1 \text{ divides } a.$$

The problem has 12 solutions. For example, one of them is

$$(21, 14, 6),$$

as

$21 + 14 + 1 = 36$, which divides 6,
 $21 + 6 + 1 = 28$, which divides 14, and
 $14 + 6 + 1 = 21$, which, of course, divides 21.

Try to find all 12 solutions!

If you prefer something simple, you may consider the case where two “wrongs” make a “right”! The puzzle

$$\begin{array}{rcccccc}
 & W & R & O & N & G & \\
 & W & R & O & N & G & \\
 \hline
 & R & I & G & H & T &
 \end{array}$$

has a few solutions (addition is assumed). One of them is

$$\begin{array}{rcccccc}
 & 2 & 5 & 7 & 3 & 4 & \\
 & 2 & 5 & 7 & 3 & 4 & \\
 \hline
 & 5 & 1 & 4 & 6 & 8 &
 \end{array}$$

Keep this in mind for some ripe opportunity!

4. Traditional Methods — Part 2

Two and two make four
Four and four make eight
Eight and eight make sixteen
Repeat! says the teacher.

Jacques Prévert, *Page d'écriture*

We've seen a slew of techniques that manipulate complete solutions. Let's now consider some of the algorithms that work with partial or incomplete solutions and solve problems by constructing solutions one piece at a time. We begin with the best-known category: greedy algorithms.

4.1 Greedy algorithms

Greedy algorithms attack a problem by constructing the complete solution in a series of steps. The reason for their popularity is obvious: simplicity! The general idea behind the greedy approach is amazingly simple: assign the values for all of the decision variables one by one and at every step make the best available decision. Of course, this approach assumes a heuristic for decision making that provides the best possible move at each step, the best “profit,” thus the name *greedy*. But it's clear that the approach is also shortsighted since taking the optimum decisions at each separate step doesn't always return the optimum solution overall.

4.1.1 Greedy algorithms and the SAT

Let's consider a few examples by starting with the SAT problem. We'll assign a truth value (TRUE or FALSE) to each variable, but we need some heuristic to guide our decision-making process. Consider the following approach:

- For each variable from 1 to n , in any order, assign the truth value that results in satisfying the greatest number of currently unsatisfied clauses. If there's a tie, choose one of the best options at random.

This heuristic is indeed greedy: at every step it tries to satisfy the largest number of currently unsatisfied clauses! Unfortunately, we can easily show that the performance of such unrestrained greed is quite poor, even for some simple problems. For example, consider the following test case:

$$\bar{x}_1 \wedge (x_1 \vee x_2) \wedge (x_1 \vee x_3) \wedge (x_1 \vee x_4).$$

If we consider the variable x_1 first, then we choose $x_1 = \text{TRUE}$ because this assignment satisfies three clauses. But the first clause is not and cannot be satisfied when x_1 is TRUE, so all of the remaining effort of the algorithm is wasted. The formula will never be satisfied for any assignment of x_2 , x_3 , and x_4 , as long as x_1 remains TRUE.

We can easily identify the problem with the above greedy approach. It's too greedy in the sense that we haven't given enough care as to what order to consider the variables. We'd be better off using some additional heuristic for choosing the order. In particular, we might start with those variables that appear in only a few clauses (like x_2 , x_3 , or x_4 in the formula above), leaving more commonly occurring variables (like x_1) for later. So, the next, upgraded greedy algorithm for the SAT might be:

- Sort all the variables on the basis of their frequency, from the smallest to the largest, for all clauses.
- For each variable, taken in the above order, assign a value that would satisfy the greatest number of currently unsatisfied clauses. In the event of a draw, make an arbitrary decision.

After some additional thought, we might discover some weaknesses of this new algorithm. For example, consider the following case:

$$(x_1 \vee x_2) \wedge (x_1 \vee x_3) \wedge (\bar{x}_1 \vee x_4) \wedge (\bar{x}_2 \vee \bar{x}_4) \wedge (x_2 \vee x_5) \wedge (x_2 \vee x_6) \wedge F,$$

where we assume that the formula F does not contain variables x_1 and x_2 , but contains many occurrences of the remaining variables. Thus, x_1 is present in only three clauses, and x_2 is in four clauses, with all the other variables having a higher frequency. The above greedy algorithm would assign x_1 the value of TRUE because this assignment satisfies two clauses. Similarly, it would assign x_2 the value of TRUE because it satisfies two additional clauses. However, it wouldn't be possible to satisfy both the third and fourth clauses,

$$(\bar{x}_1 \vee x_4) \wedge (\bar{x}_2 \vee \bar{x}_4),$$

and the algorithm would fail to find the required truth assignment.

Of course, not to be defeated just yet, there's plenty of room for further "upgrades." We could introduce an extra rule that would forbid a particular assignment if it makes any clause FALSE. Or we might consider the frequency of variables in the *remaining* (i.e., still unsatisfied) clauses. We might also try to be more sophisticated. Instead of ordering the variables solely on the basis of their frequencies, we might take into account the length of a clause. The intuition is that an occurrence of a variable in a short clause has a different significance than an occurrence of the variable in a long clause! Unfortunately, none of these approaches leads to an algorithm that will work for all SAT problems. *There is no such greedy algorithm for the SAT!*

4.1.2 Greedy algorithms and the TSP

The most intuitive greedy algorithm for the TSP is based on the nearest-neighbor heuristic: starting from a random city, proceed to the nearest unvisited city and continue until every city has been visited, at which point return to the first city. Such a tour can be far from perfect. There's often a high price to pay for making greedy choices at the beginning of the process. A simple example to illustrate the case is given in figure 4.1, where starting from city A, this greedy heuristic constructs a tour $A - B - C - D - A$, with the total cost of $2 + 3 + 23 + 5 = 33$, but the tour $A - C - B - D - A$ costs only $4 + 3 + 7 + 5 = 19$.

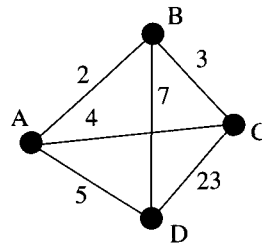


Fig. 4.1. A sample TSP with four cities. Note that the costs for each edge do not correspond to the distance between the cities as drawn.

Another greedy possibility is to construct a tour by choosing the shortest available connections (i.e., edges) between cities, avoiding a situation where a city is involved in more than two connections. In other words, we start by finding the cheapest among all available connections between every pair of cities, then we select the next cheapest available connection, and so on. Of course, once we select the connections between, say, cities $A - B$ and cities $B - C$, then we can't select any more connections that involve city B. Note that this greedy approach would produce the same answer as the previous greedy approach on the four-city TSP in figure 4.1.

Yet another greedy possibility [73] is based on the following concept. A single city is selected and the initial "tour"¹ is constructed by starting from the selected city, visiting another city, returning to the selected city, going to some other unvisited city, returning to the selected city, and so on. In other words, the initial tour consists of a path that is a union of uniform segments leading from the selected city to some other city and back. Then — this is where greedy choices are made — we consider pairs of cities (excluding the selected city). A pair of cities (say, i and j) is selected for which the largest savings can be made by going directly from i to j instead of returning inbetween to the selected city. The algorithm considers pairs of unselected cities in a non-increasing order of

¹This initial tour is not a tour in the sense of TSP, since the selected city is visited many times.

savings, modifying the tour if possible (e.g., avoiding premature cycles). Again, this approach will fail on the simple example given in figure 4.1. For every common sense heuristic you can invent, you can find a pathological case that will make it look very silly.

4.1.3 Greedy algorithms and the NLP

Let's also consider the final case of the NLP. There really are no efficient greedy algorithms for the NLP, but we can design an algorithm that displays some greedy characteristics. For example, to optimize a function of, say, two variables, x_1 and x_2 , we could set one of the variables, say, x_1 , at a constant and vary x_2 until we reach an optimum. Then, while holding the new value of x_2 constant, we could vary x_1 until a new optimum is reached, and so on. Naturally, this *line search* can be generalized into n dimensions, but as Himmelblau stated [219]:

This process, however, performs poorly if there are interactions between x_1 and x_2 , that is, if in effect terms involving products of x_1 and x_2 occur in the objective function. Hence the method cannot be recommended unless the user has the [evaluation] function such that the interactions are insignificant.

Strictly speaking, line searches aren't really greedy algorithms because they only evaluate complete solutions. Nevertheless, the intention of choosing the best available opportunity with respect to a single dimension at a time does follow the general notion of greedy algorithms.

Greedy methods, whether applied to the SAT, TSP, NLP, or other domains, are conceptually simple, but they normally pay for that simplicity by failing to provide good solutions to complex problems with interacting parameters. And those are the problems that we face routinely in the real world.

4.2 Divide and conquer

Sometimes it's a good idea to solve a seemingly complicated problem by breaking it up into smaller simpler problems. You might be able to then solve each of those easier problems and find a way to assemble an overall answer out of each part. This "divide and conquer" approach is really only cost effective if the time and effort that's required to complete the decomposition, solve all of the decomposed problems, and then reassemble an answer is less than the cost of solving the problem as it originally stands with all its inherent complexity. Also, you have to take care that when you assemble the solution from the decomposed pieces, you actually do get the answer you were looking for. Sometimes the chance for assembling an overall solution disintegrates as you break the problem apart.

The outline of the divide and conquer (D&C) approach is given in figure 4.2.

```

procedure D&C ( $P$ )
begin
  split the problem  $P$  into subproblems  $P_1, \dots, P_k$ 
  for  $i = 1$  to  $k$  do
    if  $\text{size}(P_i) < \rho$  then solve  $P_i$  (getting  $s_i$ )
    else  $s_i \leftarrow \text{D\&C}(P_i)$ 
  combine  $s_i$ 's into the final solution.
end

```

Fig. 4.2. Divide and conquer recursive procedure for solving problem P

The original problem P is replaced by a collection of subproblems, each of which is further decomposed into sub-subproblems, and so forth, often in a recursive manner. The process continues until the problems are reduced to being trivial (i.e., smaller than a conceptual constant ρ in the outline given in figure 4.2) so that they can be solved “by hand.” The algorithm then spirals “upward,” combining the solutions to construct a solution to larger subproblems.

The divide and conquer principle is intrinsic to many sorting algorithms, such as quicksort and mergesort. Polynomial and matrix multiplications can also be accomplished using this approach. To give you an explicit example here, consider the following problem that illustrates the idea of the divide and conquer approach nicely. There is a chess board with a dimension of 2^m (i.e., it consists of 2^{2m} squares) with a hole, i.e., one arbitrary square is removed. We have a number of L shaped tiles (see figure 4.3) and the task is to cover the board by these tiles. (The orientation of a tile isn’t important.)

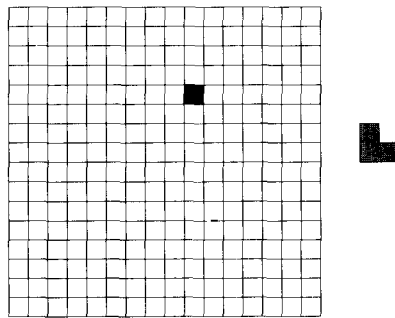


Fig. 4.3. A chess board $2^m \times 2^m$ ($m = 4$) and a tile

In this case the search space consists of many possible arrangements of tiles on the board and we have to find the right one. Many methods can be considered, however, the divide and conquer technique is ideal here. We can simply divide the board into four equal areas: the hole is located in one of them. Thus, we can place the first tile in a such way that it would cover three squares: one from each

area that doesn't contain the original hole (figure 4.4a). Then, instead of one square (with a hole), there are four smaller squares. Each of them has a hole, whether it's the original one or one created by the placement of a tile. Thus we can continue this process, dividing each of the smaller squares into four even smaller squares and place new tiles such that each of the new smaller squares has precisely one "hole" (figure 4.4b).

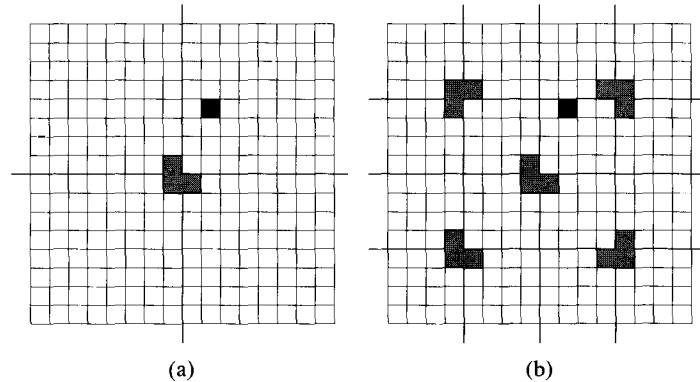


Fig. 4.4. The placement of the first tile (a) and the next four tiles (b)

Note that every square, which is now just 4×4 , has just one missing piece, i.e., one hole. Continuing our procedure, we can divide each of them into four 2×2 squares and by an appropriate placement of tiles, each of these new small squares would have precisely one hole (figure 4.5a). Then we face the easiest possible task: to place a tile into a 2×2 square with one missing piece! Thus, we easily get the final solution to the problem (figure 4.5b).

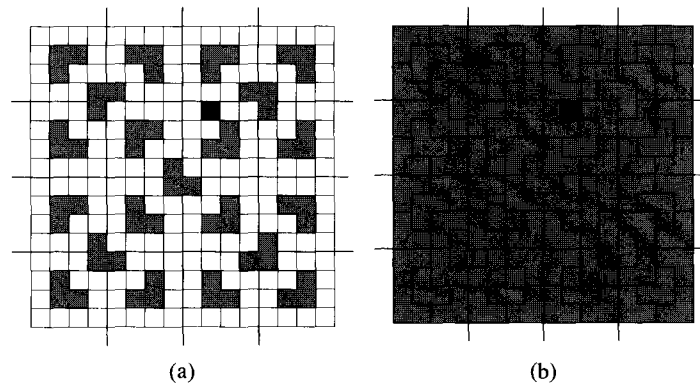


Fig. 4.5. (a) The placement of the next 16 tiles and (b) the final solution

4.3 Dynamic programming

Dynamic programming works on the principle of finding an overall solution by operating on an intermediate point that lies between where you are now and where you want to go.² The procedure is recursive, in that each next intermediate point is a function of the points already visited. A prototypical problem that is suitable for dynamic programming has the following properties:

- The problem can be decomposed into a sequence of decisions made at various stages.
- Each stage has a number of possible states.
- A decision takes you from a state at one stage to some state at the next stage.
- The best sequence of decisions (also known as a *policy*) at any stage is independent of the decisions made at prior stages.
- There is a well-defined cost for traversing from state to state across stages. Moreover, there is a recursive relationship for choosing the best decisions to make.

The method can be applied by starting at the goal and working backward to the current state. That is, we can first determine the best decision to make at the last stage. From there, we determine the best decision at the next to last stage, presuming we will make the best decision at the last stage, and so forth.

The simplest example is the *stagecoach problem* developed by Harvey Wagner [217]. The problem has a lot of window dressing involving a salesman taking his stagecoach through territory with hostile Native American Indians, but we'll dispense with that to cut straight to the core of the issue.

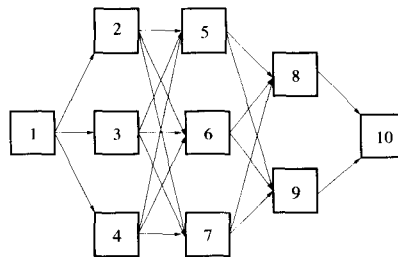


Fig. 4.6. The flow diagram indicating the choices for the salesman as he takes his stagecoach through hostile territory.

²The term *dynamic* was used to identify the approach as being useful for problems “in which times plays a significant role, and in which the order of operations may be crucial” [40].

The salesman must start from his current position and arrive at a fixed destination. He effectively has three stages at which he can make alternative choices about how to proceed (see figure 4.6). At the first stage, he has three alternative paths. Likewise, at the second stage, there are three alternatives. Finally, at the third stage there are two alternatives. The fourth stage does not offer a choice. The costs for traversing from each state at each stage to each next possible state are shown in figure 4.7. The problem is to find the least-cost path from the first state (1) to the last state (10).

	2	3	4
1	2	4	3

	5	6	7
2	7	4	6
3	3	2	4
4	4	1	5

	8	9
5	1	4
6	6	3
7	3	3

	10
8	3
9	4

Fig. 4.7. The cost for making each possible choice at each decision stage. The row indicates the current state and the column indicates the next state. The four matrices correspond to the four stages. Note that only the first three stages offer a choice of which state to choose.

We need to introduce some notation here before proceeding to work out the answer:

- There are n stages and the decision of which state to choose at the n -th stage is x_n . Here, $n = 4$.
- $f_n(s, x_n)$ is the cost of the best sequence of decisions (policy) for all of the remaining stages, given that the salesman is in state s , stage n , and chooses x_n as the immediate next state.
- $x_n^*(s)$ is the value of x_n that minimizes $f_n(s, x_n)$ and $f_n^*(s)$ is the corresponding minimum value.
- The goal is to find $f_1^*(1)$ because the salesman is starting in state 1. This is accomplished by finding $f_4^*(s)$, $f_3^*(s)$, $f_2^*(s)$, and then finally $f_1^*(1)$.

What then is $f_4^*(s)$? For which x_4 is $f_4(s, x_4)$ minimized? This is almost a trick question because there's only one state to choose while in the fourth stage: $x_4 = 10$. Thus we easily compute the values shown below, and we have completed the first phase of the dynamic programming procedure.

s	$f_4^*(s)$	$x_4^*(s)$
8	3	10
9	4	10

The above matrix displays the cost of going from states 8 or 9 to a state at the final stage (in this case, there is only one such state).

The next phase asks for the value of $f_3^*(s)$. Recall that since the decisions are independent, $f_3(s, x_3) = c_{sx_3} + f_4^*(x_3)$, where c_{sx_3} is the cost of traveling from s to x_3 . The matrix shown below indicates the relevant values of $f_3(s, x_3)$ for every possible choice of destination presuming s is 5, 6, or 7.

s	$f_3(s, 8)$	$f_3(s, 9)$	$f_3^*(s)$	$x_3^*(s)$
5	4	8	4	8
6	9	7	7	9
7	6	7	6	8

We can see, for example, that if we're in state 5, then state 8 minimizes the total remaining cost. State 9 minimizes the cost if $s = 6$, and state 8 minimizes the cost if $s = 7$.

Working backward again, we must find the value of $f_2^*(s)$. In an analogous manner to what appears directly above, the matrix shown below provides the relevant values of $f_2(s, x_2)$. Note that $f_2(s, x_2) = c_{sx_2} + f_3^*(x_2)$.

s	$f_2(s, 5)$	$f_2(s, 6)$	$f_2(s, 7)$	$f_2^*(s)$	$x_2^*(s)$
2	11	11	12	11	5 or 6
3	7	9	10	7	5
4	8	8	11	8	5 or 6

Finally, with regard to $f_1^*(s)$, the last matrix offers the costs of proceeding to states 2, 3, or 4:

s	$f_1(s, 2)$	$f_1(s, 3)$	$f_1(s, 4)$	$f_1^*(s)$	$x_1^*(s)$
1	13	11	11	11	3 or 4

This is the final table for the stagecoach problem. Now the dynamic programming method has been applied all the way back to the first stage. The values in the above matrix indicate the cost of traveling from state 1 to 2, 3, or 4.

At this point we can identify the best answer to the overall problem. In fact, here there are three best answers (policies), each with an equal cost of 11 units:

- $1 \rightarrow 3 \rightarrow 5 \rightarrow 8 \rightarrow 10$
- $1 \rightarrow 4 \rightarrow 5 \rightarrow 8 \rightarrow 10$
- $1 \rightarrow 4 \rightarrow 6 \rightarrow 9 \rightarrow 10$.

Note that the obverse procedure of choosing the least-cost path moving forward through the stages does not give the optimum solution. That greedy solution has a cost of 13 units (you can verify this). Thus, here is a case where the greedy approach fails, but dynamic programming generates the right answer. One drawback of the approach, however, is that it can be computationally intensive. If N is both the number of stages and states per stage, then the required number of operations scales as N^3 . The method can, however, be extended to handle a variety of optimization problems including the order of multiplying

matrices and finding the best organization for a tree to minimize the cost of searching [425].

Dynamic programming algorithms tend to be somewhat complicated to understand. This is because, in practice, the construction of a dynamic program depends on the problem. It is a sort of “artistic intellectual activity depending in part on the specific structure of the sequential decision problem” [432]. This is why we’ll illustrate this technique by two additional examples: matrix multiplication and the TSP.

Suppose that the dimensions of the matrices A_1 , A_2 , A_3 , and A_4 are 20×2 , 2×15 , 15×40 , and 40×4 , respectively, and that we want to know the optimum way to compute $A_1 \times A_2 \times A_3 \times A_4$, i.e., we would like to compute this product with a minimum number of multiplications. Here we assume that to multiply two matrices, P , which is $n \times k$, by Q , which is $k \times m$, it takes nmk multiplications. The resulting matrix R is $n \times m$ and

$$r_{ij} = \sum_{v=1}^k p_{iv}q_{vj},$$

for all $1 \leq i \leq n$ and $1 \leq j \leq m$.

Note that different orders of multiplications have different costs. For example,

- $A(B(CD))$ requires $15 \cdot 40 \cdot 4 + 2 \cdot 15 \cdot 4 + 20 \cdot 2 \cdot 4 = 2680$ multiplications,
- $(AB)(CD)$ requires $20 \cdot 2 \cdot 15 + 15 \cdot 40 \cdot 4 + 20 \cdot 15 \cdot 4 = 4200$ multiplications, whereas
- $((AB)C)D$ requires $20 \cdot 2 \cdot 15 + 20 \cdot 15 \cdot 40 + 20 \cdot 40 \cdot 4 = 15800$ multiplications!

In a dynamic programming approach we create a structure $M(i, j)$ where we maintain a record of the minimum number of multiplications required to multiply matrices from A_i to A_j ($i \leq j$). Clearly, $M(1, 1) = M(2, 2) = M(3, 3) = M(4, 4) = 0$, as no multiplications are required in these cases. Note that the problem involves finding $M(1, 4)$.

The connection between solutions of the smaller problems and a bigger one is

$$M(i, j) = \min_{i \leq k < j} \{M(i, k) + M(k + 1, j) + \text{cost}_{ij}^k\},$$

where cost_{ij}^k is the number of multiplications required for multiplying the product $A_i \dots A_k$ by $A_{k+1} \dots A_j$. The point is that to multiply a sequence from A_i to A_j in the optimal way, we must find the optimal breaking point k such that the total number of multiplications required for calculating the product $A_i \dots A_k$, which is $M(i, k)$, the product $A_{k+1} \dots A_j$, which is $M(k + 1, j)$, and these two products together (which is cost_{ij}^k) is minimal.

Keeping this in mind, we can proceed as follows. It’s a basic exercise to get:

$$\begin{aligned} M(1, 2) &= 600, \\ M(2, 3) &= 1200, \\ M(3, 4) &= 2400, \end{aligned}$$

because there's no room for a breaking point in multiplying two matrices. Then,

$$\begin{aligned} M(1, 3) &= 2800 \\ M(2, 4) &= 1520. \end{aligned}$$

Note that $M(1, 3)$ is the smaller of two possibilities

$$\begin{aligned} M(1, 1) + M(2, 3) + \text{cost}_{13}^1 &= 0 + 1200 + 1600 = 2800, \text{ and} \\ M(1, 2) + M(3, 3) + \text{cost}_{13}^2 &= 600 + 0 + 12000 = 12600. \end{aligned}$$

Similarly, $M(2, 4)$ is the smaller of two possibilities

$$\begin{aligned} M(2, 2) + M(3, 4) + \text{cost}_{24}^2 &= 0 + 2400 + 120 = 2520, \text{ and} \\ M(2, 3) + M(4, 4) + \text{cost}_{24}^3 &= 1200 + 0 + 320 = 1520. \end{aligned}$$

Finally, we find

$$M(1, 4) = 1680,$$

as the smallest value from three possibilities

$$\begin{aligned} M(1, 1) + M(2, 4) + \text{cost}_{14}^1 &= 0 + 1520 + 160 = 1680, \\ M(1, 2) + M(3, 4) + \text{cost}_{14}^2 &= 600 + 2400 + 1200 = 4200, \text{ and} \\ M(1, 3) + M(4, 4) + \text{cost}_{14}^3 &= 2800 + 0 + 3200 = 6000. \end{aligned}$$

Thus, the minimal cost in terms of the number of multiplications is 1680, but we still need to find the corresponding order of matrix multiplications. To find this best order we need an additional data structure, $O(i, j)$, where we keep the index of the best breaking point. In other words, $O(i, j) = k$ if and only if $M(i, j)$ attains the minimum value for $M(i, k) + M(k+1, j) + \text{cost}_{ij}^k$. The indices kept in the matrix O reveal the order we were searching for: $A_1((A_2A_3)A_4)$.

Let's conclude this section by considering an example of a dynamic program for the TSP. It's a lengthy example, so be forewarned. To make things meaningful, let's tackle a five-city TSP. The matrix L of distances between cities is given below.

$$L = \begin{bmatrix} 0 & 7 & 12 & 8 & 11 \\ 3 & 0 & 10 & 7 & 13 \\ 4 & 8 & 0 & 9 & 12 \\ 6 & 6 & 9 & 0 & 10 \\ 7 & 7 & 11 & 10 & 0 \end{bmatrix}$$

The cost of traveling from city i to city j is given in the i -th row and j -th column of the matrix L . For example, the distance between cities 1 and 3 is $L(1, 3) = 12$. This distance is different than the distance between cities 3 and 1, which is $L(3, 1) = 4$; therefore, we're facing an asymmetric TSP. Note that $L(i, i) = 0$ for all $1 \leq i \leq n$. Figure 4.8 illustrates the case.

The tour for the TSP is a cycle that visits *every* city once and only once, so the starting city is not of any real significance. Suppose we start from city 1. Now, we're ready to split the problem into smaller problems. Let $g(i, S)$ denote the length of the shortest path from city i to city 1 that passes through *each* city listed in the set S exactly once. Thus, in particular,

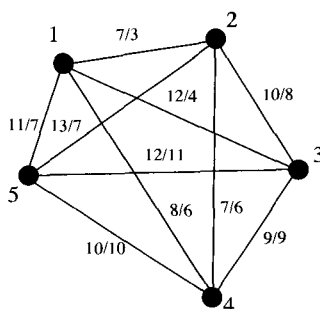


Fig. 4.8. A TSP with five cities. For each pair of cities, two numbers are given separated by a slash. The first number provides the distance from the city with the lower index to the city with the larger index, whereas the second number represents the distance in the opposite direction.

$$g(4, \{5, 2, 3\})$$

represents the shortest path that leads from city 4 through cities 5, 2, and 3 (in some unspecified order) and then returns to city 1. To find the length of the shortest complete tour for the TSP we need to find

$$g(1, V - \{1\}),$$

where V represents the set of *all* the cities for the TSP. In other words, we're trying to find the shortest path that starts from city 1, visits every city in the set $V - \{1\}$ exactly once, and returns to city 1.

The dynamic programming formulation provides a connection between solutions of the smaller problems and a bigger one. In general, we claim that

$$g(i, S) = \min_{j \in S} \{L(i, j) + g(j, S - \{j\})\}.$$

In other words, to select the shortest path that originates at city i and visits every city in S before returning to city 1, we have to find a city j from the set S such that the sum of $L(i, j)$ and its continuation, $g(j, S - \{j\})$, is minimized. If we know the solutions for the smaller problems (i.e., the problems where the cardinality of S is k), we can obtain a solution for a bigger problem (i.e., where the cardinality of S is $k + 1$).

Let's return to the example. The problem is to find

$$g(1, \{2, 3, 4, 5\}).$$

Note also that

$$\begin{aligned} g(2, \emptyset) &= L(2, 1) = 3, \\ g(3, \emptyset) &= L(3, 1) = 4, \\ g(4, \emptyset) &= L(4, 1) = 6, \text{ and} \\ g(5, \emptyset) &= L(5, 1) = 7, \end{aligned}$$

since the set S is empty and we proceed directly from the city i ($i = 2, 3, 4, 5$) to city 1. The next iteration is also straightforward: we need to find the solutions to all the problems where the cardinality of S is one. This would involve solving 12 subproblems, since we'll start from cities 2, 3, 4, and 5, and for each of these cities, we'll consider three possible singleton sets. For example, for city 2 we have

$$\begin{aligned} g(2, \{3\}) &= L(2, 3) + g(3, \emptyset) = 10 + 4 = 14, \\ g(2, \{4\}) &= L(2, 4) + g(4, \emptyset) = 7 + 6 = 13, \text{ and} \\ g(2, \{5\}) &= L(2, 5) + g(5, \emptyset) = 13 + 7 = 20. \end{aligned}$$

Note that the result $g(2, \{3\}) = 14$ means that the length of the shortest path from city 2 to 1, which before arriving at 1 goes through city 3, is 14. This result is obvious since S consists of only a single element and there is no room for any choices. Similarly, for city 3 we have

$$\begin{aligned} g(3, \{2\}) &= L(3, 2) + g(2, \emptyset) = 8 + 3 = 11, \\ g(3, \{4\}) &= L(3, 4) + g(4, \emptyset) = 9 + 6 = 15, \\ g(3, \{5\}) &= L(3, 5) + g(5, \emptyset) = 12 + 7 = 19, \end{aligned}$$

for city 4,

$$\begin{aligned} g(4, \{2\}) &= L(4, 2) + g(2, \emptyset) = 6 + 3 = 9, \\ g(4, \{3\}) &= L(4, 3) + g(3, \emptyset) = 9 + 4 = 13, \\ g(4, \{5\}) &= L(4, 5) + g(5, \emptyset) = 10 + 7 = 17, \end{aligned}$$

and for city 5,

$$\begin{aligned} g(5, \{2\}) &= L(5, 2) + g(2, \emptyset) = 7 + 3 = 10, \\ g(5, \{3\}) &= L(5, 3) + g(3, \emptyset) = 11 + 4 = 15, \\ g(5, \{4\}) &= L(5, 4) + g(4, \emptyset) = 10 + 6 = 16. \end{aligned}$$

Now we're ready for the next iteration of the dynamic program where the cardinality of S is two. In this iteration we have to solve 12 subproblems as well, since each of four cities can be considered along with any two other cities (out of three). Thus, for city 2 we have

$$\begin{aligned} g(2, \{3, 4\}) &= \min\{L(2, 3) + g(3, \{4\}), L(2, 4) + g(4, \{3\})\} \\ &= \min\{10 + 15, 7 + 13\} = \min\{25, 20\} = 20, \\ g(2, \{3, 5\}) &= \min\{L(2, 3) + g(3, \{5\}), L(2, 5) + g(5, \{3\})\} \\ &= \min\{10 + 19, 13 + 15\} = \min\{29, 28\} = 28, \\ g(2, \{4, 5\}) &= \min\{L(2, 4) + g(4, \{5\}), L(2, 5) + g(5, \{4\})\} \\ &= \min\{7 + 17, 13 + 16\} = \min\{24, 29\} = 24. \end{aligned}$$

The result $g(2, \{3, 4\}) = 20$ means that the length of the shortest path from city 2 to 1, which before arriving at 1 goes through cities 3 and 4 (in some order), is 20. This time we have to consider two possibilities (going first to city 3 or going first to city 4), and select the better option. Similarly, for city 3,

$$\begin{aligned}
g(3, \{2, 5\}) &= \min\{L(3, 2) + g(2, \{5\}), L(3, 5) + g(5, \{2\})\} \\
&= \min\{8 + 20, 12 + 10\} = \min\{28, 22\} = 22, \\
g(3, \{2, 4\}) &= \min\{L(3, 2) + g(2, \{4\}), L(3, 4) + g(4, \{2\})\} \\
&= \min\{8 + 13, 9 + 9\} = \min\{21, 18\} = 18, \\
g(3, \{4, 5\}) &= \min\{L(3, 4) + g(4, \{5\}), L(3, 5) + g(5, \{4\})\} \\
&= \min\{9 + 17, 12 + 16\} = \min\{26, 28\} = 26,
\end{aligned}$$

for city 4,

$$\begin{aligned}
g(4, \{2, 3\}) &= \min\{L(4, 2) + g(2, \{3\}), L(4, 3) + g(3, \{4\})\} \\
&= \min\{6 + 14, 9 + 15\} = \min\{20, 24\} = 20, \\
g(4, \{2, 5\}) &= \min\{L(4, 2) + g(2, \{5\}), L(4, 5) + g(5, \{2\})\} \\
&= \min\{6 + 20, 10 + 10\} = \min\{26, 20\} = 20, \\
g(4, \{3, 5\}) &= \min\{L(4, 3) + g(3, \{5\}), L(4, 5) + g(5, \{3\})\} \\
&= \min\{9 + 19, 10 + 15\} = \min\{28, 25\} = 25.
\end{aligned}$$

and for city 5,

$$\begin{aligned}
g(5, \{2, 3\}) &= \min\{L(5, 2) + g(2, \{3\}), L(5, 3) + g(3, \{2\})\} \\
&= \min\{7 + 14, 11 + 11\} = \min\{21, 22\} = 21, \\
g(5, \{2, 4\}) &= \min\{L(5, 2) + g(2, \{4\}), L(5, 4) + g(4, \{2\})\} \\
&= \min\{7 + 13, 10 + 19\} = \min\{20, 29\} = 20, \\
g(5, \{3, 4\}) &= \min\{L(5, 3) + g(3, \{4\}), L(5, 4) + g(4, \{3\})\} \\
&= \min\{11 + 15, 10 + 13\} = \min\{26, 23\} = 23.
\end{aligned}$$

Now we're ready for the next step, where the cardinality of the set S is three. Here we face only four subproblems: one for each city. Thus, for city 2 we have to select the shortest path among three paths:

$$\begin{aligned}
g(2, \{3, 4, 5\}) &= \min\{L(2, 3) + g(3, \{4, 5\}), L(2, 4) + g(4, \{3, 5\}), L(2, 5) + g(5, \{3, 4\})\} \\
&= \min\{10 + 26, 7 + 25, 13 + 23\} = \min\{36, 32, 34\} = 32.
\end{aligned}$$

The result $g(2, \{3, 4, 5\}) = 32$ means that the length of the shortest path from city 2 to 1, which before arriving at 1 goes through cities 3, 4, and 5 (in some order), is 32. Now we have to consider three possibilities, going first to city 3, 4, or 5, and select the best option. Similarly, for cities 3, 4, and 5 we have

$$\begin{aligned}
g(3, \{2, 4, 5\}) &= \min\{L(3, 2) + g(2, \{4, 5\}), L(3, 4) + g(4, \{2, 5\}), L(3, 5) + g(5, \{2, 4\})\} \\
&= \min\{8 + 24, 9 + 20, 12 + 20\} = \min\{32, 29, 32\} = 29, \\
g(4, \{2, 3, 5\}) &= \min\{L(4, 2) + g(2, \{3, 5\}), L(4, 3) + g(3, \{2, 5\}), L(4, 5) + g(5, \{2, 3\})\} \\
&= \min\{6 + 28, 9 + 22, 10 + 21\} = \min\{34, 31, 31\} = 31, \text{ and} \\
g(5, \{2, 3, 4\}) &= \min\{L(5, 2) + g(2, \{3, 4\}), L(5, 3) + g(3, \{2, 4\}), L(5, 4) + g(4, \{2, 3\})\} \\
&= \min\{7 + 20, 11 + 18, 10 + 20\} = \min\{27, 29, 30\} = 27.
\end{aligned}$$

Now, it's time for the final iteration where we start from city 1 and close the cycle. Thus we return to the original problem:

$$g(1, \{2, 3, 4, 5\}) = ?$$

Starting from city 1 we have four options: to go first either to city 2, city 3, city 4, or city 5. The other subproblems (i.e., the shortest lengths for each continuation of each possible beginning) are already known, so it's easy to make the final decision:

$$\begin{aligned} g(1, \{2, 3, 4, 5\}) &= \min\{L(1, 2) + g(2, \{3, 4, 5\}), L(1, 3) + g(3, \{2, 4, 5\}), \\ &\quad L(1, 4) + g(4, \{2, 3, 5\}), L(1, 5) + g(5, \{2, 3, 4\})\} = \\ &\quad \min\{7 + 32, 12 + 29, 8 + 31, 11 + 27\} = \min\{39, 41, 39, 38\} = 38. \end{aligned}$$

Thus the shortest tour has a length of 38.

Which tour is that? As with the matrix multiplication problem, we've found the optimum value of the evaluation function but we still have to find out which tour has given us this result. Again, this is easy if we keep track of our calculations. We need an additional data structure, W , that provides information on the next city that should be chosen to minimize a path. Thus, for example,

$$W(5, \{2, 3, 4\}) = 2,$$

since the shortest path from city 5 to city 1, which has to pass through cities 2, 3, and 4 before arriving at 1, must go to city 2 first. Similarly,

$$W(1, \{2, 3, 4, 5\}) = 5.$$

Thus, the global solution for this TSP is given by the tour

$$1 - 5 - 2 - 4 - 3 - 1,$$

and the length of this tour is, of course,

$$11 + 7 + 7 + 9 + 4 = 38.$$

Does the thought of using dynamic programming to solve a 50-city TSP strike fear in your heart?

4.4 Branch and bound

We've seen that the size of real-world problems can grow very large as the number of variables in the problem increases. Recall that there are $(n-1)!/2$ different solutions for the symmetric TSP. Exhaustive search is out of the question for $n > 20$, so it would be helpful to have some heuristic that eliminates parts of the search space where we know that we won't find an optimum solution.

Branch and bound is one such heuristic that works on the idea of successive partitioning of the search space. We first need some means for obtaining a lower bound on the cost for any particular solution (or an upper bound depending on

whether we're minimizing or maximizing). The idea is then that if we have a solution with a cost of say, c units, we know that the next solution to try has a lower bound that is greater than c , and we are minimizing, we don't have to compute just how bad it actually is. We can forget about that one and move on to the next possibility.

It helps to think about the search space as being organized like a tree. The heuristic of branch and bound prunes away branches that aren't of interest. For example, let's consider a five-city symmetric TSP. The entire search space $\mathcal{S}$ can be partitioned on the basis of, say, whether or not the edge $(1\ 2)$ belongs to a tour. Similarly, the space can be partitioned further with respect to whether or not the edge $(2\ 3)$ is included, and so forth. Figure 4.9 shows a tree that represents this principle. The leaves of the tree represent $(5-1)!/2 = 12$ possible tours.

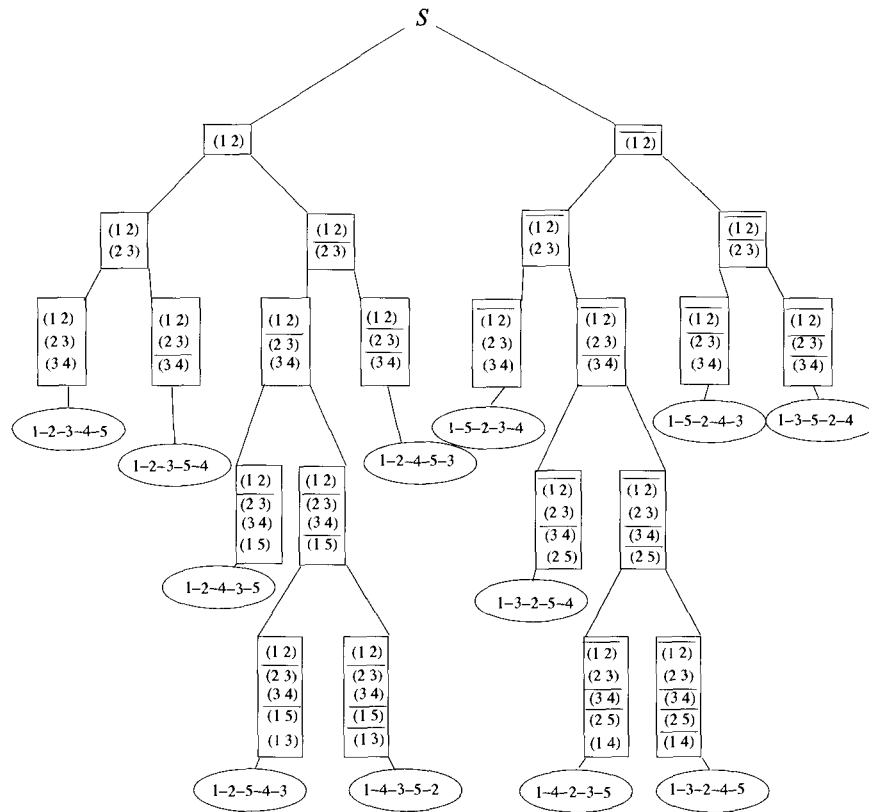


Fig. 4.9. A partition of the search space $\mathcal{S}$ of the five-city TSP according to whether a given edge $(i\ j)$ belongs to the tour, or not. In the latter case, a horizontal bar is indicated over the edge. The edges were selected in random order. The implied tours are given as well.

Suppose that the cost of traveling between cities was described by the following cost matrix:

$$\begin{bmatrix} 0 & 7 & 12 & 8 & 11 \\ 7 & 0 & 10 & 7 & 13 \\ 12 & 10 & 0 & 9 & 12 \\ 8 & 7 & 9 & 0 & 10 \\ 11 & 13 & 12 & 10 & 0 \end{bmatrix}$$

where each entry is the cost of traveling from a city in the i -th row to one in the j -th column. The zeros down the main diagonal indicate that we can't travel from a city to itself.

Given the organization of the search space as a tree, we need a heuristic for estimating a lower bound on the cost for any final solution, or even any node. If that lower bound is higher than the cost of the best solution we've found so far, we can discard the new solution and keep looking without having to actually compute its exact cost.

Here's a simple but perhaps not very effectual way to compute a lower bound for a tour. Consider an instance of a complete solution to the TSP. Every tour comprises two adjacent edges for every city: one edge gets you to the city, the other edge takes you on to the next city. Thus, if we select the two shortest edges that are connected to each city and take the sum of the lengths of these edges divided by two, we'll obtain a lower bound. Indeed, we couldn't possibly do any better because this rule selects the very best edges for every city. With respect to the above matrix, the lower bound over all possible tours is:

$$[(7 + 8) + (7 + 7) + (9 + 10) + (7 + 8) + (10 + 11)]/2 = 84/2 = 42.$$

Note that 7 and 8 in the first parentheses correspond to the lengths of the two shortest edges connected to city 1, whereas the entries 7 and 7 in the second parentheses correspond to the two shortest edges connected to city 2, and so forth.

Once some edges are specified for a tour we can incorporate that information and calculate a lower bound for that partial solution. For instance, if we knew that edge (2 3) was included but edge (1 2) was not, then the lower bound for this partial solution would be

$$[(8 + 11) + (7 + 10) + (9 + 10) + (7 + 8) + (10 + 11)]/2 = 91/2 = 45.5.$$

We can improve the lower bound by including the implied edges or excluding those that cannot occur. If edges (1 2) and (2 4) were included in a tour, from what's above we'd determine the lower bound to be 42. Upon closer examination, however, we can exclude the edge (1 4) here, so a better lower bound is

$$[(7 + 11) + (7 + 7) + (9 + 10) + (7 + 9) + (10 + 11)]/2 = 88/2 = 44.$$

The better the lower bound, the faster the algorithm for searching through the tree will be because it will eliminate more solutions. On the other hand, there is

a cost for computing these lower bounds. For the method to be useful, the cost for computing the bounds has to be made up for by the time saved in pruning the tree. So we really want the lower bounds to be as “tight” as possible.

There are many possibilities for improving the lower bound in the TSP but they’re beyond our scope here (see [344]). Instead, let’s turn our attention back to numerical optimization and consider how branch and bound can be applied.

Assume that the task is to

$$\text{minimize } f(\mathbf{x}),$$

where $\mathbf{x} \in D \subseteq \mathbb{R}^n$. In other words, we search for $\mathbf{x}^* = \{\mathbf{x} \in D \mid f(\mathbf{x}) = f^*\}$, where $f^* = \inf_{\mathbf{x} \in D} f(\mathbf{x})$. Using the notation from [296],

- D_i is an n -dimensional box in $\mathbb{R}^n$, i.e., D_i is defined by $\mathbf{l}^i$ and $\mathbf{u}^i$, and $\mathbf{x} \in D_i$ iff $l_k^i \leq x_k \leq u_k^i$ for all $1 \leq k \leq n$.
- C , termed the candidate set, is a set of boxes D_i , $i = 1, \dots, p$. For each box D_i , we obtain a lower bound of the function values of f in the box, i.e., a lower bound for f on D_i .
- f_{bound} represents the current upper limit of f^* found by the algorithm. This value is obtained, for example, as the lowest function value calculated so far.

The idea is that initially $C = \{D\}$ and the branch and bound method will reduce the set C and make it converge to the set of global minimizers $\mathbf{x}^*$. Note, however, that the issue of convergence is not straightforward here. The process depends on the selection mechanism of boxes. If the box with the smallest lower bound is always selected, the algorithm may never converge. On the other hand, if the largest box is always selected — according to some size measure such as the total length of all box’s dimensions — the algorithm will converge, but the running time might be prohibitive. Also, the term “convergence” is topology dependent, as a sequence of boxes may only approximate a curve.

Figure 4.10 displays the skeleton of the branch and bound algorithm. At any iteration, C maintains a set of active boxes D_i ’s; at any time the set of global minima $\mathbf{x}^*$ is included in the union of these boxes:

$$\mathbf{x}^* \in \bigcup D_i \subseteq D.$$

A single box D_i is selected and removed from the candidate set C at each iteration. There are many methods for choosing which box to remove, but one common method is to select a box with the smallest lower bound. We then try to reduce the size of D_i or to eliminate it altogether. This is done by testing the evaluation function f for monotonicity in D_i (i.e., its partial derivatives are always positive or negative). If this is the case for some dimension i then the box is reduced to a single point along that dimension on the boundary of the

```

procedure branch and bound
begin
  initialize  $C$ 
  initialize  $f_{\text{bound}}$ 
  while (not termination-condition) do
    remove-best-box  $C \rightarrow D_i$ 
    reduce-or-subdivide  $D_i \rightarrow D'_i$ 
    update  $f_{\text{bound}}$ 
     $C \leftarrow C \cup D'_i$ 
    for all  $D_i \in C$  do
      if (lower bound of  $f(D_i)$ )  $> f_{\text{bound}}$  then remove  $D_i$  from  $C$ 
end

```

Fig. 4.10. Branch and bound algorithm

box.³ If it isn't possible to reduce D_i , then it's replaced by two or more boxes. In any case, the candidate set C is extended by D'_i , which represents either the reduced box D_i or a subdivision of D_i . Of course, new lower bounds are obtained before D'_i is inserted into C . Note also that if a lower bound of a box is greater than f_{bound} , the box can be eliminated from the candidate set.

There are many variants of this generic branch and bound algorithm. It's possible to use an interval arithmetic version of this approach where all of the calculations are performed on intervals rather than on real numbers, or a stochastic version where the values of f are calculated at a number of random points. For more details, see [296]. For general information on interval methods for global optimization, see [378].

4.5 A^* algorithm

We've seen that taking the best available move at any given time can lead to trouble. Greedy algorithms don't always perform very well. The problem is that what's better now might not be what's needed later, but if we had an evaluation function that was sufficiently "informative" to avoid these traps, we could use such greedy methods to a better advantage. This simple idea leads to a concept called *best-first* search, and its extension, the A^* algorithm.

When we can organize the search space as a tree, which is a special case of a directed graph and we'll return to this point shortly, we have the option of how to search that tree. We can use depth-first search and proceed down the tree to some prescribed level before making any evaluations and then backtrack our way through the tree. Or we might instead try to order the available nodes

³Additional tests can be done as well, e.g., Newton-reduction can be applied [296]. This can result in a decision that D_i does not contain stationary points — all iterative sequences of a search for stationary points go out of D_i — or that there is one stationary point, e.g., all iterative sequences converge to the same point.

according to some heuristic that corresponds with our expectations of what we'll find when we get to the deepest level of our search. We'd want to search first those nodes that offer the best chance of finding something good. A skeleton of the best-first search algorithm is offered in figure 4.11.

```

procedure best-first( $v$ )
begin
  visit  $v$ 
  for each available  $w$  do
    assign a heuristic value for  $w$ 
   $q \leftarrow$  the best available node
  best-first( $q$ )
end

```

Fig. 4.11. Best-first search

The key issue is hidden in the concept of *available* nodes. The algorithm maintains two lists of nodes: *open* and *closed*. The former indicates all of the available nodes, whereas the latter indicates those nodes that have already been processed. When the best-first procedure starts from a particular node, which is placed on the *closed* list, all its children are moved to the *open* list (i.e., they become available). They are evaluated by some heuristic and the best is selected for further processing.

If the search space is a general directed graph, one of the children of a node might already be present in the open or closed lists. If that happens, you have to re-evaluate that node, and you might need to move it from the closed list to the open list. We'll ignore this possibility and focus solely on search spaces that are organized as a tree. For example, figure 4.12 provides the tree for the TSP in figure 4.8.

The main differences between depth-first search, its sister breadth-first search, and best-first search are

- Best-first search explores the most promising next node, whereas depth-first search goes as deep as possible in an arbitrary pattern and breadth-first search explores all the nodes on one level before moving to the next.
- Best-first search uses a heuristic that provides a merit value for each node, whereas depth-first search and breadth-first search do not.

The efficiency of the best-first algorithm is closely connected with the quality of the heuristic, so properly designing such heuristics demands more attention.

In the process of evaluating a partially constructed solution, q , we should take into account its two components:

- The merit of the decisions already made, $c(q)$.
- The potential inherent to the remaining decisions, $h(q)$.

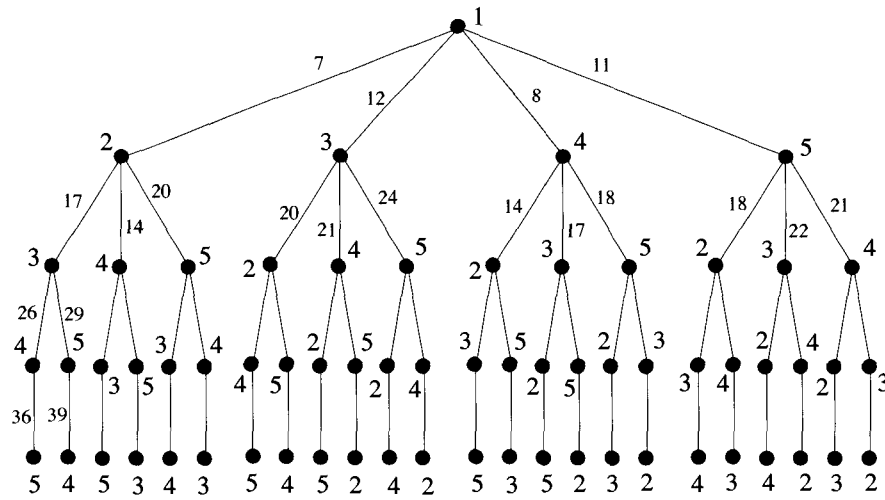


Fig. 4.12. A search space for the TSP with five cities. A number given next to some edges represents the cost of this partial solution including the node at the end of this edge, defined later as a value of c for this node. Note that the cost of the final solution requires the addition of the final edge to close the cycle.

Thus, the evaluation function $eval$ for a partial solution q is

$$eval(q) = c(q) + h(q).$$

Sometimes the problem statement provides an exact measure for c . It's relatively easy to evaluate a merit of the decisions already made in a TSP. A partial tour

$$q = 1 - 3 - 5$$

can be evaluated as

$$c(q) = dist(1, 3) + dist(3, 5).$$

It's much harder to estimate the potential quality of the remaining decisions. This is why we need a good heuristic h . But how can we judge the quality of a particular heuristic?

One criterion for judging a heuristic h relies on what's termed *admissibility*. An algorithm is *admissible* if it always terminates at the optimum solution. We would like to choose h so that we guarantee admissibility. Suppose we had some *oracle* that could provide the real cost h^* of continuing on to the goal. The *ideal* evaluation function $eval^*$ would be

$$eval^*(q) = c(q) + h^*(q),$$

which returns the real cost of the optimum solution having reached node q . It's easy to show that the best-first algorithm using $eval^*$ is admissible, but we don't

have $eval^*$ at our disposal since it's usually impossible to find an all-knowing oracle h^* . Thus, we have to estimate h^* . We can replace it by some estimate h of the minimum cost of continuing to the goal. If h is bounded from above,⁴ i.e., it's never greater than the real cost of the best continuation,

$$h(q) \leq h^*, \text{ for all nodes } q,$$

then the resulting best-first algorithm is always admissible, and this special case is called the A^* algorithm. The reason that A^* guarantees the global solution is hidden in the above inequality. Since h underestimates the remaining cost of the solution, it's impossible to miss the best path!

It's possible to have several heuristics h_i to estimate the remaining cost to the goal. Given two heuristics,

$$h_1(q) \leq h^*(q) \text{ and } h_2(q) \leq h^*(q),$$

such that $h_1(q) \leq h_2(q)$ for all nodes q , heuristic h_2 is better⁵ because it provides us with a tighter bound.

It's interesting to indicate some connections among various search techniques discussed earlier. For example, a best-first search with the evaluation function

$$eval(q) = c(q) + h(q)$$

for a partial solution q can be viewed as a breadth-first search, if

$$\begin{aligned} h(q) &= 0 \text{ for all } q, \text{ and} \\ c(q') &= c(q) + 1, \text{ where } q' \text{ is a successor of } q, \end{aligned}$$

whereas it can be viewed as a depth-first search, if

$$\begin{aligned} h(q) &= 0 \text{ for all } q, \text{ and} \\ c(q') &= c(q) - 1, \text{ where } q' \text{ is a successor of } q. \end{aligned}$$

Note also that when

$$h(q) = 0 \text{ for all } q, \text{ and } c(q) = \text{the real cost of the path from the start to node } q,$$

we have a branch and bound method!

We limited the discussion to the case when the search space is organized as a tree. But you can generalize the above algorithms to handle arbitrary graphs [354]. Doing so requires considering the cases of revisiting a node, possibly updating its evaluation value if the cost of a newly discovered path is better than the cost of the best-known path.

⁴For minimization problems.

⁵In artificial intelligence terminology, h_2 is *more informed*.

4.6 Summary

If you're observant, you might have noticed that the topic of "traditional methods" spans 50 pages of this book over two chapters. And yet, we argued forcefully in the Preface and Introduction that the traditional methods of problem solving leave much to be desired. The length of chapters 3 and 4 does not imply value, rather in this case it implies the weakness of each of the methods. Each is designed for a particular type of problem and treats that problem, and only that problem, efficiently. Once you change the problem, the traditional methods start to falter, and sometimes they fail altogether.

If you face a problem that poses a quadratic evaluation function you should use Newton-Gauss. If your situation concerns a linear evaluation function with linear equality and inequality constraints then the simplex method will yield the best solution. You need to know when each of the methods in chapters 3 and 4 will yield the global optimum solution to your problem and when they will fall short.

More than learning a set of individual methods, however, you should notice that, leaving the details aside, there are two main approaches to problem solving. You can either work with complete solutions or partial solutions. The former offers the advantage that you always have some answer ready to put into practice, even when you run out of time or if your algorithm converges to something less than best. The latter offers the potential payoff of taking advantage of the structure that's inherent in some problems. You might be able to obtain a tremendous speed up in generating an answer to a seemingly complicated problem by decomposing it into subproblems that are trivial. You might be able to organize your search space in the form of a tree and then use something like A^* to help you find the best solution in the least amount of time.

But again, you have to know that if you decompose a problem you may find that generating answers to each of the smaller subproblems doesn't yield any nice way of assembling those answers into an overall global solution. You can disassemble a 100-city TSP into twenty 5-city TSPs, each of which is trivial to solve. But what will you do with these 20 answers?! Most real-world problems do not yield to the traditional methods. This statement is almost tautological: If they could be solved by classic procedures they wouldn't be problems anymore. Thus, we usually face the case where the problem seems intractable, where there are lots of locally optimal solutions and most of these are unacceptable. For these problems, we need tools that go beyond the traditional methods.

V. What's the Color of the Bear?

Once you've found a solution to a problem, it's very tempting to quit. After all, the problem's solved, right? Well, not quite. It could be that there are multiple solutions, and it could be that some are more correct than others. Just like solutions that are only locally optimal, everything can seem fine with the solution that you have, but if you could only see a way to generate some other solution you might escape your current trap and leap into something better. Sometimes it helps to stand a problem on its head to find those other solutions. With that in mind, let's try the following puzzle about the color of a bear and a famous explorer, Mr. Brown.

One morning Mr. Brown left his tent and walked one mile south. Then he turned east and continued for the second mile. At this point he turned north, walked for another mile and arrived right back at his tent. While he was out walking, Mr. Brown saw a bear. What color was it?

As with some other problems, it might be hard to find a starting point. But you can quickly imagine that the key must be hidden in the special characteristics of Mr. Brown's sojourn. If we could find out where in the world Mr. Brown was, maybe that would give us the crucial information that we need.

After a bit of consideration about geometry on the globe, you'll likely come up with the answer that Mr. Brown had his tent on the North Pole. This seems like the only way to complete a loop going one mile south, east, and north (see figure V.1).

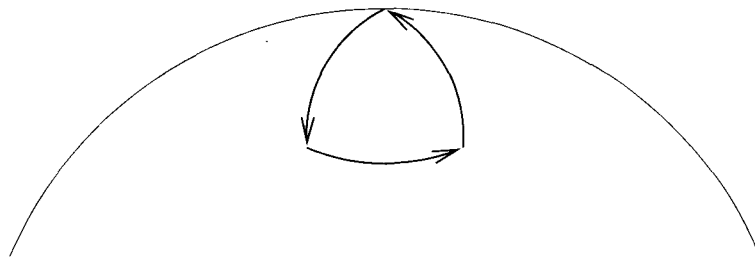


Fig. V.1. Mr. Brown's sojourn

Clearly, the bear must have been white: there are no other bears around the North Pole! If you were a betting person, you'd be thinking: "Well, they wouldn't make the color of the bear the same as the guy's name now, would they?"

Ah, but is this a unique solution? At first inspection, it seems to be unique. As we depart south, there's no other way of completing a loop by going one mile south, east, and north.

Not so fast! After some additional thought we can indeed identify another solution!

Consider a parallel on the Southern Hemisphere that has a circumference of one mile (parallel *A* in figure V.2). We can select an arbitrary point on this parallel and move 1 mile north to another parallel (parallel *B* in figure V.2). This is a possible location of Mr. Brown's tent. After leaving this place he goes south for one mile (thus arriving on parallel *A*), turns east and continues for one mile (thus completing a full circle), and returns north directly to his tent. Mr. Brown's tent can be located, of course, at any point on parallel *B*. It's not that there's just one more solution, there's an *infinitude* of solutions!

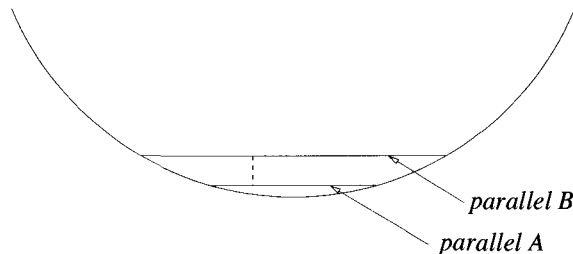


Fig. V.2. Possible locations of Mr. Brown's tent

So, is that it then? Have we found the complete infinity of solutions?

Nope. After some additional pondering it's clear that there are still more solutions because parallel *A* may have a circumference of $\frac{1}{2}$ mile, $\frac{1}{3}$ mile, etc. And in every one of these cases Mr. Brown would complete a whole number of circles always returning to the original point! Since there aren't any bears near the South Pole, however, these infinities of additional solutions don't change the answer to the real question, which we recall was to find the color of the bear.

By the way, there's another famous (and very flawed) version of the above puzzle. It goes like this: Mr. Brown, a famous explorer, celebrated his 70th birthday. During the party, a lady asked him what he did exactly 10 years ago. After some thought Mr. Brown replied, "I remember that day very well. I left my tent at sunrise and walked one mile south. Then I turned east and continued for the second mile. At this point I turned north, walked for another mile and arrived back at my tent." Now, the question is: what is Mr. Brown's birthday?

The puzzle assumes that there is only one possible location of the tent: the North Pole. As there's only one sunrise per year at this special location, which happens on March 21, Mr. Brown's birthday must be March 21. But as we know now, there are many more possible locations for his tent!

The point here is don't give up once you find a solution. Keep thinking! See if you have really exhausted all the possibilities. You might find something that you hadn't considered originally, and it might be even better than your first solution. You'll never know if you give up right away. Also, it helps to get a

mental picture of the situation. Have in mind a picture of Mr. Brown out with a walking stick and a telescope exploring the wilderness. You'll more quickly find that wilderness turning into ice floes and glaciers. If you don't have a great imagination, then draw a picture on a piece of paper. Just having something to look at can help you frame the problem and think of ways to solve it.

Let's try this out and start with a picture while introducing an additional puzzle.

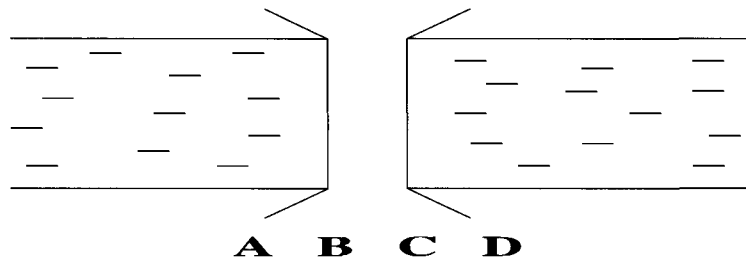


Fig. V.3. Four travelers in front of a bridge

Four travelers (let's call them Mr. A, B, C, and D) have to cross a bridge over a deep ravine (see Figure V.3). It's a very dark night and the travelers have but a single source of light: an old fashioned oil lamp. The light is essential for successfully crossing the ravine because the bridge is very old and has plenty of holes and loose boards. What's worse, its construction is really quite weak, and in its dilapidated condition, it can only at best support two of the men at any time. The question is how should the men arrange themselves to cross the bridge? The oil lamp has a limited supply of fuel. Time is running out.

Well, since only two travelers can be on the bridge at any one time, we know that they have to go across either individually or in pairs, but since they also must have some light to ensure that they don't fall through a hole in the bridge, we also know that it would be impossible for any one individual to carry the lamp without having a partner go with them. So no matter what the solution, it has to involve a sequence of pairs of travelers going across the bridge. But which pairs?

Here comes the really interesting part of the puzzle. It turns out that each traveler needs a different amount of time to cross the bridge. Mr. A is young and healthy and needs but a minute to quickly traverse the bridge. Mr. D, on the other hand, is an old man who recently had a hip replacement and will need 10 minutes to get across the bridge. Mr. B and Mr. C need two minutes and five minutes, respectively. And since each traveler needs the light to cross, whenever a pair of travelers go together, it is the slower man who determines the total time required to make the crossing.

So, now that you have all the information (think to yourself: do I really have all the information? Is there something else I might need to know?), what is the best way to schedule the travelers to get to the other side of the ravine in the shortest possible time?

Your first thought might be to send the quickest traveler, Mr. A, across the bridge with each man in turn. Mr. A could carry the lamp. So Mr. A and Mr. B could go across together – this would take two minutes – then Mr. A would return with the lamp which would take an additional minute. Then Mr. A could go across with Mr. C and then come back to do the same thing again with Mr. D. In all, this would require 19 minutes. But actually, there is a better way for them to accomplish their task. Can you find the solution?

Once you solve this puzzle, it should be easy to solve similar puzzles. For example, six travelers approach the same bridge and their respective times for crossing the bridge are 1, 3, 4, 6, 8, and 9 minutes. Again, what's the best way to schedule them to get to the other side in the shortest time? Suppose instead that there are seven travellers, with crossing times of 1, 2, 6, 7, 8, 9, and 10 minutes, but in this instance the bridge is more modern and can handle three travelers on the bridge at any time....

5. Escaping Local Optima

O! for a horse with wings!

Shakespeare, *Cymbeline*

We've discussed a few traditional problem-solving strategies. Some of them guarantee finding the global solution, others don't, but they all share a common pattern. Either they guarantee discovering the global solution, but are too expensive (i.e., too time consuming) for solving typical real-world problems, or else they have a tendency of "getting stuck" in local optima. Since there is almost no chance to speed up algorithms that guarantee finding the global solution, i.e., there is almost no chance of finding polynomial-time algorithms for most real problems (as they tend to be NP-hard), the other remaining option aims at designing algorithms that are capable of escaping local optima.

How can we do this? How can we design "a horse with wings"? We have already seen one possibility in chapter 2, where we discussed a procedure called an "iterated hill-climber." After reaching a local optimum, a new starting point is generated and the search is commenced all over again. Clearly, we can apply such a strategy with any other algorithm, but let's discuss some possibilities of escaping local optima within a single run of an algorithm.

Two main approaches that we'll discuss in this chapter are based on (1) an additional parameter (called *temperature*) that changes the probability of moving from one point of the search space to another, or (2) a memory, which forces the algorithm to explore new areas of the search space. These two approaches are called *simulated annealing* and *tabu search*, respectively. We discuss them in detail in the following two sections, but first let's provide some intuition connected with these methods and compare them to a simple local search routine.

We discussed the iterated hill-climber procedure in chapter 2. Let's now rewrite this procedure by omitting several lines of programming details and by listing only the most significant concepts. Also, let's restrict the algorithm to a single sequence of improvement iterations (i.e., $MAX = 1$, see figure 2.5). In such a case, a single run of a simple *local search* procedure can be described as follows:

```

procedure local search
begin
   $x$  = some initial starting point in  $\mathcal{S}$ 
  while improve( $x$ )  $\neq$  'no' do
     $x$  = improve( $x$ )
  return( $x$ )
end

```

The subprocedure `improve( $x$ )` returns a new point y from the neighborhood of x , i.e., $y \in N(x)$, if y is better than x , otherwise it returns a string “no,” and in that case, x is a local optimum in $\mathcal{S}$.

In contrast, the procedure *simulated annealing* (discussed fully in section 5.1) can be described as follows:

```

procedure simulated annealing
begin
   $x$  = some initial starting point in  $\mathcal{S}$ 
  while not termination-condition do
     $x$  = improve?( $x$ ,  $T$ )
    update( $T$ )
  return( $x$ )
end

```

There are three important differences between simulated annealing and local search. First, there is a difference in how the procedures halt. Simulated annealing is executed until some external “termination condition” is satisfied as opposed to the requirement of local search to find an improvement. Second, the function “improve?(x , T)” doesn’t have to return a better point from the neighborhood of x .¹ It just returns an *accepted* solution y from the neighborhood of x , where the acceptance is based on the current temperature T . Third, in simulated annealing, the parameter T is updated periodically, and the value of this parameter influences the outcome of the procedure “improve?” This feature doesn’t appear in local search. Note also that the above sketch of simulated annealing is quite simplified so as to match the simplifications we made for local search. For example, we omitted the initialization process and the frequency of changing the temperature parameter T . Again, the main point here is to underline similarities and differences.

Tabu search (discussed fully in section 5.2) is almost identical to simulated annealing with respect to the structure of the algorithm. As in simulated annealing, the function “improve?(x , H)” returns an *accepted* solution y from the neighborhood of x , which need not be better than x , but the acceptance is based on the history of the search H . Everything else is the same (at least, from a high-level perspective). The procedure can be described as follows:

¹This is why we added a question mark in the name of this procedure: “improve?”.

```

procedure tabu search
begin
   $x$  = some initial starting point in  $\mathcal{S}$ 
  while not termination-condition do
     $x$  = improve?( $x$ ,  $H$ )
    update( $H$ )
  return( $x$ )
end

```

With these intuitions in mind and an understanding of the basic concepts, we're ready to discuss the details of these two interesting search methods.

5.1 Simulated annealing

Let's recall the detailed structure of the iterated hill-climber (chapter 2) because simulated annealing doesn't differ from it very much.

```

procedure iterated hill-climber
begin
   $t \leftarrow 0$ 
  initialize  $best$ 
  repeat
     $local \leftarrow \text{FALSE}$ 
    select a current point  $\mathbf{v}_c$  at random
    evaluate  $\mathbf{v}_c$ 
    repeat
      select all new points in the neighborhood of  $\mathbf{v}_c$ 
      select the point  $\mathbf{v}_n$  from the set of new points
        with the best value of evaluation function  $eval$ 
      if  $eval(\mathbf{v}_n)$  is better than  $eval(\mathbf{v}_c)$ 
        then  $\mathbf{v}_c \leftarrow \mathbf{v}_n$ 
      else  $local \leftarrow \text{TRUE}$ 
    until  $local$ 
     $t \leftarrow t + 1$ 
    if  $\mathbf{v}_c$  is better than  $best$ 
      then  $best \leftarrow \mathbf{v}_c$ 
  until  $t = MAX$ 
end

```

Fig. 5.1. Structure of an iterated hill-climber

Note again that the inner loop of the procedure (figure 5.1) *always* returns a local optimum. This procedure only “escapes” local optima by starting a new search (outer loop) from a new (random) location. There are MAX attempts

altogether. The best solution overall (*best*) is returned as the final outcome of the algorithm.

Let's modify this procedure in the following way:

- Instead of checking *all* of the strings in the neighborhood of a current point $\mathbf{v}_c$ and selecting the best one, select only one point, $\mathbf{v}_n$, from this neighborhood.
- Accept this new point, i.e., $\mathbf{v}_c \leftarrow \mathbf{v}_n$, with some probability that depends on the relative merit of these two points, i.e., the difference between the values returned by the evaluation function for these two points.

By making such a simple modification, we obtain a new algorithm, the so-called *stochastic hill-climber* (figure 5.2):

```

procedure stochastic hill-climber
begin
   $t \leftarrow 0$ 
  select a current string  $\mathbf{v}_c$  at random
  evaluate  $\mathbf{v}_c$ 
  repeat
    select the string  $\mathbf{v}_n$  from the neighborhood of  $\mathbf{v}_c$ 
    select  $\mathbf{v}_n$  with probability  $\frac{1}{1 + e^{\frac{eval(\mathbf{v}_c) - eval(\mathbf{v}_n)}{T}}}$ 
     $t \leftarrow t + 1$ 
  until  $t = MAX$ 
end

```

Fig. 5.2. Structure of a stochastic hill-climber

Let's discuss some features of this algorithm (note that the probabilistic formula for accepting a new solution is based on maximizing the evaluation function). First, the stochastic hill-climber has only one loop. We don't have to repeat its iterations starting from different random points. Second, the newly selected point is "accepted" with some probability p . This means that the rule of moving from the current point $\mathbf{v}_c$ to the new neighbor, $\mathbf{v}_n$, is probabilistic. It's possible for the new accepted point to be *worse* than the current point. Note, however, that the probability of acceptance depends on the difference in merit between these two competitors, i.e., $eval(\mathbf{v}_c) - eval(\mathbf{v}_n)$, and on the value of an additional parameter T . Also, T remains constant during the execution of the algorithm. Looking at the probability of acceptance:

$$p = \frac{1}{1 + e^{\frac{eval(\mathbf{v}_c) - eval(\mathbf{v}_n)}{T}}},$$

what is the role of the parameter T ? Assume, for example, that the evaluation for the current and next points are $eval(\mathbf{v}_c) = 107$ and $eval(\mathbf{v}_n) = 120$ (remember, the formula is for maximization problems only; if you are minimizing, you

have to reverse the minuend and subtrahend of the subtraction operation). The difference in our example is $eval(\mathbf{v}_c) - eval(\mathbf{v}_n)$ or -13 , meaning that the new point $\mathbf{v}_n$ is better than $\mathbf{v}_c$. What is the probability of accepting this new point based on different values of T ? Table 5.1 details some individual cases.

Table 5.1. Probability p of acceptance as a function of T for the case where the new trial $\mathbf{v}_n$ is 13 points better than the current solution $\mathbf{v}_c$

T	$e^{-\frac{13}{T}}$	p
1	0.000002	1.00
5	0.0743	0.93
10	0.2725	0.78
20	0.52	0.66
50	0.77	0.56
10^{10}	0.9999...	0.5...

The conclusion is clear: the greater the value of T , the smaller the importance of the relative merit of the competing points $\mathbf{v}_c$ and $\mathbf{v}_n$! In particular, if T is huge (e.g., $T = 10^{10}$), the probability of acceptance approaches 0.5. The search becomes random. On the other hand, if T is very small (e.g., $T = 1$), the stochastic hill-climber reverts into an ordinary hill-climber! Thus, we have to find an appropriate value of the parameter T for a particular problem: not too low and not too high.

To gain some additional perspective, suppose that $T = 10$ for a given run. Let's also say that the current solution $\mathbf{v}_c$ evaluated to 107, i.e., $eval(\mathbf{v}_c) = 107$. Then, the probability of acceptance depends only on the value of the new string $\mathbf{v}_n$ as shown in table 5.2.

Table 5.2. Probability of acceptance as a function of $eval(\mathbf{v}_n)$ for $T = 10$ and $eval(\mathbf{v}_c) = 107$

$eval(\mathbf{v}_n)$	$eval(\mathbf{v}_c) - eval(\mathbf{v}_n)$	$e^{\frac{eval(\mathbf{v}_c) - eval(\mathbf{v}_n)}{10}}$	p
80	27	14.88	0.06
100	7	2.01	0.33
107	0	1.00	0.50
120	-13	0.27	0.78
150	-43	0.01	0.99

Now the picture is complete: if the new point has the same merit as the current point, i.e., $eval(\mathbf{v}_c) = eval(\mathbf{v}_n)$, the probability of acceptance is 0.5. That's reasonable. It doesn't matter which you choose because each are of equal quality. Furthermore, if the new point is better, the probability of acceptance is greater than 0.5. Moreover, the probability of acceptance grows together

with the (negative) difference between these evaluations. In particular, if the new solution is *much* better than the current one (say, $eval(\mathbf{v}_n) = 150$), the probability of acceptance is close to 1.

The main difference between the stochastic hill-climber and simulated annealing is that the latter changes the parameter T during the run. It starts with high values of T making the procedure more similar to a purely random search, and then gradually decreases the value of T . Towards the end of the run, the values of T are quite small so the final stages of simulated annealing merely resemble an ordinary hill-climber. In addition, we always accept new points if they are better than the current point. The procedure *simulated annealing* is given in figure 5.3 (again, we've assumed a maximization problem).

```

procedure simulated annealing
begin
   $t \leftarrow 0$ 
  initialize  $T$ 
  select a current point  $\mathbf{v}_c$  at random
  evaluate  $\mathbf{v}_c$ 
  repeat
    repeat
      select a new point  $\mathbf{v}_n$ 
        in the neighborhood of  $\mathbf{v}_c$ 
      if  $eval(\mathbf{v}_c) < eval(\mathbf{v}_n)$ 
        then  $\mathbf{v}_c \leftarrow \mathbf{v}_n$ 
      else if  $random[0, 1) < e^{\frac{eval(\mathbf{v}_n) - eval(\mathbf{v}_c)}{T}}$ 
        then  $\mathbf{v}_c \leftarrow \mathbf{v}_n$ 
    until (termination-condition)
     $T \leftarrow g(T, t)$ 
     $t \leftarrow t + 1$ 
  until (halting-criterion)
end

```

Fig. 5.3. Structure of simulated annealing

Simulated annealing — also known as Monte Carlo annealing, statistical cooling, probabilistic hill-climbing, stochastic relaxation, and probabilistic exchange algorithm — is based on an analogy taken from thermodynamics. To grow a crystal, you start by heating a row of materials to a molten state. You then reduce the temperature of this *crystal melt* until the crystal structure is *frozen in*. Bad things happen if the cooling is done too quickly. In particular, some irregularities are locked into the crystal structure and the trapped energy level is much higher than in a perfectly structured crystal.² The analogy between the physical system and an optimization problem should be evident. The basic “equivalent” concepts are listed in table 5.3.

²A similar problem occurs in metallurgy when heating and cooling metals.

Table 5.3. Analogies between a physical system and an optimization problem

Physical System	Optimization Problem
state	feasible solution
energy	evaluation function
ground state	optimal solution
rapid quenching	local search
temperature	control parameter T
careful annealing	simulated annealing

As with any search algorithm, simulated annealing requires the answers for the following problem-specific questions (see chapter 2):

- What is a solution?
- What are the neighbors of a solution?
- What is the cost of a solution?
- How do we determine the initial solution?

These answers yield the structure of the search space together with the definition of a neighborhood, the evaluation function, and the initial starting point. Note, however, that simulated annealing also requires answers for additional questions:

- How do we determine the initial “temperature” T ?
- How do we determine the cooling ratio $g(T, t)$?
- How do we determine the termination condition?
- How do we determine the halting criterion?

The temperature T must be initialized before executing the procedure. Should we start with $T = 100$, $T = 1000$, or something else? How should we choose the termination condition after which the temperature is decreased and the annealing procedure reiterates? Should we execute some number of iterations or should we instead use some other criterion? Then, how much or by what factor should the temperature be decreased? By one percent or less? And finally, when should the algorithm halt, i.e., what is the “frozen” temperature?

Most implementations of simulated annealing follow a simple sequence of steps:

```

STEP 1:  $T \leftarrow T_{max}$ 
        select  $\mathbf{v}_c$  at random

STEP 2: pick a point  $\mathbf{v}_n$  from the neighborhood of  $\mathbf{v}_c$ 
        if  $eval(\mathbf{v}_n)$  is better than  $eval(\mathbf{v}_c)$ 
            then select it ( $\mathbf{v}_c \leftarrow \mathbf{v}_n$ )
            else select it with probability  $e^{\frac{-\Delta eval}{T}}$ 
        repeat this step  $k_T$  times

STEP 3: set  $T \leftarrow rT$ 
        if  $T \geq T_{min}$ 
            then goto STEP 2
            else goto STEP 1

```

Here we have to set the values of the parameters T_{max} , k_T , r , and T_{min} , which correspond to the initial temperature, the number of iterations, the cooling ratio, and the frozen temperature, respectively. Let's examine some possibilities by applying the simulated annealing technique to the SAT, TSP, and NLP, respectively.

Spears [444] applied simulated annealing (SA) on hard satisfiability problems. The procedure SA-SAT described in [444] is displayed in figure 5.4. Its control structure is similar to that of the GSAT (chapter 3). The outermost loop variable called "tries" of SA-SAT corresponds to the variable i of GSAT (see figure 3.5 from chapter 3). These variables keep track of the number of independent attempts to solve the problem. T is set to T_{max} at the beginning of each attempt in SA-SAT (by setting the variable j to zero) and a new random truth assignment is made. The inner repeat loop tries different assignments by probabilistically flipping each of the Boolean variables. The probability of a flip depends on the improvement δ of the flip and the current temperature. As usual, if the improvement is negative, the flip is quite unlikely to be accepted and vice versa: positive improvements make accepting flips likely.

There's a major difference between GSAT and SA-SAT, and this is the essential difference between any local search technique and simulated annealing: GSAT can make a backward move (i.e., a decrease in the number of satisfied clauses) if other moves are not available. At the same time, GSAT cannot make two backward moves in a row, as one backward move implies the existence of the next improvement move! On the other hand, SA-SAT can make an arbitrary sequence of backward moves, thus it can escape local optima!

The remaining parameters of SA-SAT have the same meaning as in other implementations of simulated annealing. The parameter r represents a decay rate for the temperature, i.e., the rate at which the temperature drops from T_{max} to T_{min} . The drop is caused by incrementing j , as

$$T = T_{max} \cdot e^{-j \cdot r}.$$

Spears [444] used

```

procedure SA-SAT
begin
  tries  $\leftarrow$  0
  repeat
    v  $\leftarrow$  random truth assignment
     $j \leftarrow 0$ 
    repeat
      if v satisfies the clauses then return v
       $T = T_{max} \cdot e^{-j \cdot r}$ 
      for  $k = 1$  to the number of variables do
        begin
          compute the increase (decrease)  $\delta$  in the
            number of clauses made true if  $v_k$  was flipped
          flip variable  $v_k$  with probability  $(1 + e^{-\frac{\delta}{T}})^{-1}$ 
          v  $\leftarrow$  new assignment if the flip is made
        end
       $j \leftarrow j + 1$ 
    until  $T < T_{min}$ 
    tries  $\leftarrow$  tries + 1
  until tries = MAX-TRIES
end

```

Fig. 5.4. Structure of the SA-SAT algorithm

$$T_{max} = 0.30 \text{ and } T_{min} = 0.01$$

for the maximum and minimum temperatures, respectively. The decay rate r depended on the number of variables in the problem and the number of the “try” (r was the inverse of the product of the number of variables and the number of the try). Spears commented on the choice of these parameters:

Clearly these choices in parameters will entail certain tradeoffs. For a given setting of MAX-TRIES, reducing T_{min} and/or increasing T_{max} will allow more tries to be made per independent attempt, thus decreasing the number of times that ‘tries’ can be incremented before the MAX-TRIES cutoff is reached. A similar situation occurs if we decrease or increase the decay rate. Thus, by increasing the temperature range (or decreasing the decay rate) we reduce the number of independent attempts, but search more thoroughly during each attempt. The situation is reversed if one decreases the temperature range (or increases the decay rate). Unfortunately it is not at all clear whether it is generally better to make more independent attempts, or to search more thoroughly during each attempt.

The experimental comparison between GSAT and SA-SAT on hard satisfiability problems indicated that SA-SAT appeared to satisfy at least as many formulas

as GSAT, with less work [444]. Spears also presented experimental evidence that the relative advantage of SA-SAT came from its backward moves, which helped escape local optima.

It's interesting to note that the TSP was one of the very first problems to which simulated annealing was applied! Moreover, many new variants of simulated annealing have also been tested on the TSP. A standard simulated annealing algorithm for the TSP is essentially the same as the one displayed in figure 5.3, where $\mathbf{v}_c$ is a tour and $eval(\mathbf{v}_c)$ is the length of the tour $\mathbf{v}_c$. The differences between implementations of simulated annealing are in (1) the methods of generating the initial solution, (2) the definition of a neighborhood of a given tour, (3) the selection of a neighbor, (4) the methods for decreasing temperature, (5) the termination condition, (6) the halting condition, and (7) the existence of a postprocessing phase.

All the above decisions are important and far from straightforward. They are also not independent. For example, the number of steps at each temperature should be proportional to the neighborhood size. For each of these facets there are many possibilities. We might start from a random solution, or instead take an output from a local search algorithm as the initial tour. We can define neighborhoods of various sizes and include a postprocessing phase where a local search algorithm climbs a local peak (as simulated annealing doesn't guarantee that this will happen for all schedules for reducing T over time).

For more details on various possibilities of applying simulated annealing to the TSP, with an excellent discussion on techniques for accelerating the algorithm and other improvements, see [242].

Simulated annealing can also easily be applied to the NLP. Since we are now concerned with continuous variables, the neighborhood is often defined on the basis of a Gaussian distribution (for each variable), where the mean is kept at the current point and the standard deviation is set to one-sixth of the length of the variable's domain (so that the length from the midpoint of the domain to each boundary equals three standard deviations). Thus, if the current point is

$$\mathbf{x} = (x_1, \dots, x_n),$$

where $l_i \leq x_i \leq u_i$ for $i = 1, \dots, n$, then the neighbor $\mathbf{x}'$ of $\mathbf{x}$ is selected as

$$x'_i \leftarrow x_i + N(0, \sigma_i),$$

where $\sigma_i = (u_i - l_i)/6$ and $N(0, \sigma_i)$ is an independent random Gaussian number with mean of zero and standard deviation σ_i .

At this stage the change in the function value can be calculated:

$$\Delta eval = eval(\mathbf{x}) - eval(\mathbf{x}').$$

If $\Delta eval > 0$ (assume we face a minimization problem), the new point $\mathbf{x}'$ is accepted as a new solution; otherwise, it is accepted with probability

$$e^{\Delta eval/T}.$$

The parameter T again indicates the current temperature.

In the area of numerical optimization, the issues of generating the initial solution, defining the neighborhood of a given point, and selecting particular neighbors are straightforward. The usual procedure employs a random start and Gaussian distributions for neighborhoods. But implementations differ in the methods for decreasing temperature, the termination condition, the halting condition, and the existence of a postprocessing phase (e.g., where we might include a gradient-based method that would locate the local optimum quickly). Note that continuous domains also provide for an additional flexibility: the size of the neighborhood can decrease together with the temperature. If parameters σ_i decrease over time, the search concentrates around the current point resulting in better fine tuning.

5.2 Tabu search

The main idea behind tabu search is very simple. A “memory” forces the search to explore new areas of the search space. We can memorize some solutions that have been examined recently and these become tabu (forbidden) points to be avoided in making decisions about selecting the next solution. Note that tabu search is basically deterministic (as opposed to simulated annealing), but it’s possible to add some probabilistic elements to it [186].

The best way to explain the basic concepts of tabu search is by means of an example. We’ll start with tabu search applied to the SAT. Later we’ll provide a general discussion on some interesting aspects of this method and we’ll conclude by providing an additional example of this technique regarding the TSP.

Suppose we’re solving the SAT problem with $n = 8$ variables. Thus, for a given logical formula F we’re searching for a truth assignment for all eight variables such that the whole formula F evaluates to TRUE. Assume further that we have generated an initial assignment for $\mathbf{x} = (x_1, \dots, x_8)$; namely,

$$\mathbf{x} = (0, 1, 1, 1, 0, 0, 0, 1).$$

As usual, we need some evaluation function that provides feedback for the search. For example, we might calculate a weighted sum of a number of satisfied clauses, where the weights depend on the number of variables in the clause. In this case, the evaluation function should be maximized (i.e., we’re trying to satisfy all of the clauses), and let’s assume that the above random assignment provides the value of 27. Next, we have to examine the neighborhood of $\mathbf{x}$, which consists of eight other solutions, each of which can be obtained by flipping a single bit in the vector $\mathbf{x}$. We evaluate them and select the best one. At this stage of the search, this is the same as in a hill-climbing procedure.

Suppose that flipping the third variable generates the best evaluation (say, 31), so this new vector yields the current best solution. Now the time has come to introduce the new facet of tabu search: a memory. In order to keep a record of our actions (moves), we’ll need some memory structures for bookkeeping.

We'll remember the index of a variable that was flipped, as well as the "time" when this flip was made, so that we can differentiate between older and more recent flips. In the case of the SAT problem, we need to keep a time stamp for each position of the solution vector: the value of the time stamp will provide information on the recency of the flip at this particular position. Thus, a vector M will serve as our memory. This vector is initialized to 0 and then at any stage of the search, the entry

$$M(i) = j \text{ (when } j \neq 0\text{)}$$

might be interpreted as " j is the most recent iteration when the i -th bit was flipped" (of course, $j = 0$ implies that the i -th bit has never been flipped). It might be useful to change this interpretation so we can model an additional aspect of memory: After some period of time (i.e., after some number of iterations), the information stored in memory is erased. Assuming that any piece of information can stay in a memory for at most, say, five iterations, a new interpretation of an entry

$$M(i) = j \text{ (when } j \neq 0\text{)}$$

could be: "the i -th bit was flipped $5-j$ iterations ago." Under this interpretation, the contents of the memory structure M after one iteration in our example is given in figure 5.5. Recall that the flip on the third position gave the best result. Note that the value "5" can be interpreted as "for the next five iterations, the third bit position is not available (i.e., tabu)."

0	0	5	0	0	0	0	0
---	---	---	---	---	---	---	---

Fig. 5.5. The contents of the memory after iteration 1

It might be interesting to point out that the main difference between these two equivalent interpretations is simply a matter of implementation. The latter approach interprets the values as the number of iterations for which a given position is not available for any flips. This interpretation requires that *all* nonzero entries of the memory are decreased by one at every iteration to facilitate the process of forgetting after five iterations. On the other hand, the former interpretation simply stores the iteration number of the most recent flip at a particular position, so it requires a current iteration counter t which is compared with the memory values. In particular, if $t - M(i) > 5$, i.e., the flip on the i -th position occurred earlier than five iterations ago (if at all), it should be forgotten. This interpretation, therefore, only requires updating a *single* entry in the memory per iteration, and increasing the iteration counter. Later on we'll assume the latter interpretation in an example, but most *implementations* of tabu search use the former interpretation for efficiency.

Let's say that after four additional iterations of selecting the best neighbor — which isn't necessarily better than the current point, and this is why we can

escape from local optima — and making an appropriate flip, the memory has the contents as shown in figure 5.6.

3	0	1	5	0	4	2	0
---	---	---	---	---	---	---	---

Fig. 5.6. The contents of the memory after five iterations

The numbers present in the memory (figure 5.6) provide the following information:

Bits 2, 5, and 8 are available to be flipped any time. Bit 1 is not available for the next three iterations, bit 3 isn't available but only for the next iteration, bit 4 (which was just flipped) is not available for the next five iterations, and so on.

In other words, the most recent flip (iteration 5) took place at position 4 (i.e., $M(4) = 5$: bit 4 was flipped $5 - 5 = 0$ iterations ago). Then, the other previously flipped bits are:

bit 6 (at iteration 4), (as $M(6) = 4$),
bit 1 (at iteration 3), (as $M(1) = 3$),
bit 7 (at iteration 2), (as $M(7) = 2$) and, of course,
bit 3 (at iteration 1).

By consequence, the current solution is $\mathbf{x} = (1, 1, 0, 0, 0, 1, 1, 1)$ and let's say that it evaluates out at 33. Now, let's examine the neighborhood of $\mathbf{x}$ carefully. It consists of eight solutions,

$\mathbf{x}_1 = (0, 1, 0, 0, 0, 1, 1, 1)$,
 $\mathbf{x}_2 = (1, 0, 0, 0, 0, 1, 1, 1)$,
 $\mathbf{x}_3 = (1, 1, 1, 0, 0, 1, 1, 1)$,
 $\mathbf{x}_4 = (1, 1, 0, 1, 0, 1, 1, 1)$,
 $\mathbf{x}_5 = (1, 1, 0, 0, 1, 1, 1, 1)$,
 $\mathbf{x}_6 = (1, 1, 0, 0, 0, 0, 1, 1)$,
 $\mathbf{x}_7 = (1, 1, 0, 0, 0, 1, 0, 1)$, and
 $\mathbf{x}_8 = (1, 1, 0, 0, 0, 1, 1, 0)$,

which correspond to flips on bits 1 to 8, respectively. After evaluating each we know their respective merits, but tabu search utilizes the memory to force the search to explore new areas of the search space. The memorized flips that have been made recently are tabu (forbidden) for selecting the next solution. Thus at the next iteration (iteration 6), it's impossible to flip bits 1, 3, 4, 6, and 7, since all of these bits were tried "recently." These forbidden (tabu) solutions (i.e., the solutions that are obtained from the forbidden flips) are not considered, so the next solution is selected from a set of $\mathbf{x}_2$, $\mathbf{x}_5$, and $\mathbf{x}_8$.

Suppose that the best evaluation of these three possibilities is given by $\mathbf{x}_5$ which is 32. Note that this value represents a decrease in the evaluation between the current best solution and the new candidate. After this iteration the contents of the memory change as follows: All nonzero values are decreased by one to reflect the fact that all of the recorded flips took place one generation earlier. In particular, the value $M(3) = 1$ is changed to $M(3) = 0$; i.e., after five generations the fact that the third bit was flipped is removed from the memory. Also, since the selected new current solution resulted from flipping the fifth bit, the value of $M(5)$ is changed from zero to five (for the next five iterations, this position is tabu). Thus, after the sixth iteration the contents of memory now appear as shown in figure 5.7.

2	0	0	4	5	3	1	0
---	---	---	---	---	---	---	---

Fig. 5.7. The contents of the memory after six iterations

Further iterations are similarly executed. At any stage, there is a current solution being processed which implies a neighborhood, and from this neighborhood, tabu solutions are eliminated from possible exploration.

Upon reflection, such a policy might be too restrictive. It might happen that one of the tabu neighbors, say $\mathbf{x}_6$, provides an excellent evaluation score. One that's much better than the score of any solution considered previously. Perhaps we should make the search more flexible. If we find an outstanding solution, we might forget about the principles!³ In order to make the search more flexible, tabu search considers solutions from the *whole* neighborhood, evaluates them all, and under "normal" circumstances selects a non-tabu solution as the next current solution, whether or not this non-tabu solution has a better evaluation score than the current solution. But in circumstances that aren't "normal," i.e., an outstanding tabu solution is found in the neighborhood, such a superior solution is taken as the next point. This override of the tabu classification occurs when a so-called *aspiration criterion* is met.

Of course there are also other possibilities for increasing the flexibility of the search. For example, we could change the previous deterministic selection procedure into a probabilistic method where better solutions have an increased chance of being selected. In addition, we could change the memory horizon during the search: sometimes it might be worthwhile to remember "more," and at other times to remember "less" (e.g., when the algorithm hill-climbs a promising area of the search space). We might also connect this memory horizon to the size of the problem (e.g., remembering the last $\sqrt{n}$ moves, where n provides a size measure of the instance of the problem).

Another option is even more interesting! The memory structure discussed so far can be labeled as *recency-based* memory, as it only records some ac-

³This is quite typical in various situations in ordinary life. When seeing a great opportunity it's easy to forget about some principles!

tions of the last few iterations. This structure might be extended by a so-called *frequency-based* memory, which operates over a much longer horizon. For example (referring back to the SAT problem considered earlier), a vector H may serve as a long-term memory. This vector is initialized to 0 and at any stage of the search the entry

$$H(i) = j$$

is interpreted as “during the last h iterations of the algorithm, the i -th bit was flipped j times.” Usually, the value of the horizon h is quite large, at least in comparison with the horizon of recency-based memory. Thus after, say 100 iterations with $h = 50$, the long-term memory H might have the values displayed in figure 5.8.

5	7	11	3	9	8	1	6
---	---	----	---	---	---	---	---

Fig. 5.8. The contents of the frequency-based memory after 100 iterations (horizon $h = 50$)

These frequency counts show the distribution of moves throughout the last 50 iterations. How can we use this information? The principles of tabu search indicate that this type of memory might be useful to *diversify* the search. For example, the frequency-based memory provides information concerning which flips have been under-represented (i.e., less frequent) or not represented at all, and we can diversify the search by exploring these possibilities.

The use of long-term memory in tabu search is usually restricted to some special circumstances. For example, we might encounter a situation where all non-tabu moves lead to a worse solution. This is a special situation indeed: all legal directions lead to inferior solutions! Thus, to make a meaningful decision about which direction to explore next, it might be worthwhile to refer to the contents of the long-term memory.

There are many possibilities here for incorporating this information into the decision-making process. The most typical approach makes the most frequent moves less attractive. Usually the value of the evaluation score is decreased by some penalty measure that depends on the frequency, and the final score implies the winner.

To illustrate this point by an example, assume that the value of the current solution $\mathbf{x}$ for our SAT problem is 35. All non-tabu flips, say on bits 2, 3, and 7, provide values of 30, 33, and 31, respectively, and none of the tabu moves provides a value greater than 37 (the highest value found so far), so we can't apply the aspiration criterion. In such circumstances we refer to the frequency-based memory (see figure 5.8). Let's assume that the evaluation formula for a new solution $\mathbf{x}'$ used in such circumstances is

$$eval(\mathbf{x}') - penalty(\mathbf{x}'),$$

where *eval* returns the value of the original evaluation function (i.e., 30, 33, and 31, respectively, for solutions created by flipping the 2nd, 3rd, and 7th bit), and

$$\text{penalty}(\mathbf{x}') = 0.7 \cdot H(i),$$

where 0.7 serves as a coefficient and $H(i)$ is the value taken from the long-term memory H :

- 7 — for a solution created by flipping the 2nd bit,
- 11 — for a solution created by flipping the 3rd bit, and
- 1 — for a solution created by flipping the 7th bit.

The new scores for the three possible solutions are then

$$30 - 0.7 \cdot 7 = 25.1, \quad 33 - 0.7 \cdot 11 = 25.3, \quad \text{and} \quad 31 - 0.7 \cdot 1 = 30.3,$$

and so the third solution (i.e., flipping the 7th bit) is the one we select.

The above option of including frequency values in a penalty measure for evaluating solutions diversifies the search. Of course, many other options might be considered in connection with tabu search. For example, if we have to select a tabu move, we might use an additional rule (so-called *aspiration by default*) to select a move that is the “oldest” of all those considered. It might also be a good idea to memorize not only the set of recent moves, but also whether or not these moves generated any improvement. This information can be incorporated into search decisions (so-called *aspiration by search direction*). Further, it’s worthwhile to introduce the concept of *influence*, which measures the degree of change of the new solution either in terms of the distance between the old and new solution, or perhaps the change in the solution’s feasibility if we’re dealing with a constrained problem. The intuition connected with influence is that a particular move has a larger influence if a corresponding “larger” step was made from the old solution to the new. This information can be also incorporated into the search (so-called *aspiration by influence*). For more details on the many available options in tabu search plus some practical hints for implementation see [186].

We conclude this section by providing an additional example of possible structures used in tabu search while approaching the TSP. For this problem, we can consider moves that swap two cities in a particular solution.⁴ The following solution (for an eight-city TSP),

$$(2, 4, 7, 5, 1, 8, 3, 6),$$

has 28 neighbors, since there are 28 different pairs of cities, i.e., $\binom{8}{2} = \frac{7 \cdot 8}{2} = 28$, that we can swap. Thus, for a recency-based memory we can use the structure given in figure 5.9, where the swap of cities i and j is recorded in the i -th row and the j -th column (for $i < j$). Note that we interpret i and j as *cities*, and not as their positions in the solution vector, but this might be another possibility

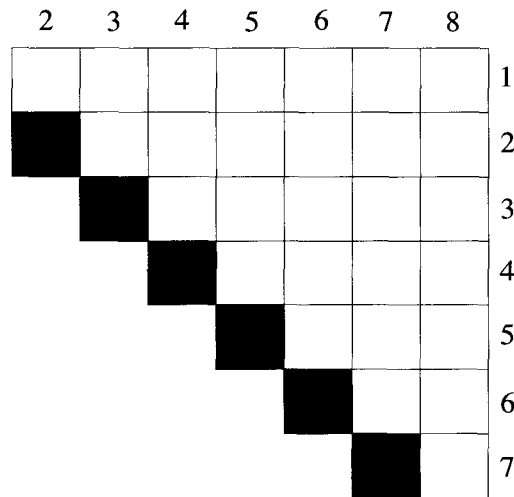


Fig. 5.9. The structure of the recency-based memory for the TSP

to consider. Note that the same structure can also be used for frequency-based memory.

For clarity, we'll maintain the number of remaining iterations for which a given swap stays on the tabu list (recency-based memory) as with the previous SAT problem, while the frequency-based memory will indicate the totals of all swaps that occurred within some horizon h . Assume both memories were initialized to zero and 500 iterations of the search have been completed. The current status of the search then might be as follows. The current solution is

$$(7, 3, 5, 6, 1, 2, 4, 8)$$

with the total length of the tour being 173. The best solution encountered during these 500 iterations yields a value of 171. The status of the recency-based and frequency-based memories are displayed in figures 5.10 and 5.11, respectively.

As with the SAT problem, it's easy to interpret the numbers in these memories. The value $M(2, 6) = 5$ indicates that the most recent swap was made for cities 2 and 6, i.e., the previous current solution was

$$(7, 3, 5, 2, 1, 6, 4, 8).$$

Therefore, swapping cities 2 and 6 is *tabu* for the next five iterations. Similarly, swaps of cities 1 and 4, 3 and 7, 4 and 5, and 5 and 8, are also on the tabu list. Out of these, the swap between cities 1 and 4 is the oldest (i.e., it happened five iterations ago) and this swap will be removed from the tabu list after the

⁴We do not make any claim that this decision is the best. For example, it's often more meaningful to swap edges of a tour.

	2	3	4	5	6	7	8	
	0	0	1	0	0	0	0	1
		0	0	0	5	0	0	2
			0	0	0	4	0	3
				3	0	0	0	4
					0	0	2	5
						0	0	6
							0	7

Fig. 5.10. The contents of the recency-based memory M for the TSP after 500 iterations. The horizon is five iterations.

	2	3	4	5	6	7	8	
	0	2	3	3	0	1	1	1
		2	1	3	1	1	0	2
			2	3	3	4	0	3
				1	1	2	1	4
					4	2	1	5
						3	1	6
							6	7

Fig. 5.11. The contents of the frequency-based memory F for the TSP after 500 iterations. The horizon is 50 iterations.

next iteration. Note that only 5 swaps (out of 28 possible swaps) are forbidden (tabu).

The frequency-based memory provides some additional statistics of the search. It seems that swapping cities 7 and 8 was the most frequent (it happened

6 times in the last 50 swaps), and there were pairs of cities (like 3 and 8) that weren't swapped within the last 50 iterations.

The neighborhood of a tour was defined by a swap operation between two cities in the tour. This neighborhood is not the best choice either for tabu search or for simulated annealing. Many researchers have selected larger neighborhoods. For example Knox [263] considered neighborhoods based on a *2-interchange* move (see figure 3.6) that's defined by deleting two nonadjacent edges from the current tour and adding two other edges to obtain a new feasible tour. The outline of a particular implementation of tabu search reported in [263] is given in figure 5.12.

```

procedure tabu search
begin
  tries  $\leftarrow$  0
  repeat
    generate a tour
    count  $\leftarrow$  0
    repeat
      identify a set  $\mathcal{T}$  of 2-interchange moves
      select the best admissible move from  $\mathcal{T}$ 
      make appropriate 2-interchange
      update tabu list and other variables
      if the new tour is the best-so-far for a given 'tries'
      then update local best tour information
      count  $\leftarrow$  count + 1
    until count = ITER
    tries  $\leftarrow$  tries + 1
    if the current tour is the best-so-far (for all 'tries')
    then update global best tour information
  until tries = MAX-TRIES
end

```

Fig. 5.12. Particular implementation of tabu search for the TSP

Knox [263] made a tour tabu if *both* added edges of the interchange were on the tabu list. The tabu list was updated by placing the added edges on the list (deleted edges were ignored). What's more, the tabu list was of fixed size. Whenever it became full, the oldest element in the list was replaced by the new deleted edge. At initialization, the list was empty and all of the elements of the aspiration list were set to large values. Note that the algorithm examines *all* neighbors, that is, all of the *2-interchange* tours.

Knox [263] indicated that the best results were achieved when

- The length of the tabu list was $3n$ (where n is the number of cities of the problem).

- A candidate tour could override the tabu status if both edges passed an aspiration test, which compared the length of the tour with aspiration values for both added edges. If the length of the tour was better (i.e., smaller) than *both* aspiration values, the test was passed.
- The values present on the aspiration list were the tour costs prior to the interchange.
- The number of searches (MAX-TRIES) and the number of interchanges (ITER) depended on the size of the problem. For problems of 100 cities or less, MAX-TRIES was 4, and ITER was set to $0.0003 \cdot n^4$.

Of course, there are many other possibilities for implementing tabu lists, aspiration criteria, generating initial tours, etc. The above example illustrates only one possible implementation of tabu search for the TSP. You might generate initial tours at random, or by some other means to ensure diversity. The maximum number of iterations might be some function of the improvement made so far. No doubt you can imagine any number of other possibilities.

5.3 Summary

Simulated annealing and tabu search were both designed for the purpose of escaping local optima. However, they differ in the methods used for achieving this goal. Tabu search usually makes uphill moves only when it is stuck in local optima, whereas simulated annealing can make uphill moves at any time. Additionally, simulated annealing is a stochastic algorithm, whereas tabu search is deterministic.

Compared to the classic algorithms offered in chapter 4, we can see that both simulated annealing and tabu search work on complete solutions (as did the techniques in chapter 3). You can halt these algorithms at any iteration and you'll have a solution at hand, but you might notice a subtle difference between these methods and the classic techniques. Simulated annealing and tabu search have more parameters to worry about, such as temperature, rate of reduction, a memory, and so forth. Whereas the classic methods were designed just to "turn the crank" to get the answer, now you have to start really thinking, not just whether or not the algorithm makes sense for your problem, but how to choose the parameters of the algorithm so that it performs optimally. This is a pervasive issue that accompanies the vast majority of algorithms that can escape local optima. The more sophisticated the method, the more you have to use your judgment as to how it should be utilized.

VI. How Good Is Your Intuition?

It's human nature to try to guess at the solution to a complex problem. Some guesses seem more intuitive than others. Even some *people* seem more intuitive than others! But they aren't, are they? They just seem that way because of the small sample size that you consider. We seem to remember the lucky person who guessed the number on the roulette wheel three times in a row, and we forget the hundreds of others who didn't perform that marvelous feat. But the lucky person isn't clairvoyant. It's all just a matter of random sampling. If you could somehow take that lucky person back to the roulette and ask them to call out the next three numbers, the probability of their success would be exactly the same as before: $(\frac{1}{38})^3$, assuming a roulette wheel with both 0 and 00.

One famous trick that gets played on people who want to make money in the stock market goes like this. A fraudulent stock broker picks 1024 people as possible clients — “suckers” might be a better term. Each day, for the next 10 days, he mails each one of these people a prediction about whether or not the stock market will increase or decrease that next day. Of course, the trick is that in 10 days, there are 2^{10} different possible outcomes — and guess what — he mails each person one of these possible outcomes, played out over 10 days.

At the end of the 10 days, one unfortunate victim will have seen this stock broker predict the market's direction correctly an amazing 10 out of 10 times! The victim's intuition must be that this guy is a psychic! Even those who received 8 or 9 correct predictions out of 10 have to be impressed with the broker's prowess. But what about the others who get about as many right guesses as wrong ones, or even the lucky person who sees the broker get 10 incorrect predictions in a row? They forget about him, of course! So now the broker fixes in on the people he gave all the correct predictions to and tries to bilk them out of some money. Watch out for your intuition and hold on to your wallet!

Sometimes, you have to try really hard to reject your intuition. Even seemingly simple straightforward everyday events demand some serious scrutiny. Try these out for size.

This first puzzle is very simple, but it illustrates that the term “average” often isn't well understood. You drive a car at a constant speed of 40 km/h from Washington D.C. to New York City and return immediately, but at a higher speed of 60 km/h. What was your average speed for the whole trip?

It's likely that at least 90 percent of all people would say that the average was 50 km/h, as the intuition of "average" is very clear. The correct answer of 48 km/h would seem very counterintuitive! Note that the "average speed" is defined as a ratio between the distance and the time, thus

$$v_{avg} = \frac{D}{t},$$

where D and t represent the total distance and the time of the whole trip, respectively. In that case $D = 2d$ (where d is the distance between Washington D.C. and New York City) and $t = t_{WN} + t_{NW}$ (where t_{WN} and t_{NW} are the times to get from Washington D.C. to New York City and from New York City to Washington D.C., respectively). Note also that

$$t_{WN} = d/40 \text{ and } t_{NW} = d/60.$$

Thus,

$$v_{avg} = \frac{D}{t} = \frac{2d}{t_{WN} + t_{NW}} = \frac{2d}{d/40 + d/60} = \frac{2}{1/40 + 1/60} = 48.$$

Here's an additional upgrade for this puzzle. Suppose that you go from Washington D.C. to New York City at a constant speed of 40 km/h. What should your constant speed be for the return trip if you want to obtain an average speed of 80 km/h?

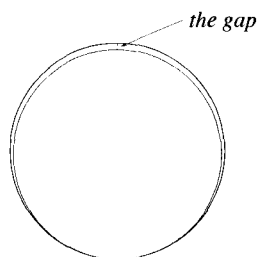


Fig. VI.1. The Earth and the ring

Here's another interesting counterintuitive result. Suppose that in the times of Ancient Greece, Zeus commissioned a blacksmith to make an iron ring that would go around the Earth, and the blacksmith was asked to make the diameter of the ring match the diameter of the Earth exactly.⁵ The poor blacksmith, however, made a mistake. He made a ring that was just one meter longer in circumference than it was supposed to be. Nevertheless, Zeus placed the ring "around" the Earth, and it was made to touch the Earth at one point (see figure VI.1). The question is, how much did the ring "stick out" on the other side of the Earth? What kind of animal would be able to squeeze under the gap between the Earth and the ring? An ant? A mouse? A rat? A cat?

The counterintuitive answer is that that gap is almost 32 cm! That's over one foot! Even a small child could squeeze under the ring! This is so because the difference in circumference between the Earth and the ring is 100 cm:

⁵Assuming here that the Earth is a perfect sphere!

$$2\pi r_1 - 2\pi r_2 = 100,$$

so the difference in their diameters ($2r_1 - 2r_2$) is $100/\pi \approx 31.83$ cm.

The third puzzle is even more tricky, and it illustrates the difficulty many people have confusing “average” with “probability.” Which event is more likely: to throw 1 six when 6 dice are rolled, or to throw 2 sixes when 12 dice are rolled? Of course, it’s possible to extend the question to “or throw 3 sixes when 18 dice are rolled,” etc. Also, we can ask a similar question. Which event is more likely: to throw *at least* 1 six when 6 dice are rolled, or to throw *at least* 2 sixes when 12 dice are rolled?

Usually our intuition fails here. It doesn’t seem like there’s that much difference because the expected values are “1 six” and “2 sixes” in these two scenarios, but the probabilities of these events are in fact very different!

In the former case, the probability of throwing precisely 1 six out of 6 dice is

$$\binom{6}{1} \cdot \frac{1}{6} \cdot \frac{5}{6} \cdot \frac{5}{6} \cdot \frac{5}{6} \cdot \frac{5}{6} \cdot \frac{5}{6},$$

as

- $\binom{6}{1}$ is the number of combinations of the 6 dice that have exactly one six showing. It may appear on the first die, the second die, etc.
- $\frac{1}{6}$ is the probability of throwing a six on any die.
- $\frac{5}{6}$ is the probability of *not* throwing a six on a specific die; this must happen on 5 remaining dice.

So, the probability of throwing precisely 1 six out of 6 dice is 0.4019.

On the other hand, the probability of throwing precisely 2 sixes out of 12 dice is

$$\binom{12}{2} \cdot \left(\frac{1}{6}\right)^2 \cdot \left(\frac{5}{6}\right)^{10} = 0.2961,$$

which is quite a difference! By the way, the probability of throwing precisely k sixes with n dice is

$$\binom{n}{k} \cdot \left(\frac{1}{6}\right)^k \cdot \left(\frac{5}{6}\right)^{n-k}.$$

OK, here’s your last chance! There are three large identical boxes. One of them contains an expensive item (e.g., a 1953 classic Corvette convertible, with a red interior and loaded with features), whereas the other two boxes contain prizes of much smaller value (e.g., a pencil and a few candies, respectively). Of course, you don’t know which box contains which items.

You’re asked to point to one of the boxes, which you do. At this time a person who runs the show and knows exactly which box contains the car, opens one of the other boxes. This one, much to your relief, doesn’t contain the car. Now you’re told to make your final pick: either you can stay with your original choice, or you can switch to the other unopened box. Whatever you do, this is

final and you get what's in the box you select. What should you do? Should you stay with the original guess or should you switch?

At first it seems that it doesn't matter. After all, the car can be in either of the unopened boxes, so what would be the point of changing the initially selected box? A simple argument should convince you, however, that changing your selection will double your chances of winning the car! When you made your initial selection, the probability of pointing to the winning box was precisely $1/3$. Thus, the probability that the car was in one of the other two boxes was $2/3$. If one of the other boxes were opened, the chance that the car is in one of the other two boxes would still be $2/3$, so the probability that the car is in the other unopened box is $2/3$. In other words, if you stay with your original choice, your chances of winning stay at $1/3$; however, if you switch to the other unopened box, the chances of winning jump to $2/3$. Amazing? You can easily verify this by taking three playing cards, one king and two small cards, mixing them, and placing them face down on a table. Select a card and play it out. Do 20 such experiments where you stick with the original choice, and 20 experiments where you switch. Of course, you'll need a friend who knows the location of the winning card (the king) and turns the other card face up. Preferably, a friend with a 1953 classic Corvette!

6. An Evolutionary Approach

The works of nature must all be accounted good.

Cicero, *De Senectute*

In the previous three chapters we discussed various classic problem-solving methods, including dynamic programming, branch and bound, and local search algorithms, as well as some modern heuristic methods like simulated annealing and tabu search. Some of these techniques were seen to be deterministic. Essentially you “turn the crank” and out pops the answer. For these methods, given a search space and an evaluation function, some would always return the same solution (e.g., dynamic programming), while others could generate different solutions based on the initial configuration or starting point (e.g., a greedy algorithm or the hill-climbing technique). Still other methods were probabilistic, incorporating random variation into the search for optimal solutions. These methods (e.g., simulated annealing) could return different final solutions even when given the same initial configuration. No two trials with these algorithms could be expected to take exactly the same course. Each trial is much like a person’s fingerprint: although there are broad similarities across fingerprints, no two are exactly alike.

There’s an interesting observation to make with all of these techniques: each relies on a single solution as the basis for future exploration with each iteration. They either process complete solutions in their entirety, or they construct the final solution from smaller building blocks. Greedy algorithms, for example, successively build solutions according to maximizing locally available improvements. Dynamic programming solves many smaller subproblems before arriving at the final complete solution. Branch and bound methods organize the search space into several subspaces, then search and eliminate some of these in a systematic manner. In contrast, local search methods, simulated annealing, and tabu search process complete solutions, and you could obtain a potential answer (although quite likely a suboptimal one) by stopping any of these methods at any particular iteration. They always have a *single* “current best” solution stored that they try to improve in the next step. Despite these differences, every one of these algorithms works on or constructs a single solution at a time.

Speaking quite generally, the approach of keeping the best solution found so far and attempting to improve it is intuitively sound, remarkably simple, and often quite efficient. We can use (1) deterministic rules — e.g., hill-climbing uses the rule: if an examined neighbor is better, proceed to that neighbor and continue searching from there; otherwise, continue searching in the current neighborhood; (2) probabilistic rules — e.g., simulated annealing uses the rule: if an

examined neighbor is better, accept this as the new current position; otherwise, either probabilistically accept this new weaker position anyway or continue to search in the current neighborhood; or even (3) the history of the search up to the current time — e.g., tabu search uses the rule: take the best available neighbor, which need not be better than the current solution, but which isn't listed in memory as a restricted or “tabu” move.

Now let's consider a revolutionary idea, one that doesn't appear in any of the methods that we discussed previously. Let's abandon the idea of processing only a single solution. What would happen if we instead maintained several solutions simultaneously? That is, what would happen if our search algorithm worked with a *population* of solutions?

At first blush, it might seem that this idea doesn't provide us with anything really new. Certainly, we can process several solutions in parallel if a parallel computer is available! If this were the case, we could implement a parallel search method, like a parallel simulated annealing or tabu search, where each processor would maintain a single solution and each processor would execute the same algorithm in parallel. It would seem that all there is to be gained here is just the speed of computation: instead of running an algorithm k times to increase the probability of arriving at the global optimum, a k -processor computer would complete this task in a single run, and therefore uses much less real time.

But there's an additional component that can make population-based algorithms essentially different from other problem-solving methods: the concept of competition between solutions in a population. That is, let's simulate the evolutionary process of competition and selection and let the candidate solutions in our population fight for room in future generations! Moreover, we can use random variation to search for new solutions in a manner similar to natural evolution.

In case you haven't heard this analogy before, let's digress with some poetic license to a somewhat whimsical example of rabbits and foxes. In any population of rabbits,¹ some are faster and smarter than others. These faster, smarter rabbits are less likely to be eaten by foxes, and therefore more of them survive to do what rabbits do best: make more rabbits! This surviving population of rabbits starts breeding. Breeding mixes the rabbits' genetic material, and the behaviors of the smarter and faster rabbits are heritable. Some slow rabbits breed with fast rabbits, some fast with fast, some smart ones with not-so-smart ones, and so on. On top of that, nature throws in a “wild hare” every once in a while because all of the genes in each rabbit have a chance of mutating along the way. The resulting baby rabbits aren't copies of their parents, but are instead random perturbations of these parents. Many of these babies will grow up to express the behaviors that help them compete even better with foxes, and other rabbits! Thus, over many generations, the rabbits may be expected to become smarter and faster than those that survived the initial generation. And don't forget that the foxes are evolving at the same time, putting added pressure on

¹An individual rabbit represents a solution for a problem. It represents a single point of the search space.

the rabbits. The interaction of the two populations would seem to push both of them to their physical limits. Foxes are forced to get better at finding a meal and rabbits get better at avoiding being someone else's lunch!

Let's re-tell the above story in terms of search spaces, evaluation functions, and potential solutions. Suppose we start with a population of initial solutions, perhaps generated by sampling randomly from the search space $\mathcal{S}$. Sometimes, to emphasize a link with genetics, these solutions are called *chromosomes*, but there is no need to think of biological terms.² The algorithms that we will explore are just that — algorithms — mathematical procedures based on analogies to natural evolutionary processes, and there is no need for our so-called chromosomes to behave in any way like nature's chromosomes. We may simply describe the candidate solutions as vectors, or whatever representation and data structure we select.

The evaluation function can be used to determine the relative merit of each of our initial solutions. This isn't always straightforward because multiple objectives need to be combined in a useful way. For example, which rabbit is better, the one that is fast and dumb, or slow but smart? Natural selection makes this decision by using a quality control method that might be called "trial by foxes." The proof of which rabbit is better able to meet the demands imposed by the environment rests in which ones can survive encounters with the foxes, or evade them all together. But whereas nature doesn't have a mathematical function to judge the quality of an individual's behavior, we often have to determine such a function. At least, the chosen evaluation function must be capable of differentiating between two individuals, i.e., it has to be able to rank one solution ahead of another. Those solutions that are better, as determined by the evaluation function, are favored to become parents for the next generation of offspring.

How should we generate these offspring? Well, actually we've seen the basic concept in most other algorithms: new solutions are generated probabilistically in the neighborhood of old solutions. When applying an evolutionary algorithm, however, we don't have to rely only on the neighborhoods of each individual solution. We can also examine the neighborhoods of pairs of solutions. That is, we can use more than one parent solution to generate a new candidate solution. One way we can do this is by taking parts of two parents and putting them together to form an offspring. For example, we might take the first half of one parent together with the second half of another. The first half of one parent might be viewed as an "idea," and similarly so with the second half of the second parent. Sometimes this recombination can be very useful: You put cookie dough together with ice cream and you get a delicious treat. But sometimes things don't work out so well: You put the first half of Shakespeare's *Hamlet* together with the second half of *King Lear* and you still get Shakespeare but it just doesn't make any sense any more. Another way of using two solutions to generate a new possibility occurs when facing continuous optimization problems. In these cases we can blend parameters of both parents, essentially performing

²In fact, the use of biological terms may offer the illusion that there is more of direct connection to natural evolution than can be justified in a particular example.

a weighted average component by component. Again, sometimes this can be advantageous, and others times it can be a waste of time. The appropriateness of every search operator depends on the problem at hand.

Since our evolutionary algorithms only exist in a computer, we don't have to rely on real biological constraints. For example, in the vast majority of sexual organisms (not including bacteria), mating occurs only between pairs of individuals, but our evolutionary searches can rely on "mating" or "blending" more than two parents, perhaps using *all* of the individuals in the population of each generation to determine the probabilities for selecting each new candidate solution.

With each generation, the individuals compete — either only among themselves or also against their parents — for inclusion in the next iteration. It's very often the case that after a series of generations, we observe a succession of improvements in the quality of the tested solution and a convergence toward the neighborhood of a nearly optimum solution.

Without any doubt, this is an appealing idea! Why should we labor to solve a problem by calculating difficult mathematical expressions or developing complicated computer programs to address approximate models of a problem if we can simulate the essential and basic process of evolution and discover nearly optimal solutions using models of much greater fidelity, particularly when the difficult part of the search essentially comes "for free." Things that are free are usually too good to be true, so it is appropriate to ask some probing questions. What kind of effort is expected from a user of an evolutionary algorithm? In other words, how much work does it take to implement these concepts in an algorithm? Is this cost effective? And can we *really* solve real-world problems using such an evolutionary approach? We'll answer these questions in the remainder of the book.

In the following sections we revisit the three problems introduced in chapter 1: the satisfiability problem (SAT), the traveling salesman problem (TSP), and a nonlinear programming problem (NLP), and we demonstrate some possibilities for constructing evolutionary algorithms to address each one.

6.1 Evolutionary approach for the SAT

Suppose that we wanted to use an evolutionary algorithm to generate solutions for an SAT of 100 variables. Let's say that the problem is essentially like that of chapter 1, section 1.1, where we have a compound Boolean statement in the form

$$F(\mathbf{x}) = (x_{17} \vee \bar{x}_{37} \vee x_{73}) \wedge (\bar{x}_{11} \vee \bar{x}_{56}) \wedge \dots \wedge (x_2 \vee x_{43} \vee \bar{x}_{77} \vee \bar{x}_{89} \vee \bar{x}_{97}),$$

and we need to find a vector of truth assignments for all 100 variables x_i , $i = 1, \dots, 100$, such that $F(\mathbf{x}) = 1$ (TRUE). Since we are dealing with variables that can only take on two states, TRUE or FALSE, a natural representation to choose is a binary string (vector) of length 100. Each component in the string

signifies the truth assignment for the variable which corresponds to that location in the string. For example, the string $\langle 1010\dots 10 \rangle$ would assign TRUE to all odd variables, $x_1, x_3, \dots, x_{99}$, and FALSE to all even variables, $x_2, x_4, \dots, x_{100}$, under the coding of 1 being TRUE and 0 being FALSE. Any binary string of 100 variables would constitute a possible solution to the problem. We could use $F(\mathbf{x})$ to check any such vector and determine whether it satisfied the necessary conditions to make $F(\mathbf{x}) = 1$.

We need an initial population to start this approach, so our next consideration is the population size. How many individuals do we want to have? Let's say that we want 30 parent vectors. One way to determine this collection would be to have a 50–50 chance of setting each bit in each vector to be either 1 or 0. This would, hopefully, give us a diverse set of parents. Typically, this diversity is desirable, although in some cases we might want to initialize all of the available parents to some best-known solution and proceed from there. For the sake of our example, however, let's say that we use this randomized procedure to determine the initial population. Note immediately, then, that every trial of our evolutionary algorithm will have a potentially different initial population based on the random sampling that we generate.

Suppose that we have the initial randomized population. What we need to do next is evaluate each vector to see if we have discovered the optimum solution and if not, we need to know the quality of the solutions we have at hand; that is, we need an evaluation function. This presents a difficulty. It is straightforward to use $F(\mathbf{x})$ to check if any of our initial vectors are perfect solutions, but let's suppose that none of these vectors turns out to be perfect; i.e., for each randomly selected initial vector, $\mathbf{x}$, $F(\mathbf{x}) = 0$. Now what? This doesn't provide any useful information for determining how close a particular vector might be to the perfect solution. If we use $F(\mathbf{x})$ as the evaluation function, and if the perfect solution were $\langle 11\dots 11 \rangle$ and two solutions in the initial population were $\langle 00\dots 00 \rangle$ and $\langle 11\dots 10 \rangle$, these latter solutions would both receive an evaluation of 0, but the first vector has 100 incorrectly set variables while the second is only one bit away from perfection! Clearly $F(\mathbf{x})$ does not provide the information we need to guide a search for successively improved strings.

This is a case where the objective is not suggestive of a suitable evaluation function. The objective of making the compound Boolean statement $F(\mathbf{x}) = 1$ is insufficient. What *is* sufficient? At first, we might be tempted to simply compare the number of bits that differ between each candidate solution and the perfect solution and use this as an error function. The more bits that are incorrectly specified, the worse the error, and then we could seek to minimize the error. The trouble with this is that it presumes we already know the perfect vector in order to make the comparison, but if we knew the perfect vector, we wouldn't need to search for a solution! Instead, what we might do is consider each part of the compound Boolean expression $F(\mathbf{x})$ that is separated by an “and” symbol ($\wedge$), e.g., $(x_{17} \vee \bar{x}_{37} \vee x_{73})$, and count up how many of these separate subexpressions are TRUE. We could then use the number of TRUE subexpressions as a evaluation function, with the goal of maximizing the number of TRUE subexpressions,

because, given $F(\mathbf{x})$ as it appears above, if all of the subexpressions are TRUE then it follows that $F(\mathbf{x}) = 1$, just as we desire. Do we know that this evaluation function will lead us to the perfect solution? Unfortunately, we don't. All we can say before some experimentation is that it appears to be a reasonable choice for the evaluation function.

Let's presume that we use this function to judge the quality of our 30 initial vectors, and let's further presume that there are 20 subexpressions that comprise $F(\mathbf{x})$; therefore, the best score we can get is 20 and the worst is 0. Suppose that the best vector in the population after this initial scoring has a score of 10, the worst has a score of 1, and the average has a score somewhere between 4 and 5. What we might do next is think about how we want to generate the individuals to survive as parents for the subsequent generation. That is, we need to apply a selection function.

One method for doing this assigns a probability of being selected to each individual in proportion to their relative fitness. That is, a solution with a score of 10 is 10 times more likely to be chosen than a solution with a score of 1. This *proportional selection* is also sometimes called *roulette wheel selection* because a common method for accomplishing this procedure is to think of a roulette wheel being spun once for each available slot in the next population, where each solution has a slice of the roulette wheel allocated in proportion to their fitness score (see figure 6.1). If we wanted to keep the population at a size of 30, we would allocate 30 slices to the wheel, one for each existing individual, and we would then spin the wheel 30 times, once for each available slot in the next population. Note, then, that we might obtain multiple copies of some solutions that happened to be chosen more than once in the 30 spins, and some solutions very likely wouldn't be selected at all. In fact, even the best solution in the current population might not be selected, just by random chance. The roulette wheel merely biases the selection toward better solutions, but just like spinning the roulette wheel in a casino, there are no guarantees!

Suppose that we have spun the wheel 30 times and have chosen our 30 candidates for the next generation. Perhaps the best solution is now represented three times, and the worst solution failed to be selected even once. It has now disappeared from consideration. Regardless of the resulting distribution of our solutions, we haven't generated anything new, we have only modified the distribution of our previous solutions. Next we need to use random variation to search for new solutions.

One possibility for doing this would be to randomly include the chance that some of our new 30 candidates will recombine. For each individual we might have a probability of, say, 0.9 that it will not pass unchanged into the next generation, but instead will be broken apart and reattached with another candidate solution chosen completely at random from the remaining 29 other solutions. This two-parent operator would essentially be a cut-and-splice procedure designed, hopefully, to ensure that successive components in candidate strings that tend to generate better solutions have a chance of staying together. Whether or not this would be effective partly depends on the degree of independence of the com-

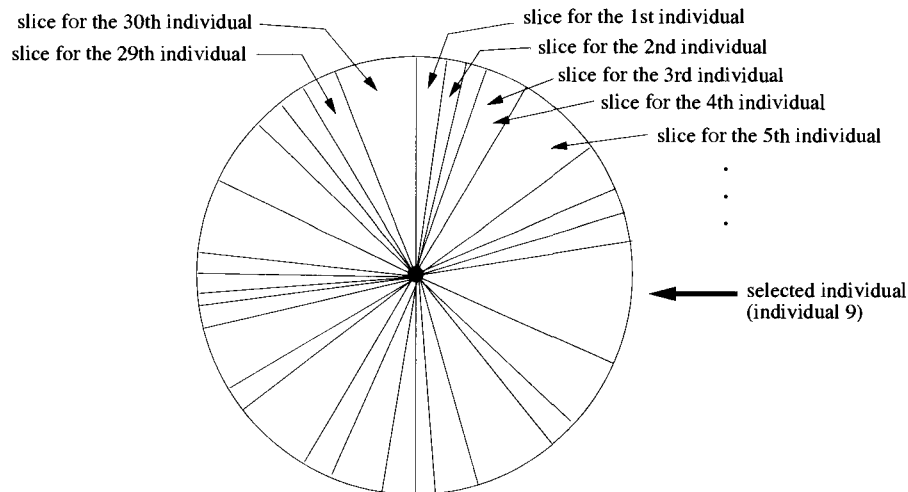


Fig. 6.1. A roulette wheel with 30 slices. The size of each slice corresponds to the fitness of the appropriate individual. From the graphic here, individual 9 had the highest fitness.

ponents. If the components of the candidate vectors aren't independent in light of the evaluation function, this recombination operator might end up metaphorically cutting and splicing *Hamlet* and *King Lear*. Another potential difficulty is that this recombination operator can only generate new solutions for evaluation if there is sufficient diversity in the current population. If we are unlucky and the roulette wheel happened to generate a homogeneous population, we'd be completely stuck, even if our current best solution wasn't a local optimum! As some insurance, we might also include another operator that flips a single bit in a chosen string at random, and then apply this with a low probability. In this way, we'd ensure that we always have the potential to introduce novel variations.

These five steps of specifying (1) the representation, (2) the initial population, (3) the evaluation function, (4) the selection procedure, and (5) the random variation operators define the essential aspects of one possible evolutionary approach to the SAT. By iterating over successive generations of evaluation, selection, and variation, our hope is that the population will discover increasingly appropriate vectors that satisfy more and more of the subexpressions of $F(\mathbf{x})$, and ultimately satisfy the condition that $F(\mathbf{x}) = 1$.

6.2 Evolutionary approach for the TSP

Suppose that we wanted to use an evolutionary algorithm to generate solutions for a TSP of 100 cities. To make things easier, let's say that it's possible to

travel from any city to any other city, and that the distances between the cities are fixed, with the objective of minimizing the distance that the salesman has to travel while visiting each city once and only once, and then returning to his home base.

Our first consideration is again the representation of a possible solution. We have several possible choices but the natural one that comes immediately to mind is to use an ordered list of the cities to be visited. For example, the vector

$$[17, 1, 43, 4, 6, 79, \dots, 100, 2]$$

would mean that the salesman starts at city 17, then goes to city 1, then on to cities 43, 4, 6, 79, and so on through all of the cities until they pass through city 100, and lastly city 2. They still have to get home to city 17, but we don't need any special modification to our representation to handle that. Once we move the salesman to the last city in the list, it's implied that there is one more city to travel to, namely, the first city in the list. It's easy to see that any permutation of this list of cities generates a new candidate solution that meets the constraints of visiting each city one time and returning home.

To begin, we have to initialize the population, so let's again figure that we have 30 candidate tours. We can generate 30 random permutations of the integers $[1..100]$ to serve as the starting population. If we had some hints about good solutions, maybe from some salesmen who have traveled these routes before, we could include these, but let's say for the sake of example that we don't have any such prior information.

Once we have generated our 30 candidate tours, we have to evaluate their quality. In this case, the objective of minimizing the total distance of the tour suggests using the evaluation function that assigns a score to each tour that is equivalent to its total length. The longer the tour, the worse the score, and our goal then is to find the tour that minimizes the evaluation function. This part of the evolutionary approach appears much more straightforward for the TSP than for the SAT.

Let's say that we use the total distance to assess our 30 initial tours. We are now left with the tasks of applying selection and random variation, but we don't have to apply them in this order. As an alternative to the example for the SAT, suppose that we decide to generate new solutions first, evaluate them, and then apply selection.

Our first decision then concerns how to generate new solutions from the existing candidate tours. It's always nice to take advantage of knowledge about the problem we face [309], and this is one case where we can do this. Our evaluation function is based on distances in Euclidean space, so consider this question: Which is longer, the sum of the lengths of the diagonals of quadrilateral, or the sum of the lengths of either pair of opposite sides? Figure 6.2 illustrates the case. By Euclidean geometry, the sum of the diagonals (e and f) are always longer than the sum of the opposing sides (a and c or b and d), and by consequence this means that tours that cross their own path are longer than those

that don't. So perhaps we can devise a variation operator that would remove instances where a parent tour crosses its own path.

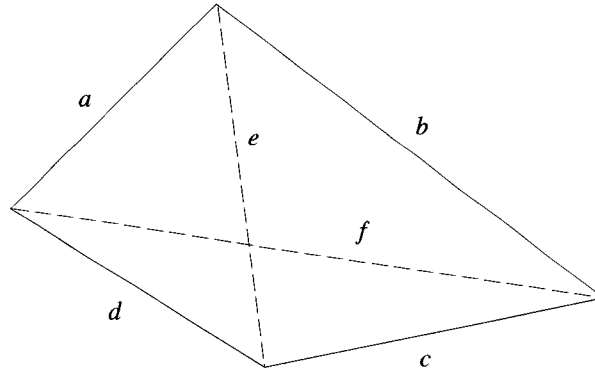


Fig. 6.2. A quadrilateral. Note that $a + c < e + f$ and $b + d < e + f$.

Indeed, this is very easily done for the representation we have chosen. For any ordered list of cities, if we pick two cities along the list and reverse all of the cities between the starting and the ending point, we have the ability to repair a tour that crosses its own path. Of course, if we apply this operator to solutions that don't cross their own paths, then we may introduce the same problems! But we can't know beforehand which solutions will be crossing over on themselves and which won't, so one reasonable approach is to simply pick two cities on the list at random and generate the new solution. If the variation isn't successful, we'll rely on selection to eliminate it from further consideration.

Like the example of the SAT before, we might think about using two or more tours to generate a new solution, but this would require some careful consideration. It's plain that we couldn't just take the first part of one solution and splice on the second part of another. In that case we would very likely generate a solution that had multiple instances of some cities and no instances of others (figure 6.3). It would violate the constraints of the problem that demand we visit every city once and only once. We could in turn simply eliminate these sorts of solutions, essentially giving them an error score of infinity, but they would be generated so frequently that we'd spend most of our time in vain, discovering infeasible solutions and then rejecting them. Under our chosen representation, we'd have to be fairly inventive to construct variation operators that would rely on two parents and generate feasible solutions.

For example, we might take two parents and begin at the first location in each tour by randomly selecting the first city in either the first parent or the second parent to be the first city in the offspring's tour list. Then we could proceed to the second position in both parents, again choose randomly from either parent, and continue on down the list. We'd have to ensure at each step that we didn't copy a city that was already copied in a previous step. To do

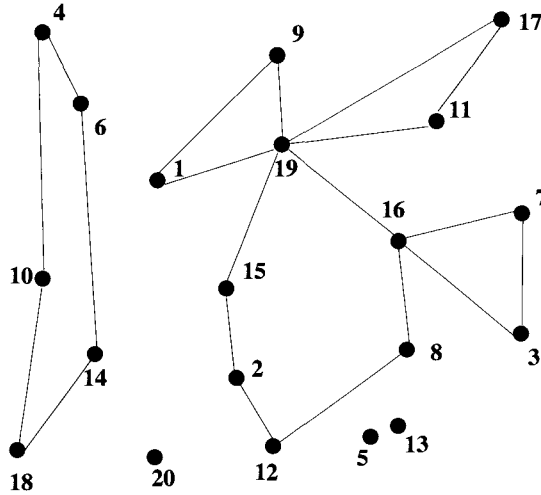


Fig. 6.3. An illegal solution for the TSP. The depicted routing is infeasible because it fails to meet the constraints of visiting each city once and only once while completing a circuit.

that, we could instead simply choose to copy from the other parent, or in the case where the cities in both parents at a particular location had already been placed in the list, we could simply choose a new city at random such that we didn't violate the constraints of visiting each city once and only once.

Suppose that instead of generating a number of offspring that is equal to the number of parents, we decide to generate three offspring per parent. We can choose any number we believe to be appropriate. For each of the 30 parents, three times in succession, we pick two cities at random in the parent, flip the list of cities between these points, and enter this new solution in the population. When we are through, we have 90 offspring tours.

We now have a choice: we can put the 90 offspring in competition with the 30 parents and select new parents from among all $30 + 90 = 120$ solutions, or we can discard the parents and just focus on the offspring. Although discarding the parents might not seem prudent, there are some supposed justifications that can be offered for this choice, but these will have to wait until chapter 7. For now, let's say that we will focus on the 90 offspring, and from these we will keep the best 30 to be parents for the next generation. This form of selection is described as (μ, λ) where there are μ parents and λ offspring, and then selection is applied to maintain the best μ solutions considering only the offspring. Here, we have a $(30, 90)$ procedure.

We could execute this process of random variation by using the two-city reversal to each parent, or the modified two-parent operator, generating three offspring per parent, and then applying selection to cull out the 60 worst new

solutions. Over time, our expectation is that the population should converge toward nearly optimal tours.

6.3 Evolutionary approach for the NLP

Suppose we wanted to solve the NLP given in chapter 1 (figure 1.2) for the case of 10 variables. Given the above descriptions of possible ways to address the SAT and TSP, the NLP should be pretty straightforward. After all, the first thing to choose is a representation, and here the most intuitive choice is a vector of 10 floating-point values, each corresponding to a parameter in the problem. When we think about initializing the population, we have the constraints to worry about, but even then we could just choose values for the parameters uniformly at random from $[0,10]$ and then check to be certain that we didn't violate the problem constraints. If we did, we could just discard that initial vector and generate a new one until we filled the initial population. The volume in $[0,10]^{10}$ that violates the constraints is relatively small so this shouldn't waste too much computational time.

Next we'll need to generate offspring. For floating-point vectors, a common variation operator is to add a Gaussian³ random variable to each parameter. You will recall that a Gaussian density function looks like a bell curve (figure 6.4). To define a Gaussian random variable we need a mean and a variance. For our example here, we choose the mean to be 0 and the variance to be 1 (i.e., a standard Gaussian). We can prove later that there are better choices, but this will suffice for now. To generate an offspring, we add a different Gaussian random variable to each component of a parent, making 10 changes, each one independent of the previous one. When we are done, we have an offspring that on average looks a lot like its parent, and the probability of it looking radically different in any parameter drops quite quickly because most of the probability density in a Gaussian lies within one standard deviation of the mean. This is often a desirable feature because we need to be able to maintain a link between parents and offspring. If we set the variance too high, the link can be broken, resulting in essentially a purely random search for new points. This is to be avoided because it fails to take into account what has already been learned over successive generations of evolution. The Gaussian variation also has the advantage of being able to generate any possible new solution, so the evolutionary process will never stall indefinitely at a suboptimum. Given a sufficiently long waiting time, all points in the space will be explored.

Once all of the offspring are generated, we have to evaluate them. Here again, as with the TSP, the objective suggests an evaluation function. We could just use the mathematical expression to be maximized as the evaluation function directly except that we also have constraints to deal with. And while the expression for the objective is defined over all the reals in each dimension, the feasible region is a small finite subset of the entire real space. One possibility in

³Also known as a *normal*.

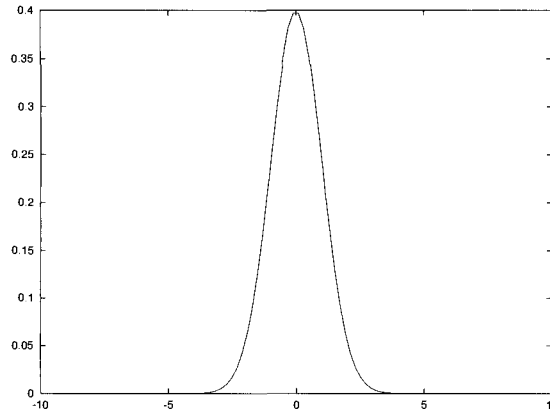


Fig. 6.4. A normal distribution $f(x) = \frac{1}{\sigma\sqrt{2\pi}}e^{-(x-m)^2/(2\sigma^2)}$ with standard deviation $\sigma = 1$ and mean $m = 0$

such cases is to apply a penalty function where the score an individual receives is the value generated by plugging it into the mathematical expression of the objective. This objective is also modified by a penalty for the degree to which it violates the applicable constraints. The further the individual moves into an infeasible region, the greater the penalty. Scaling the penalty function appropriately is a matter of judgment and experience, but for now we might just say that the evaluation function is the mathematical expression of the objective minus the distance from the individual to a boundary of the feasible region (if the individual is infeasible). The goal then is to maximize this sum.⁴

For selection, we could apply the forms we saw in the SAT or TSP, but we can introduce a third choice here: ranking selection using a tournament. Suppose that we started with 30 individuals and generated 30 more through our Gaussian variation (one offspring per parent). We'd then have a collection of 60 vectors. Suppose further that we'll keep the population at the start of each generation constant at 30. To choose these 30 new parents, one tournament method might be to pick pairs of solutions at random and determine the better one, which is placed in the next generation. After 30 such pairings we have a new population where those individuals that are better on average have a greater chance of surviving, but are not completely ensured of passing on into the next generation.

Over successive iterations of variation, evaluation (with penalties), and this form of tournament selection, we could expect to evolve an individual that meets the constraints and resides close to the global optimum. There are a number of ways that might accelerate the rate at which good solutions could be discovered

⁴Alternatively, for a minimization problem, the penalty would be added rather than subtracted.

by changing parameters of the evolutionary operators, but these can be deferred to later chapters.

6.4 Summary

The structure of an evolutionary algorithm is shown in figure 6.5.

```

procedure evolutionary algorithm
begin
   $t \leftarrow 0$ 
  initialize  $P(t)$ 
  evaluate  $P(t)$ 
  while (not termination-condition) do
    begin
       $t \leftarrow t + 1$ 
      select  $P(t)$  from  $P(t - 1)$ 
      alter  $P(t)$ 
      evaluate  $P(t)$ 
    end
  end

```

Fig. 6.5. The structure of an evolutionary algorithm

The evolutionary algorithm maintains a population of individuals, $P(t) = \{x_1^t, \dots, x_n^t\}$ for iteration t . Each individual represents a potential solution to the problem at hand and, in any evolutionary algorithm, is implemented as some data structure S . Each solution x_i^t is evaluated to give some measure of its “fitness.” A new population at iteration $t+1$ is then formed by selecting the more fit individuals (the “select” step). Some members of the new population undergo transformations (the “alter” step) by means of variation operators to form new solutions. There are unary transformations $u_i : S \rightarrow S$, that create new individuals by a change in a single individual ($m_i : S \rightarrow S$), and higher-order transformations c_j that create new individuals by combining parts from several (two or more) individuals ($c_j : S \times \dots \times S \rightarrow S$). The algorithm is executed until a predefined halting criterion is reached, which might include finding a suitable solution, generating a specified number of candidate solutions, or simply running out of available time.

The history of applying these sorts of algorithms dates back into the early 1950s and was actually invented as many as ten times by different scientists, completely independent of each other [149]. Each procedure was slightly different from the others. Some of the researchers in the 1960s gave their algorithms different names like *genetic algorithms*, *evolution strategies*, or *evolutionary programming*, but not everyone chose to christen their inventions. Over time, more specialized techniques developed that were associated with particular data structures, such as *classifier systems* and *genetic programming*. In the

1990s, however, there has been considerable theoretical and empirical evidence that none of these canonical approaches can offer anything that another approach cannot achieve. What's more, as interest in evolutionary algorithms has rapidly increased, ideas have been borrowed, exchanged, and modified across all of these approaches. As a result, there is no longer any scientific basis for discriminating between evolutionary approaches. In light of the current state-of-the-art, we'll adopt the term *evolutionary algorithm* to describe any of the algorithms that use population-based random variation and selection throughout the remainder of the text, and we encourage you to think in these broad terms as well.

Let's summarize some of the ideas discussed in earlier sections of this chapter and point out a few additional possibilities. To begin, we want to emphasize that evolutionary algorithms may incorporate any representation that appears suitable for the task at hand, whether it's a binary string as suggested in the SAT problem, a permutation of integers as offered for the TSP, or a vector of floating-point numbers as we saw in the NLP. Of course, we're not limited to these representations. We could use matrices, graphs, finite state machines, and other more complex data structures to represent potential solutions. In particular, individuals may represent a set of rules (having a variable-length structure), a blueprint of some complex design, or even a computer program. Indeed, the possibilities are limitless!

Note, however, that the choice of variation operators, which are responsible for changing parents into offspring, depends strongly on the chosen representation. We have already seen (section 6.1) a flip-mutation, which flips the value of a bit, and the so-called "1-point crossover," two operators that have commonly been applied to binary representations. We have also looked (section 6.2) at a swap or reversal operator for integer representations together with special crossover that builds a permutation of integer numbers out of two distinct permutations. We also described (section 6.3) a Gaussian mutation for floating-point representations.

Despite the many differences in approach to these problems, the implementation of an evolutionary algorithm was seen to be fairly easy. Whether or not the problem was an SAT, TSP, or NLP, the recipe was essentially the same: (1) create a population of individuals that represent potential solutions to the problem at hand, (2) evaluate them, (3) introduce some selective pressure to promote better individuals or eliminate those of lesser quality, and (4) apply some variation operators that generate new solutions to be tested. Then repeat the evolutionary loop of evaluation, selection, and variation (steps 2–4) several times.⁵

There are a few other interesting observations to make. We won't provide a detailed analysis here, but rather a few remarks. First, the broadly encompassing approach of the evolutionary technique should be apparent. For example, we can argue that simulated annealing is a special case of an evolutionary algo-

⁵Alternatively, the loop can proceed with variation, selection, and evaluation, depending on the particular implementations.

rithm, where the population size is limited to a single individual, the variation operator relies on a single parent, and the selection procedure is based on an extrinsic parameter called *temperature*. In chapter 10 we'll discuss evolutionary algorithms that change the scope or rates of their mutation operators, and this connection between simulated annealing and evolutionary algorithms will be made more directly.

In contrast to annealing, tabu search is based on very different concepts, but even there, it's possible to extend an evolutionary algorithm using the concept of memory and then tabu search becomes a special case. It's relatively easy to imagine adding either a memory unit to each individual (e.g., in terms of adding additional parameters to the represented structure and some interpretation rule that would operate on these additional parameters to guide the search; see also chapter 11), or a global memory (like a library of past ideas) for the whole population (e.g., cultural algorithms [388]). Evolutionary algorithms can in fact incorporate the main concepts underlying tabu search quite naturally.

In chapter 3, we argued that one particular classification of optimization algorithms can be based on how an algorithm "perceives" the search space. One class of algorithms (e.g., greedy algorithms, dynamic programming, branch and bound, A^*) evaluates subspaces, rather than individual solutions. Algorithms in the other class (e.g., hill-climbers, simulated annealing, tabu search) consider the whole search space as a uniform set of potential solutions. Only single complete solutions are evaluated and there is no concept of a subspace.⁶ On the other hand, evolutionary algorithms may combine these two categories by allowing individuals to describe subspaces as well as particular solutions.

There are also various additional possibilities to explore. Many of these are discussed in length later in this text, but we will mention some of them right here to give you a better perspective on this exciting field!

Most algorithms require a setup of some parameters (e.g., to apply simulated annealing, you have to fix the starting temperature, cooling rate, the maximum number of iterations per given temperature, etc.). Similarly, evolutionary algorithms require a few parameters as well. These parameters include the population size, probabilities (rates) of various operators, a parameter to regulate the selective pressure of the system, penalty coefficients (if the penalty function approach is used for solving a constrained optimization problem), and others such as the mutation step size, the number of crossover points, etc. The values of such parameters may determine whether or not the algorithm will find a near-optimum solution and, what's also important, whether or not it will find a reasonable solution efficiently. Choosing effective parameter values, however, is a time-consuming task and considerable effort has gone into developing good heuristics for these choices across many problems.

Recall, though, that evolutionary algorithms implement the idea of evolution, and that evolution itself must have evolved to reach its current state of sophistication. It is thus natural to expect adaptation to be used not only for finding solutions to a problem, but also for tuning the algorithm to the par-

⁶Except of course a neighborhood, which constitutes a subspace.

ticular problem. Technically speaking, this amounts to modifying the values of parameters during the execution of the algorithm taking the actual search process into account. As discussed later in the text, the issue of controlling the values of various parameters of an evolutionary algorithm has the potential of tuning the algorithm to the problem while solving the problem!

Further, many real-world problems require multiple solutions. This is common when there are multiple objectives to meet and it is difficult to aggregate them with a normalizing function. It would be great if we could have an algorithm that returned more than one good solution! Having a few alternatives to choose from is often very useful!

Evolutionary algorithms can be adapted easily to meet this new challenge. It's possible to utilize feedback on the diversity of the population, and in particular, on the presence of other individuals in a neighborhood of a chosen individual. Such feedback, duly incorporated into the evaluation function, can be used to control the number of returned solutions.

In chapter 1, we mentioned that many problems require time-adaptive solutions. We often get a new piece of information that contributes to the evaluation function while solving the problem. This might occur, for example, while traversing the best tour for the TSP. Perhaps we discover that, due to unforeseen circumstances, a particular segment of the tour is no longer available. Or while scheduling a factory, we learn that one of the machines has broken down. These unexpected events occur all the time in the real world, and they demand quick adaptation of current solutions to meet the new challenges.

A vast majority of classic optimization algorithms assume a fixed evaluation function. Any change in the evaluation function requires restarting the algorithm from scratch! In contrast, evolutionary algorithms are inherently adaptive; individuals in the population *adapt* to the current environment, and the quality of that adaptation is measured by the evaluation function. Consequently, an unexpected change in the model, or even in the evaluation function itself, can cause a short-lived disturbance of the evolutionary process, but after some period of time, which varies by problem, individuals in the population can adjust to the requirements of the new setting. Evolutionary techniques don't require restarting every time something changes. Instead, they continue their adaptation in light of similarities between previous experience and current demands. This illustrates a great potential of evolutionary algorithms.

Another advantage of the evolutionary approach to problem solving is that it's often easy to hybridize evolutionary algorithms with other methods. Special variation and selection operators that do the work of hill climbing or other methods can be incorporated into the process directly.

Here's another possible advantage. Recall our discussion from chapter 1, section 1.3 concerning the owner of a supermarket chain who needs to decide where to place a new store, while at the same time, his competitors are making their decisions as well. Of course, they are trying to maximize their profits, in turn quite likely minimizing the profits of the owner of the supermarket chain. Thus, the proper decision making process is very complex (he thinks that they

think that he thinks that they think...), but we can model this situation by running two opposing evolutionary processes. The first evolutionary algorithm optimizes the strategy for the supermarket chain's owner, while the other evolutionary algorithm handles the opponent's strategies. Note that the evaluation of each side's strategy is on the basis of the current (or perhaps projected) strategy of the opponent. This type of evolutionary competition is called a *co-evolutionary model*. These sorts of models can often be used to discover solutions to problems that appear too unwieldy to describe in precise mathematics due to the joint relationship of the players.

Another item should be mentioned while discussing advantages of evolutionary algorithms: parallelism. Parallelism promises to put within our reach solutions to problems that were heretofore intractable. Evolutionary algorithms are very suitable for parallel implementations as they are explicitly parallel "in nature"! It's relatively straightforward to assign a processor to each individual in the population, or to split a population into several subpopulations (the latter can be enhanced further by adding the concept of migration of individuals from one subpopulation to another).

While listing potential advantages of evolutionary algorithms, it's worthwhile to mention the final "argument," which is related more to the psychology of programming than the efficiency of the approach. When running an artificial evolutionary process, you are playing the role of *creator*, and you determine the rules of the simulation, much like determining the physics of the real world. You may choose to pay close attention to specific mechanisms in natural evolution, or perhaps consider things not found in biological systems (e.g., mating more than two parents). You can even violate the laws of nature completely and introduce *Lamarckian evolution*, where individuals acquire characteristics during their "lifetime" and then pass those characteristics along to their offspring. The barrier between the *phenotype* (an individual's expressed behavior) and the *genotype* (an individual's genetic composition) doesn't have to exist for you!

You might introduce the concept of gender among the individuals, assign them an age (some number of generations), and then use this to determine how long they might survive. You might include a memory in an individual, or explore the concept of families of individuals and social learning. You could even put entire populations in competition with each other and perhaps migrate solutions between populations. You might explore including the *Baldwin effect* in which individuals learn during their lives and in doing so can affect their fitness and alter the course of evolution. All of these avenues lead to other choices. For example, which solutions should migrate: the best, the average, the worst? Indeed, the possibilities open to you as a programmer and problem solver are virtually unlimited!